nabia

Methodology, implementation and user guide

A C++ numerical library, its Python bindings, and sentiment-modulated risk budgeting

nabia documentation

October 2026

Abstract

nabia is a platform for researching systematic, risk-budgeted portfolios. This document is its single reference. It starts with a user guide (building, installing and running every part), then describes the architecture, every numerical method of the C++ library stated mathematically with proofs where a property is relied on, the full derivations behind the forecasters, the optimizers and the elastic net, the command-line application, the Python bindings and how they are packaged as wheels for every platform, the Python package, and the sentiment-modulated risk-budgeting pipeline: LLM news sentiment and market indicators combined per asset by an elastic net into a score that modulates risk budgets, long-only or long-short. Throughout, each concept points to the code that implements it, by file and line range, and the key code is shown in excerpts *extracted from the source when the document is built*: they are the code that runs, and a renamed function stops the build instead of leaving a stale reference.

Contents

I User guide **5**

1 Getting started **6**
1.1 What is in the repository . . . 6
1.2 Prerequisites . . . 6
1.3 Step 1: the C++ library . . . 6
1.4 Step 2: the wheel . . . 7
1.5 Step 3: the Python project . . . 7
1.6 Step 4: the sentiment risk-budgeting pipeline . . . 7
1.7 Debugging the C++ code from Python . . . 8
1.8 Wheels for every platform and releases . . . 8
1.9 Building this document . . . 8

II Architecture **9**

2 Purpose and architecture **10**
2.1 Purpose . . . 10
2.2 Components . . . 10
2.3 Directory map . . . 11
2.4 Build and test summary . . . 12

III The numerical core (C++) **13**

3 Time series containers **14**

4 Signal extraction **15**
4.1 Exponentially weighted moving average . . . 15
4.2 Butterworth band-pass . . . 16

5 Forecasting **17**
5.1 Simple exponential smoothing . . . 17
5.2 Kalman filter . . . 18
5.3 Gaussian process regression . . . 19

6 Optimization **21**
6.1 L-BFGS . . . 21
6.2 Augmented Lagrangian . . . 22
6.3 Sequential quadratic programming . . . 24
6.4 Nelder–Mead with box bounds . . . 25
6.5 Differential evolution . . . 26

7 Risk measures **29**

8 Risk budgeting **31**

9 The walk-forward portfolio engine **33**
9.1 Timing and absence of look-ahead . . . 33
9.2 Covariance estimation . . . 34
9.3 Allocation, budget schedule and volatility targeting . . . 34
9.4 Phase 1: parallel solves . . . 35
9.5 Phase 2: drift, trading and accounting . . . 37

10 Performance metrics **39**

11 Elastic net factor regression **41**
11.1 The elastic net . . . 41
11.2 The regression engine . . . 43

IV Derivations **46**

12 Simple exponential smoothing **47**
12.1 The recursion . . . 47
12.2 The statistical model: ETS(A,N,N) . . . 48
12.3 Estimation . . . 49
12.4 Implementation . . . 50

13 The Kalman filter **52**
13.1 Model . . . 52
13.2 The filter . . . 52
13.3 Likelihood . . . 54
13.4 Forecasting . . . 55
13.5 Smoothing . . . 55
13.6 Built-in models . . . 55
13.7 Implementation . . . 56

14 Gaussian process forecasting **59**
14.1 Model . . . 59
14.2 Kernels . . . 59
14.3 Posterior prediction . . . 61
14.4 Marginal likelihood . . . 61
14.5 Computation . . . 62
14.6 Hyperparameter optimization . . . 63
14.7 Implementation . . . 63

15 Convex optimization **65**
15.1 Module overview . . . 65
15.2 Problem statement . . . 65
15.3 From penalties to augmented Lagrangians . . . 66
15.4 Convergence in the convex case . . . 68
15.5 Practical algorithm . . . 68
15.6 The inner solver: L-BFGS . . . 69
15.7 Sequential quadratic programming . . . 70
15.8 Box constraints . . . 72

15.9 Using the implementation . . . 72

16 Elastic net factor regression: derivations **75**
16.1 Problem and notation . . . 75
16.2 The elastic net . . . 76
16.3 The regression engine . . . 79
16.4 Implementation . . . 82
16.5 Verification . . . 85
16.6 Limitations . . . 86

V Application, Python bindings and packaging **87**

17 Command-line application **88**

18 Backtest bindings for the Python package **89**

19 The full library in Python: `nabia_python_bridge` **91**
19.1 Modules . . . 91
19.2 Design . . . 91
19.3 pandas interface . . . 92
19.4 Development build and tests . . . 94

20 Wheels, cibuildwheel and GitHub Actions **95**
20.1 Wheels . . . 96
20.2 Many wheels . . . 100
20.3 GitHub Actions . . . 104
20.4 Doing it yourself . . . 108

VI The Python package **112**

21 Portfolio backtests: `nabia.backtest` **113**

22 Market data **114**
22.1 Universe and sources . . . 114
22.2 Storage and incremental updates . . . 115

23 Analytics on price frames **118**

VII Sentiment-modulated risk budgeting **121**

24 Overview of the pipeline **122**

25 Indicator tree, point-in-time features and target **125**
25.1 The indicator tree . . . 125
25.2 Point-in-time alignment and transforms . . . 125
25.3 The target . . . 126

26 The LLM news pipeline **127**
26.1 Prompts, schemas and the client . . . 127
26.2 Deduplication and novelty . . . 128
26.3 Weights and decay-weighted aggregation . . . 128

26.4 The five features per node . . . 129
26.5 The daily digest . . . 130

27 Elastic net per asset and the sentiment score **131**
27.1 The model . . . 131
27.2 Purged walk-forward training . . . 131
27.3 Positivity and stability selection . . . 132
27.4 The score and its normalization . . . 132
27.5 In-sample and out-of-sample diagnostics . . . 133

28 From sentiment to positions **134**
28.1 Risk budgeting . . . 134
28.2 Long-only: modulated budgets . . . 134
28.3 Long-short: directions . . . 134
28.4 The long-short simulator . . . 135

29 Synthetic data, experiments and results **138**
29.1 Synthetic market data . . . 138
29.2 Synthetic news with a known skill . . . 138
29.3 Results . . . 139
29.4 Running and extending . . . 141

VIII Verification and limitations **145**

30 Verification **146**

31 Limitations and future work **147**

A Glossary **148**

Part I

User guide

Chapter 1

Getting started

1.1 What is in the repository

Folder	Open in	Contents
`cpp/`	CLion (CMake)	the C++ library (`include`, `src`), Catch2 tests, the command-line backtest, and the pybind11 bindings that build the `nabia_python_bridge` wheel
`nabia_python/`	PyCharm	the Python project: the `nabia` package (portfolio backtests, analytics, market data, and the sentiment risk-budgeting pipeline `nabia.sentiment_rb`), its tests, its single `pyproject.toml` and its `.venv`
`doc/`	—	this document (`nabia.tex`, chapters, the snippet extractor, `build.sh`) and the specification of the sentiment pipeline (`sentiment_risk_budgeting_spec.md`)
`dist/`	—	built wheels (not versioned)
`.github/workflows/wheels.yml`	—	wheels for every platform in GitHub Actions

The two projects meet only through the wheel: `cpp/` builds `nabia_python_bridge` into `dist/`, and `nabia_python/` installs it into its `.venv`. The order of the steps below follows from that.

1.2 Prerequisites

- **C++**: CMake 3.25 or newer and a C++20 compiler (Apple clang from Xcode, GCC 12+, MSVC 2022). Eigen, iir1, Catch2, nlohmann/json and pybind11 are found on the system or downloaded by CMake.
- **Python**: CPython 3.10–3.14, and uv to build wheels for several versions. pdflatex (TeX Live, MacTeX) to build this document.
- **Optional**: the GitHub CLI (`gh`, logged in) to download published wheels or start CI; an Anthropic API key for the pipeline's `-live` mode.

1.3 Step 1: the C++ library

```
cd cpp
cmake --preset debug                    # configure (once, or after editing a CMakeLists.txt)
cmake --build --preset debug -j         # library, tests, command-line application, C++ examples
ctest --preset debug                    # the Catch2 suite and an end-to-end run of the application
```

Presets: `debug`, `release`, `releaseWithDebugInfo`, `asan` (address and undefined-behaviour sanitizers) and `python-bridge` (a development build of the bindings, §1.7). In CLion, open `cpp/`; it reads `CMakePresets.json`: enable the `debug` preset and run or debug `nabia_tests` or `nabia_rb_backtest`. The command-line backtest (Chapter 17):

```
./build/debug/apps/rb_backtest/nabia_rb_backtest --prices apps/rb_backtest/example/prices.csv \
    --config apps/rb_backtest/example/portfolio.json --output /tmp/result.csv
```

1.4 Step 2: the wheel

The bindings are packaged as a wheel, one per Python version (Chapter 20 explains wheels):

```
cpp/scripts/build_wheels.sh 3.14        # the Python version of nabia_python/.venv (no argument:
    ↪ 3.10-3.14)
NABIA_WHEEL_BUILD_TYPE=RelWithDebInfo cpp/scripts/build_wheels.sh 3.14   # with debug symbols
cpp\scripts\build_wheels.ps1 -Versions 3.12   # Windows (PowerShell): cibuildwheel, win_amd64
```

Each wheel is built in isolation, installed in a clean environment and tested with `nabia_python/tests/bridge` before it is copied to `dist/`. Instead of building, the wheels published by the CI can be downloaded: `nabia_python/scripts/fetch_wheels.sh` (the newest successful run, or `fetch_wheels.sh release v0.1.0`).

1.5 Step 3: the Python project

```
nabia_python/scripts/setup_env.sh       # creates nabia_python/.venv: the wheel from dist/ + nabia (
    ↪ editable)
cd nabia_python
.venv/bin/python -m pytest              # the bridge tests, the package tests, the pipeline tests
```

`setup_env.sh` (`setup_env.ps1` on Windows) reinstalls the wheel every time, so run it again after rebuilding the C++ code; Python changes are seen at once (editable install). `PYTHON=python3.12` chooses the interpreter of a new `.venv`. In PyCharm, open `nabia_python/` and select `.venv/bin/python` as the interpreter. The whole library is then available from Python (Chapter 19):

```
import nabia_python_bridge as nb
from nabia_python_bridge import frames

rb = nb.risk.RiskBudgeting(nb.risk.RiskBudgetingParams(budgets=[0.5, 0.3, 0.2]), nb.risk.Volatility(cov
    ↪ ))
weights = rb.optimize()
reg = frames.regress(fund_returns, factor_returns, min_factors=1, max_factors=3, train_fraction=0.7)
```

1.6 Step 4: the sentiment risk-budgeting pipeline

From `nabia_python/` (Part VII describes the method):

```
.venv/bin/python -m nabia.sentiment_rb                    # long-only, offline (synthetic data, mocked
    ↪ LLM), ~20 s
.venv/bin/python -m nabia.sentiment_rb --show             # also open the figures
.venv/bin/python -m nabia.sentiment_rb --config <(echo '{"budgets": {"mode": "long_short"}}')
.venv/bin/python -m nabia.sentiment_rb --ic-sweep 0 0.1 0.2 0.3 0.5    # alpha against news quality
.venv/bin/python -m nabia.sentiment_rb --out output/my_run             # another output folder
.venv/bin/nabia-sentiment --help                          # the same command, installed as a script
```

- **Configuration.** Every setting lives in `CONFIG` (`nabia_python/nabia/sentiment_rb/config.py:14--76`): windows, horizon, elastic-net grid, refit schedule, normalization, budget bounds,

long-only/long-short, LLM models, taxonomy, indicators, assets. `-config file.json` deep-merges overrides; long-short mode is described in §28.3:

```
{"normalize": {"method": "phi"}, "model": {"refit": "ME"}}
{"budgets": {"mode": "long_short", "short_below": 0.35, "long_above": 0.5, "modulate_long_short":
    ↪ false}}
```

- **Live LLM**: `.venv/bin/pip install -e ".[llm]"`, set `ANTHROPIC_API_KEY`, add `-live`: the news of the last `llm.live_days` days is scored by Claude (models in `llm.triage_model`, `scoring_model`, `digest_model`); the rest stays mocked.
- **Outputs**, in `output/sentiment_rb` by default:
  - tables: `comparison`, `ic`, `llm_ic`, `snapshot`, `budgets`, `signal`, `selection_frequency`, `ic_in_out_of_sample`, `equity`, `news` (CSV);
  - the LLM audit log `llm_audit.jsonl`;
  - figures `portfolio.png`, `budgets.png`, `model.png`; for a sweep, `sweep.png` and `sweep.csv`.

1.7 Debugging the C++ code from Python

1. Build a wheel with debug symbols (`NABIA_WHEEL_BUILD_TYPE=RelWithDebInfo`) and run `setup_env.sh`; or, without a wheel, `cmake --preset python-bridge && cmake --build --preset python-bridge` in `cpp/`, which writes the modules next to the package sources, and run the script with `PYTHONPATH=<repository>/cpp/bindings`.
2. Start the script in PyCharm, stopped at a breakpoint before the C++ call.
3. In CLion: Run → Attach to Process, the Python process. Breakpoints in `cpp/src` are hit.

1.8 Wheels for every platform and releases

```
gh workflow run wheels.yml                          # Linux x86_64/aarch64, macOS arm64/x86_64, Windows
    ↪ AMD64
gh run list --workflow wheels.yml                   # follow it
gh run download <run-id> -n all-wheels -D dist      # every wheel and the sdist
git tag v0.2.0 && git push origin v0.2.0            # a GitHub Release with all of them attached
```

Pushes to `main` and pull requests that change `cpp/` also start the workflow. Chapter 20 explains every job and how to write new ones.

1.9 Building this document

```
doc/build.sh                # doc/nabia.pdf: extracts the excerpts from the current code, runs
    ↪ pdflatex 3 times
doc/build.sh --figures      # first reruns the pipeline (long-only, long-short, both sweeps) for the
    ↪ figures/tables
```

The excerpts, their file:line locations and the packaging file pieces are extracted from the repository by `doc/extract_snippets.py`; a function that no longer exists stops the build. The figures and result tables of Part VII are copied from `nabia_python/output` into `doc/figures` by `-figures`.

Part II

Architecture

Chapter 2

Purpose and architecture

2.1 Purpose

nabia researches *systematic, risk-budgeted portfolios* across equity indices, government bond and credit markets, and commodities. A strategy has four stages, and the project provides a component for each:

1. **Signals**: filters and forecasters that extract trends from noisy prices (Chapters 4 and 5), a factor regression engine (Chapter 11), and news sentiment produced by a language model and combined with market indicators per asset (Part VII).
2. **Allocation**: *risk budgeting* decides each asset's share of portfolio risk (Chapter 8); the sentiment pipeline modulates those budgets, or the side of each position (long or short).
3. **Simulation**: a walk-forward engine turns budgets into weights at each rebalance, without look-ahead, and accounts for drift and costs (Chapter 9).
4. **Evaluation**: performance metrics (Chapter 10), information coefficients and in-sample versus out-of-sample diagnostics (Part VII).

The central design decision is to separate *what decides the risk budgets* (a fixed rule, equal risk, or the sentiment model) from *how budgets become positions* (a tested numerical engine). A signal never sets weights directly: it changes budgets within bounds, or the side of a position, and the engine computes the weights, so each decision is bounded and reproducible from its stored budgets.

2.2 Components

C++ library (`cpp/include/nabia`, `cpp/src`): the static library `nabia::nabia`, built with CMake, in C++20, on Eigen (linear algebra) and iir1 (filter design). Each module exposes an abstract interface whose parameter struct is owned through `std::unique_ptr`: `IFilter<P>` (`\snippet@ifilter`), `IForecaster<P>` (`\snippet@iforecaster`), `IOptimizer<P,R>` (`cpp/include/nabia/optimization/IOptimizer.h:16--39`), `IRiskMeasure` (`cpp/include/nabia/risk/IRiskMeasure.h:18--43`). Part III describes every method, Part IV derives the forecasters, the optimizers and the elastic net.

Application `nabia_rb_backtest` (`cpp/apps/rb_backtest`): a risk-budgeting backtest from a prices CSV and a JSON configuration (Chapter 17).

Bindings (`cpp/bindings`): one Python package, `nabia_python_bridge`, with two pybind11 modules: `_core` (the whole library) and `_rb_backtest` (the application's JSON backtest). `cpp/pyproject.toml` builds them into a wheel per Python version and platform (Chapters 19–20).

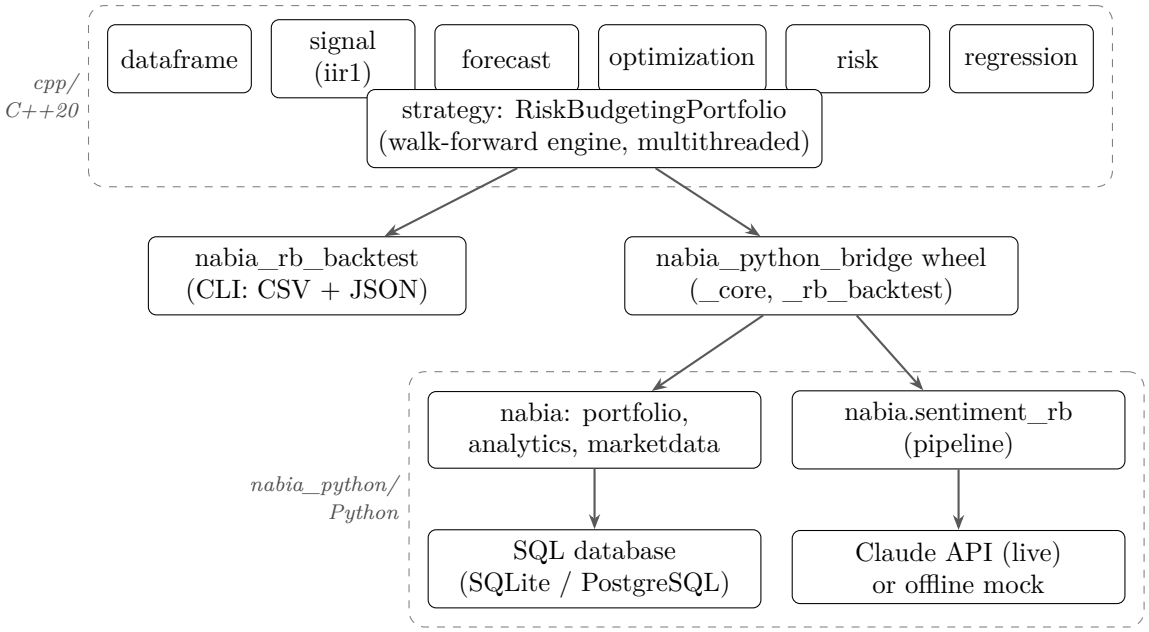


Figure 2.1: Components and dependencies. Arrows point from a component to what uses it.

Python package (`nabia_python/nabia`): `nabia.backtest` and `nabia.latest_weights` on pandas data (Chapter 21), analytics (Chapter 23), market data (Chapter 22), and the sentiment risk-budgeting pipeline `nabia.sentiment_rb` (Part VII).

2.3 Directory map

Path	Contents
`cpp/include/nabia/dataframe`	`TimeSeries`, `DataFrame`, the `TimeIndex` concept
`cpp/include/nabia/signal`	`IFilter`, `EWMA`, `ButterworthBandPass`, smoothing helpers
`cpp/include/nabia/forecast`	`IForecaster`, SES, Kalman filter, Gaussian process, kernels
`cpp/include/nabia/optimization`	`IOptimizer`; `convex/`: constraints, L-BFGS, augmented Lagrangian, SQP; `nongradopt/`: Nelder–Mead, differential evolution
`cpp/include/nabia/risk`	`IRiskMeasure`, volatility, VaR, ES, `RiskBudgeting<Measure>`
`cpp/include/nabia/stats,strategy,regression`	statistics; the portfolio engine and metrics; elastic net and the regression engine
`cpp/src,cpp/tests`	implementations (same layout); Catch2 tests
`cpp/apps/rb_backtest`	the command-line backtest
`cpp/bindings`	pybind11 sources (`src`) and the Python files of the wheel (`nabia_python_bridge`)
`cpp/pyproject.toml,cpp/scripts`	wheel build (scikit-build-core, cibuildwheel), `build_wheels.sh`, `.ps1`
`nabia_python/nabia`	`portfolio.py`, `analytics.py`, `marketdata/`, `sentiment_rb/`
`nabia_python/nabia/sentiment_rb`	`config.py`, `pipeline.py`, `__main__.py`; `data/`, `features/`, `llm/`, `model/`, `portfolio/`, `report/`, `prompts/`
`nabia_python/tests,scripts`	pytest suites (`tests/bridge` tests the wheel); `setup_env`, `fetch_wheels`
`doc`	this document and the pipeline's specification

2.4 Build and test summary

Task	Command (Chapter 1)
C++ build and tests	`cd cpp && cmake -preset debug && cmake -build -preset debug && ctest -preset debug`
Wheels	`cpp/scripts/build_wheels.sh [versions]`; CI: `gh workflow run wheels.yml`
Python environment and tests	`nabia_python/scripts/setup_env.sh`; `cd nabia_python && .venv/bin/python -m pytest`
Pipeline	`.venv/bin/python -m nabia.sentiment_rb [-config c.json] [-ic-sweep ...]`
This document	`doc/build.sh [-figures]`

Part III

The numerical core (C++)

Chapter 3

Time series containers

All numerical code works on Eigen matrices, but financial data is indexed by time. `TimeSeries<Index>` and `DataFrame<Index>` pair an Eigen vector or matrix (rows are times) with a *strictly increasing* index:

- The index type satisfies the `TimeIndex` concept (copyable and totally ordered): calendar dates, timestamps or integers.
- Construction validates the shapes, the ordering and unique column names.
- Time lookups use binary search.

Alignment. Markets have different holidays. A cross-asset backtest keeps only the dates where every asset has a finite price.

Returns. Simple returns $r_t = P_t/P_{t-1} - 1$ (`pct_change`) or log returns $\log(P_t/P_{t-1})$ are computed between consecutive *kept* rows. A return can therefore span a removed holiday. That is the right convention for a portfolio that can only trade when all its markets are open.

Code 3.1: Alignment: keep rows where every column is finite
`cpp/include/nabia/dataframe/DataFrame.h:108-122`

```
[[nodiscard]] DataFrame drop_missing() const {
    std::vector<Index> index;
    std::vector<Eigen::Index> keep;
    for (Eigen::Index i = 0; i < rows(); ++i) {
        if (values_.row(i).allFinite()) {
            keep.push_back(i);
            index.push_back(index_[static_cast<std::size_t>(i)]);
        }
    }
    Eigen::MatrixXd out(static_cast<Eigen::Index>(keep.size()), cols());
    for (std::size_t k = 0; k < keep.size(); ++k) {
        out.row(static_cast<Eigen::Index>(k)) = values_.row(keep[k]);
    }
    return DataFrame(std::move(index), columns_, std::move(out));
}
```

Code

`TimeIndex` concept — `cpp/include/nabia/dataframe/TimeIndex.h:17`
`TimeSeries` — `cpp/include/nabia/dataframe/TimeSeries.h:47--106`
`DataFrame` — `cpp/include/nabia/dataframe/DataFrame.h:23--146`
simple returns — `cpp/include/nabia/dataframe/DataFrame.h:126--132`
log returns — `cpp/include/nabia/dataframe/DataFrame.h:134--140`

Chapter 4

Signal extraction

Filters implement `IFilter<P>`. They are *causal*: output t uses inputs up to t only. Two modes are supported:

- streaming: `process(x)`, one sample at a time, keeping state;
- batch: `apply(signal)`, from a fresh state, without disturbing the streaming state.

Filter design is delegated to iir1, behind a private implementation struct (pimpl), so iir1 never appears in public headers.

4.1 Exponentially weighted moving average

The EWMA is the one-pole low-pass

$$y_t = \alpha x_t + (1-\alpha) y_{t-1}, \qquad H(z) = \frac{\alpha}{1-(1-\alpha)z^{-1}}, \qquad \alpha \in (0, 1].$$

Its DC gain is $H(1) = 1$, so a constant input passes unchanged.

Proposition 4.1.1 (Seeding)**.** *Let $\tilde{y}$ be the EWMA of $x_t - x_0$ started from a zero state. Then $y_t = x_0 + \tilde{y}_t$ equals the recursion started at $y_{-1} = x_0$:*

$$y_t = \alpha \sum_{k=0}^{t} (1-\alpha)^k x_{t-k} + (1-\alpha)^{t+1} x_0.$$

Proof. Unrolling the recursion from y_{-1} gives $y_t = \alpha \sum_{k=0}^{t} (1-\alpha)^k x_{t-k} + (1-\alpha)^{t+1} y_{-1}$. With a zero state, $\tilde{y}_t = \alpha \sum_{k=0}^{t} (1-\alpha)^k (x_{t-k} - x_0)$. Using $\alpha \sum_{k=0}^{t} (1-\alpha)^k = 1 - (1-\alpha)^{t+1}$, adding x_0 gives the formula with $y_{-1} = x_0$. □

Starting from zero instead would bias the early outputs towards zero, so the filter seeds with the first sample by default.

Parameterizations. Two helpers convert the usual parameterizations to α:

- `alpha_from_span`$(N) = 2/(N+1)$. The mean age of the EWMA weights is $\sum_k k\,\alpha(1-\alpha)^k = (1-\alpha)/\alpha$, and that of an N-point moving average is $(N-1)/2$. Equating the two gives $\alpha = 2/(N+1)$.
- `alpha_from_halflife`$(h) = 1 - 2^{-1/h}$. This solves $(1-\alpha)^h = \frac{1}{2}$: the weight h steps back is half the newest weight.

Code 4.1: EWMA on iir1's one-pole section, with first-sample seeding
`cpp/src/signal/EWMA.cpp:16-36`

```
struct EWMA::Impl {
    Iir::Custom::OnePole section;
    Iir::DirectFormI state;
    bool seed = true;
    bool started = false;
    double offset = 0.0;

    double process(double x) {
        if (!started) {
            offset = seed ? x : 0.0;
            started = true;
        }
        return section.filter(x - offset, state) + offset;
    }

    void reset() {
        state.reset();
        started = false;
        offset = 0.0;
    }
};
```

Code

class and parameters — `cpp/include/nabia/signal/EWMA.h:23--43`
span and half-life conversions — `cpp/src/signal/EWMA.cpp:72--79`
simple moving average helper (rolling sum, $O(n)$) — `cpp/src/signal/smoothing.cpp:7--28`

4.2 Butterworth band-pass

A causal Butterworth band-pass isolates trend cycles between two frequencies $0 < f_\ell < f_h < f_s/2$. It is built in three steps.

1. **Analog prototype.** The order-N low-pass prototype is maximally flat:

$$|H(j\Omega)|^2 = \frac{1}{1+\Omega^{2N}},$$

   with poles $s_k = e^{j\pi(2k+N-1)/(2N)}$, $k = 1, \dots, N$, on the left half of the unit circle.

2. **Low-pass to band-pass.** The substitution

$$s \mapsto \frac{s^2+\Omega_0^2}{sB}$$

   maps it to a band-pass with $2N$ poles, centred on Ω_0 with bandwidth B.

3. **Bilinear transform.** The map $s = 2f_s\frac{1-z^{-1}}{1+z^{-1}}$, with the band edges prewarped by $\Omega = 2f_s\tan(\pi f/f_s)$, gives the digital filter. It is run as a cascade of second-order sections, which is numerically stable at high orders.

iir1 takes the band as a centre and a width, so the edges map to centre $(f_\ell + f_h)/2$ and width $f_h - f_\ell$. The filter then has $|H| = 1/\sqrt{2}$ ($-3\,\mathrm{dB}$) at both edges and unit gain at the centre. The tests check exactly these properties, and strong attenuation outside the band. Being causal, the filter delays the signal: the group delay grows with the order and with narrower bands.

Code

parameters (order, sample rate, edges) — `cpp/include/nabia/signal/ButterworthBandPass.h:11--16`
design: edges to centre and width — `cpp/src/signal/ButterworthBandPass.cpp:19--22`

Chapter 5

Forecasting

Forecasters implement `IForecaster<P>`:

- `fit(y)` estimates the model on a series;
- `forecast(H)` returns the mean $\hat{y}_{T+h}$ and the predictive variance v_h for $h = 1..H$;
- Gaussian prediction intervals follow as $\hat{y} \pm z_{(1+c)/2}\sqrt{v_h}$.

Chapters 12–14 derive each model in full. This section states what the code computes.

5.1 Simple exponential smoothing

The model is the ETS(A,N,N) state space

$$y_t = \ell_{t-1} + \varepsilon_t, \qquad \ell_t = \ell_{t-1} + \alpha\varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \sigma^2).$$

The fit initializes $\ell = y_1$ and runs the error-correction form $\ell \leftarrow \ell + \alpha(y_t - \ell)$, accumulating $\mathrm{SSE} = \sum_t (y_t - \ell_{t-1})^2$. It then estimates $\hat{\sigma}^2 = \mathrm{SSE}/T$.

Proposition 5.1.1. *The h-step forecast is $\hat{y}_{T+h} = \ell_T$, with variance $\sigma^2\big(1 + (h-1)\alpha^2\big)$.*

Proof. Iterating the state equation gives $\ell_{T+h-1} = \ell_T + \alpha\sum_{j=1}^{h-1}\varepsilon_{T+j}$, so $y_{T+h} = \ell_T + \alpha\sum_{j=1}^{h-1}\varepsilon_{T+j} + \varepsilon_{T+h}$. The errors are independent, so the variance is $(h-1)\alpha^2\sigma^2 + \sigma^2$. □

Code 5.1: SES fit and forecast variance
`cpp/src/forecast/exponential_smoothing.cpp:27-45`

```
    // Initialise the level with the first observation.
    level_ = y(0);
    for (Eigen::Index t = 0; t < y.size(); ++t) {
        in_sample_(t) = level_;
        const double err = y(t) - level_;
        sse_ += err * err;
        level_ += alpha * err; // equivalent to alpha*y + (1-alpha)*level
    }
    fitted_ = true;
}

Forecast SimpleExponentialSmoothing::forecast(Eigen::Index horizon) const {
    require_fitted("SimpleExponentialSmoothing::forecast");
    require_horizon(horizon, "SimpleExponentialSmoothing::forecast");
    const double alpha = parameters_->alpha;
    const double sigma2 = sse_ / static_cast<double>(in_sample_.size());
    Forecast out{Eigen::VectorXd::Constant(horizon, level_), Eigen::VectorXd(horizon)};
    for (Eigen::Index h = 0; h < horizon; ++h) {
        out.variance(h) = sigma2 * (1.0 + static_cast<double>(h) * alpha * alpha);
```

5.2 Kalman filter

The model is the linear Gaussian state space

$$x_t = Fx_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q); \qquad y_t = Hx_t + v_t, \quad v_t \sim \mathcal{N}(0, R),$$

with prior $x_0 \sim \mathcal{N}(\hat{x}_0, P_0)$, where the prior describes the first state. Each step first predicts:

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1}, \qquad P_{t|t-1} = FP_{t-1|t-1}F^\top + Q.$$

Then it computes the innovation and its variance:

$$e_t = y_t - H\hat{x}_{t|t-1}, \qquad S_t = HP_{t|t-1}H^\top + R.$$

Finally it corrects, with gain $K_t = P_{t|t-1}H^\top / S_t$:

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t e_t, \qquad P_{t|t} = (I - K_t H)P_{t|t-1}(I - K_t H)^\top + R\,K_t K_t^\top.$$

- **Joseph form.** The covariance update above is the posterior covariance for *any* gain. It is symmetric and positive semi-definite by construction, so it stays valid under rounding, unlike the short form $(I - KH)P$.
- **Missing observations.** Non-finite y_t skip the correction.
- **Log-likelihood.** $\ell = -\frac{1}{2}\sum_t \left(\log 2\pi + \log S_t + e_t^2/S_t\right)$, summed over the observed points.
- **Forecasts.** They iterate the prediction step: mean Hx, variance $HPH^\top + R$.
- **Smoothing.** The Rauch–Tung–Striebel smoother uses the gain $C_t = P_{t|t}F^\top P_{t+1|t}^{-1}$ (computed by an $LDL^\top$ solve, not an inverse) and $\hat{x}_t^s = \hat{x}_{t|t} + C_t(\hat{x}_{t+1}^s - \hat{x}_{t+1|t})$.

Two ready-made models are provided:

- *local level*: $F = H = 1$, so the level is a random walk observed with noise;
- *local linear trend*: $F = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$, $H = (1, 0)$, $Q = \mathrm{diag}(q_\ell, q_s)$, so level and slope both drift.

Both use a diffuse prior ($P_0 = 10^7 I$). The local linear trend's slope state is a direct estimate of the trend.

Code 5.2: Kalman predict and Joseph-form correction
`cpp/src/forecast/KalmanFilter.cpp:91-117`

```cpp
if (t > 0) {  // predict; the prior already describes x[0]
    x = F * x;
    P = F * P * F.transpose() + p.process_noise;
}
const auto ts = static_cast<std::size_t>(t);
predicted_states_.col(t) = x;
predicted_covariances_[ts] = P;

const double y_hat = H.dot(x);
const VectorXd PHt = P * H.transpose();
const double S = H.dot(PHt) + p.observation_noise;
one_step_(t) = y_hat;
innovation_variances_(t) = S;

if (std::isfinite(y(t))) {  // correct
    const double e = y(t) - y_hat;
    const VectorXd K = PHt / S;
    x += K * e;
    const MatrixXd A = I - K * H;
    P = A * P * A.transpose() + p.observation_noise * K * K.transpose();
    innovations_(t) = e;
```

```
    log_likelihood_ -= 0.5 * (log_2pi + std::log(S) + e * e / S);
} else {
    innovations_(t) = std::numeric_limits<double>::quiet_NaN();
}
filtered_states_.col(t) = x;
filtered_covariances_[ts] = P;
```

Code

parameters F, H, Q, R, x_0, P_0 — `cpp/include/nabia/forecast/KalmanFilter.h:16--34`
local level and local linear trend — `cpp/src/forecast/KalmanFilter.cpp:25--48`
forecast — `cpp/src/forecast/KalmanFilter.cpp:122--136`
RTS smoother — `cpp/src/forecast/KalmanFilter.cpp:168--182`

5.3 Gaussian process regression

The series is regressed on time $t = 0, \dots, T-1$ in three steps.

1. **Standardization.** The observations are standardized, $z = (y - \bar{y})/s$, where s is the sample standard deviation when normalizing.

2. **Model.** $z \sim \mathcal{N}(0, K)$ with $K = K_f + \sigma_n^2 I$ and $(K_f)_{ij} = k(t_i, t_j)$. The available kernels, with $r = t - t'$:

$$
\begin{aligned}
k_{\mathrm{SE}} &= \sigma^2 \exp\!\big(-r^2/2\ell^2\big), \\
k_{\mathrm{M52}} &= \sigma^2 (1 + u + u^2/3) e^{-u}, \quad u = \sqrt{5}|r|/\ell, \\
k_{\mathrm{per}} &= \sigma^2 \exp\!\big(-2\sin^2(\pi r/p)/\ell^2\big).
\end{aligned}
$$

   Sums and products of kernels are kernels too.

3. **Hyperparameters.** With $\alpha = K^{-1} z$ (from a Cholesky factorization $K = LL^\top$), the log marginal likelihood is

$$
\log p(z) = -\tfrac{1}{2} z^\top \alpha - \textstyle\sum_i \log L_{ii} - \tfrac{T}{2} \log 2\pi.
$$

   The hyperparameters θ (logs of variances, length scales, periods and, optionally, the noise) maximize it, using L-BFGS with the analytic gradient

$$
\frac{\partial(-\log p)}{\partial \theta_j} = -\tfrac{1}{2} \operatorname{tr}\!\Big((\alpha\alpha^\top - K^{-1}) \frac{\partial K}{\partial \theta_j} \Big).
$$

   Working in logs keeps the parameters positive. Values with $|\theta_j| > 20$, or a K that is not positive definite, are returned as $+\infty$, which the line search rejects. The noise has a floor (`min_noise_variance`).

Prediction. At new times t_*, with $v = L^{-1} k_*$:

$$
\text{mean} = \bar{y} + s\, k_*^\top \alpha, \qquad \text{variance} = s^2 \big(k(t_*, t_*) - v^\top v + \sigma_n^2 \big).
$$

The reported likelihood is converted back to the units of y by subtracting $T \log s$.

Code 5.3: Gaussian process: negative log-likelihood gradient
`cpp/src/forecast/GaussianProcess.cpp:100-111`

```
// d(-log p)/d theta_j = -1/2 tr((alpha alpha' - K^-1) dK/d theta_j)
const MatrixXd W = f->alpha * f->alpha.transpose() - f->llt.solve(MatrixXd::Identity(T, T));
grad.setZero();
for (Index j = 0; j < T; ++j) {
    for (Index i = 0; i < T; ++i) {
```

```
        grad.head(nk) -= 0.5 * W(i, j) * work->gradient(times_(i), times_(j));
    }
}
if (p.optimize_noise) {
    grad(nk) = -0.5 * noise * W.trace();  // dK/dlog(noise) = noise I
}
return -f->log_ml;
```

Code

Cholesky and log marginal likelihood — `cpp/src/forecast/GaussianProcess.cpp:31--42`
fit with hyperparameter optimization — `cpp/src/forecast/GaussianProcess.cpp:56--133`
predictive mean and variance — `cpp/src/forecast/GaussianProcess.cpp:135--147`
kernels and their log-parameter gradients — `cpp/src/forecast/Kernels.cpp:41--226`

Chapter 6

Optimization

Optimizers implement `IOptimizer<P,R>`: `optimize(initial_guess)` returns the solution, and `result()` returns a solver-specific result object (status, iterations, residuals). Linear constraints are expressed in matrix form: $A_{\text{eq}}x = b_{\text{eq}}$ and $A_{\text{in}}x \le b_{\text{in}}$. Box bounds $l \le x \le u$ convert to inequality rows.

Code

optimizer interface — `cpp/include/nabia/optimization/IOptimizer.h:16--39`
linear constraints and their conversion — `cpp/include/nabia/optimization/convex/Constraints.h:22--47`

6.1 L-BFGS

L-BFGS minimizes a smooth f with search direction $d = -H_k g_k$. Here H_k is the BFGS inverse-Hessian approximation built from the last m pairs $s_j = x_{j+1} - x_j$, $y_j = g_{j+1} - g_j$, with $\rho_j = 1/s_j^\top y_j$. It is applied in $O(mn)$ by the two-loop recursion:

1. $q \leftarrow g$. For j from newest to oldest: $a_j = \rho_j s_j^\top q$, then $q \leftarrow q - a_j y_j$.
2. $r \leftarrow \gamma q$, with $\gamma = s^\top y / y^\top y$ from the newest pair. On the first iteration $\gamma = 1/\max(1, \|g\|)$.
3. For j from oldest to newest: $\beta = \rho_j y_j^\top r$, then $r \leftarrow r + (a_j - \beta)s_j$. Finally $d = -r$.

Safeguards keep the method well defined:

- A pair is stored only if $s^\top y > 10^{-12}\|s\|\|y\|$ (the curvature condition), which keeps H positive definite.
- If d is not a descent direction, the memory is cleared and the method restarts from steepest descent.
- Iteration stops when $\|g\|_\infty \le \varepsilon$.

Line search. Backtracking from $t = 1$ halves t until the step is accepted. A non-finite f counts as rejection, which is how barrier and domain constraints are enforced. A step is accepted under either condition:

Armijo $f(x + td) \le f(x) + c_1 t\, g^\top d$, with $c_1 = 10^{-4}$.

Approximate Wolfe (Hager–Zhang), when f is flat to within rounding: $f(x + td) \le f(x) + \epsilon_f |f(x)|$ and $\sigma\, g^\top d \le g(x + td)^\top d \le (2c_1 - 1)\, g^\top d$, with $\epsilon_f = 10^{-10}$ and $\sigma = 0.9$.

Near a minimum the decrease Armijo requires is smaller than the rounding error in f, while directional derivatives remain accurate. The second test lets the method keep converging there. On 500-asset risk budgeting it took the attainable budget accuracy from about $3 \cdot 10^{-8}$ to 10^{-8}.

Code 6.1: Two-loop recursion
cpp/src/optimization/convex/LBFGS.cpp:63-77

```
// Two-loop recursion: d = -H g.
VectorXd q = g;
std::vector<double> alpha(s_hist.size());
for (std::size_t j = s_hist.size(); j-- > 0;) {
    alpha[j] = rho_hist[j] * s_hist[j].dot(q);
    q -= alpha[j] * y_hist[j];
}
const double gamma = s_hist.empty() ? 1.0 / std::max(1.0, g.norm())
                                    : s_hist.back().dot(y_hist.back()) / y_hist.back().squaredNorm();
VectorXd d = gamma * q;
for (std::size_t j = 0; j < s_hist.size(); ++j) {
    const double beta = rho_hist[j] * y_hist[j].dot(d);
    d += (alpha[j] - beta) * s_hist[j];
}
d = -d;
```

Code 6.2: Line search: Armijo or approximate Wolfe
cpp/src/optimization/convex/LBFGS.cpp:88-116

```
double t = 1.0;
VectorXd x_new;
VectorXd g_new(x.size());
double f_new = fx;
bool accepted = false;
for (int ls = 0; ls < p.max_line_search; ++ls) {
    x_new = x + t * d;
    f_new = value_and_gradient_(x_new, g_new);
    ++result_.evaluations;
    if (!std::isfinite(f_new)) {
        t *= p.backtrack;
        continue;
    }
    // Armijo sufficient decrease, or Hager-Zhang's approximate Wolfe
    // conditions when f is flat to within rounding: near the minimum
    // the decrease in f drops below machine precision, but the
    // directional derivative is still accurate.
    const double new_slope = g_new.dot(d);
    const bool armijo = f_new <= fx + p.armijo_c1 * t * slope;
    const bool approx_wolfe = f_new <= fx + p.flat_tolerance * std::abs(fx) &&
                              new_slope <= (2.0 * p.armijo_c1 - 1.0) * slope &&
                              new_slope >= p.wolfe_c2 * slope;
    if (armijo || approx_wolfe) {
        accepted = true;
        break;
    }
    t *= p.backtrack;
}
if (!accepted) {
```

Code

parameters and defaults — cpp/include/nabia/optimization/convex/LBFGS.h:12--25
full algorithm — cpp/src/optimization/convex/LBFGS.cpp:30--148

6.2 Augmented Lagrangian

For $\min f(x)$ subject to $h(x) = A_{\text{eq}}x - b_{\text{eq}} = 0$ and $g(x) = A_{\text{in}}x - b_{\text{in}} \le 0$, the Powell–Hestenes–Rockafellar augmented Lagrangian is

$$L_\rho(x;\lambda,\mu) = f(x) + \lambda^\top h + \frac{\rho}{2}\|h\|^2 + \frac{1}{2\rho}\sum_i \Big(\max(0,\mu_i + \rho g_i)^2 - \mu_i^2\Big),$$

with gradient $\nabla f + A_{\text{eq}}^\top(\lambda + \rho h) + A_{\text{in}}^\top \max(0, \mu + \rho g)$. Each outer iteration does four things:

1. **Minimize** L_ρ with L-BFGS, warm-started at the previous iterate. The inner tolerance starts at $\max(\varepsilon, 0.1)$ and shrinks tenfold per outer iteration down to ε.

2. **Measure** the violation $v = \max\left(\|h\|_\infty, \|\max(g, -\mu/\rho)\|_\infty\right)$. The second term is zero if and only if $g \le 0$ and $\mu_i g_i = 0$, so it also measures complementarity.

3. **Update the multipliers**: $\lambda \leftarrow \lambda + \rho h$ and $\mu \leftarrow \max(0, \mu + \rho g)$. After this update, $\nabla_x L_\rho = \nabla f + A_{\text{eq}}^\top \lambda + A_{\text{in}}^\top \mu$, so the inner gradient norm is the KKT stationarity.

4. **Test and adapt.** Stop when $v \le \varepsilon$ and stationarity $\le \varepsilon$. Otherwise, if $v > 0.25\, v_{\text{prev}}$, set $\rho \leftarrow \min(10\rho, 10^{10})$ (starting from $\rho_0 = 10$).

Code 6.3: Augmented Lagrangian value and gradient
`cpp/src/optimization/convex/AugmentedLagrangian.cpp:58-73`

```
// L_rho(x) = f + lambda'h + rho/2 |h|^2 + 1/(2 rho) sum(max(0, mu + rho g)^2 - mu^2)
// grad     = grad f + A_eq'(lambda + rho h) + A_ineq' max(0, mu + rho g)
const LBFGS::ValueAndGradient augmented = [&](const VectorXd& z, VectorXd& grad) {
    double value = objective_(z);
    grad = gradient_(z);
    if (has_eq) {
        const VectorXd h = lf.A_eq * z - lf.b_eq;
        value += lambda.dot(h) + 0.5 * rho * h.squaredNorm();
        grad.noalias() += lf.A_eq.transpose() * (lambda + rho * h);
    }
    if (has_ineq) {
        const VectorXd shifted = (mu + rho * (lf.A_ineq * z - lf.b_ineq)).cwiseMax(0.0);
        value += (shifted.squaredNorm() - mu.squaredNorm()) / (2.0 * rho);
        grad.noalias() += lf.A_ineq.transpose() * shifted;
    }
    return value;
```

Code 6.4: Multiplier and penalty updates
`cpp/src/optimization/convex/AugmentedLagrangian.cpp:93-119`

```
// Violation measured before the multiplier update; for inequalities
// max(g, -mu/rho) is zero iff g <= 0 and mu * g = 0 (complementarity).
double violation = 0.0;
if (has_eq) {
    const VectorXd h = lf.A_eq * x - lf.b_eq;
    violation = h.lpNorm<Eigen::Infinity>();
    lambda += rho * h;
}
if (has_ineq) {
    const VectorXd g = lf.A_ineq * x - lf.b_ineq;
    violation = std::max(violation, g.cwiseMax(-mu / rho).lpNorm<Eigen::Infinity>());
    mu = (mu + rho * g).cwiseMax(0.0);
}

// After the update, grad L_rho(x) = grad f + A_eq' lambda + A_ineq' mu.
result_.stationarity = inner.gradient_norm;
result_.constraint_violation = violation;
if (violation <= p.tolerance && inner.gradient_norm <= p.tolerance) {
    result_.converged = true;
    break;
}

if (violation > options_.required_decrease * previous_violation) {
    rho = std::min(rho * options_.penalty_growth, options_.max_penalty);
}
previous_violation = violation;
inner_tol = std::max(p.tolerance, 0.1 * inner_tol);
```

Code

options and defaults — `cpp/include/nabia/optimization/convex/AugmentedLagrangian.h:15--23`
full algorithm — `cpp/src/optimization/convex/AugmentedLagrangian.cpp:33--128`

6.3 Sequential quadratic programming

At x_k, SQP solves the quadratic subproblem

$$\min_d \; g_k^\top d + \tfrac{1}{2} d^\top B_k d \quad \text{s.t.} \quad A_{\text{eq}} d = b_{\text{eq}} - A_{\text{eq}} x_k, \;\; A_{\text{in}} d \le b_{\text{in}} - A_{\text{in}} x_k.$$

It uses the augmented Lagrangian of §6.2 at tolerance 10^{-10}, or a Cholesky solve $d = -B^{-1} g$ when there are no constraints.

Convergence is declared when three KKT residuals at x_k, computed with the QP multipliers, are all below ε:

- stationarity $\|g + A_{\text{eq}}^\top \lambda + A_{\text{in}}^\top \mu\|_\infty$;
- constraint violation;
- complementarity $\max_i |\mu_i g_i|$.

Merit function $\phi(x) = f(x) + \nu V_1(x)$, with $V_1 = \|h\|_1 + \sum_i \max(g_i, 0)$. The penalty is kept at $\nu \ge \|(\lambda, \mu)\|_\infty + 10^{-3}$. Because the constraints are linear and d satisfies the linearized constraints, $V_1(x + td) \le (1 - t) V_1(x)$. The directional derivative is therefore bounded by $g^\top d - \nu V_1(x) < 0$, so d descends ϕ. Backtracking applies Armijo to ϕ with this slope.

Restart If the line search fails, B resets to the identity. A failure from $B = I$ ends the iteration.

Proposition 6.3.1 (Damped BFGS, Powell)**.** *Let $r = \theta y + (1 - \theta) Bs$, with $\theta = 1$ if $s^\top y \ge 0.2\, s^\top Bs$ and $\theta = 0.8\, s^\top Bs / (s^\top Bs - s^\top y)$ otherwise. The update*

$$B^+ = B - \frac{Bss^\top B}{s^\top Bs} + \frac{rr^\top}{s^\top r}$$

keeps B positive definite.

Proof. If $\theta < 1$, then $s^\top r = s^\top Bs - \theta(s^\top Bs - s^\top y) = 0.2\, s^\top Bs > 0$. If $\theta = 1$, then $s^\top r = s^\top y \ge 0.2\, s^\top Bs > 0$. The BFGS update with a pair satisfying $s^\top r > 0$ preserves positive definiteness. □

After the first step with $s^\top y > 0$, B is scaled to $(y^\top y / s^\top y) I$ before the update, which matches the curvature along s.

Code 6.5: Damped BFGS update
`cpp/src/optimization/convex/SQP.cpp:41-53`

```cpp
void damped_bfgs_update(MatrixXd& B, const VectorXd& s, const VectorXd& y) {
    const VectorXd Bs = B * s;
    const double sBs = s.dot(Bs);
    if (!(sBs > 0.0)) {
        return;
    }
    const double sy = s.dot(y);
    const double theta = sy >= 0.2 * sBs ? 1.0 : 0.8 * sBs / (sBs - sy);
    const VectorXd r = theta * y + (1.0 - theta) * Bs;
    B.noalias() -= Bs * Bs.transpose() / sBs;
    B.noalias() += r * r.transpose() / s.dot(r);
    B = 0.5 * (B + B.transpose());  // remove round-off asymmetry
}
```

Code 6.6: ℓ_1 merit penalty and line search
`cpp/src/optimization/convex/SQP.cpp:156-187`

```cpp
// l1 merit function phi = f + nu * V1; nu must exceed the multipliers
// for d to be a descent direction of phi.
double max_multiplier = 0.0;
```

```
if (lambda.size() > 0) {
    max_multiplier = lambda.lpNorm<Eigen::Infinity>();
}
if (mu.size() > 0) {
    max_multiplier = std::max(max_multiplier, mu.lpNorm<Eigen::Infinity>());
}
nu = std::max(nu, max_multiplier + options_.merit_penalty_margin);
const double v1 = l1_violation(x);
const double merit = fx + nu * v1;
// Directional derivative of phi along d (the constraints are linear and
// d satisfies them, so V1(x + t d) <= (1 - t) V1(x)).
double slope = gx.dot(d) - nu * v1;
if (!(slope < 0.0)) {
    slope = -1e-16 * std::max(1.0, std::abs(merit));  // d ~ 0: accept the first decrease
}

double t = 1.0;
VectorXd x_new;
double f_new = fx;
bool accepted = false;
for (int ls = 0; ls < options_.max_line_search; ++ls) {
    x_new = x + t * d;
    f_new = objective_(x_new);
    ++result_.evaluations;
    if (std::isfinite(f_new) && f_new + nu * l1_violation(x_new) <= merit + options_.merit_c1 * t *
    ↪ slope) {
        accepted = true;
        break;
    }
    t *= options_.backtrack;
```

Code

options and defaults — `cpp/include/nabia/optimization/convex/SQP.h:15--23`
QP subproblem through the augmented Lagrangian — `cpp/src/optimization/convex/SQP.cpp:26--38`
full algorithm — `cpp/src/optimization/convex/SQP.cpp:66--218`

6.4 Nelder–Mead with box bounds

Nelder–Mead keeps a simplex of $n+1$ vertices sorted by value, $f(x_0) \le \cdots \le f(x_n)$, with c the centroid of the best n. Each iteration:

1. **Reflect**: $x_r = c + \alpha(c - x_n)$.
2. If $f_r < f_0$, **expand** to $x_e = c + \gamma(x_r - c)$ and keep the better of x_e and x_r.
3. Else, if $f_r < f_{n-1}$, accept x_r.
4. Else **contract**:
   - outside, $x_c = c + \rho(x_r - c)$, if $f_r < f_n$ (accepted if $f_c < f_r$);
   - inside, $x_c = c + \rho(x_n - c)$, otherwise (accepted if $f_c < f_n$).
5. Else **shrink** every vertex towards the best: $x_j \leftarrow x_0 + \sigma(x_j - x_0)$.

The *adaptive* coefficients of Gao and Han, $\alpha = 1$, $\gamma = 1 + 2/n$, $\rho = \frac{3}{4} - \frac{1}{2n}$, $\sigma = 1 - \frac{1}{n}$, avoid the stagnation of the standard values $(1, 2, \frac{1}{2}, \frac{1}{2})$ in higher dimensions.

Bounds are enforced by projecting every trial point onto the box. The box is convex, so a shrink stays inside it.

Termination. The iteration stops when both conditions hold:

$$f_n - f_0 \le \tau_f(1 + |f_0|) \quad \text{and} \quad \max_j \|x_j - x_0\|_\infty \le \tau_x(1 + \|x_0\|_\infty).$$

A NaN objective counts as $+\infty$.

Code 6.7: One Nelder–Mead iteration
`cpp/src/optimization/nongradopt/NelderMead.cpp:119-154`

```
const VectorXd centroid = simplex.leftCols(n).rowwise().mean();
const VectorXd worst = simplex.col(n);
const VectorXd xr = box.project(centroid + alpha * (centroid - worst));
const double fr = f(xr);

if (fr < values(0)) {
    const VectorXd xe = box.project(centroid + gamma * (xr - centroid));
    const double fe = f(xe);
    if (fe < fr) {
        simplex.col(n) = xe;
        values(n) = fe;
    } else {
        simplex.col(n) = xr;
        values(n) = fr;
    }
    continue;
}
if (fr < values(n - 1)) {
    simplex.col(n) = xr;
    values(n) = fr;
    continue;
}
// Contraction: outside if the reflection beat the worst vertex, inside otherwise.
const bool outside = fr < values(n);
const VectorXd xc = outside ? VectorXd(box.project(centroid + rho * (xr - centroid)))
                            : VectorXd(box.project(centroid + rho * (worst - centroid)));
const double fc = f(xc);
if (fc < (outside ? fr : values(n))) {
    simplex.col(n) = xc;
    values(n) = fc;
    continue;
}
// Shrink towards the best vertex (stays inside the box: the box is convex).
for (Index j = 1; j <= n; ++j) {
    simplex.col(j) = simplex.col(0) + sigma * (simplex.col(j) - simplex.col(0));
    values(j) = f(simplex.col(j));
```

Code

adaptive coefficients — `cpp/src/optimization/nongradopt/NelderMead.cpp:43--46`
box projection — `cpp/src/optimization/nongradopt/box.hpp:17--23`
full algorithm, initial simplex — `cpp/src/optimization/nongradopt/NelderMead.cpp:24--162`

6.5 Differential evolution

Population. N_p vectors (default $15n$, at least 4, or 6 for rand/2), drawn uniformly in the box. An initial guess, if given, replaces member 0.

Mutation. Each generation draws one $F \sim U[F_{\min}, F_{\max}]$ (dithering), then builds a mutant per member i from distinct random members $r_1, \dots, r_5 \neq i$:

$$\begin{aligned}
\text{rand/1:} \quad & v = x_{r_1} + F(x_{r_2} - x_{r_3}), \\
\text{best/1:} \quad & v = x_{\text{best}} + F(x_{r_1} - x_{r_2}), \\
\text{current-to-best/1:} \quad & v = x_i + F(x_{\text{best}} - x_i) + F(x_{r_1} - x_{r_2}), \\
\text{rand/2:} \quad & v = x_{r_1} + F(x_{r_2} - x_{r_3}) + F(x_{r_4} - x_{r_5}).
\end{aligned}$$

Crossover. Binomial: $u_{ij} = v_{ij}$ if $U < \mathrm{CR}$ or $j = j_{\mathrm{rand}}$, else x_{ij}, so at least one coordinate comes from the mutant. A coordinate outside the box bounces back to the midpoint between the parent and the violated bound.

Selection. Greedy: u replaces x_i if $f(u) \leq f(x_i)$. Ties go to the trial, which keeps the population moving on plateaus.

Termination. The run stops when the population's objective values have $\mathrm{sd}(f) \leq \tau_a + \tau|\overline{f}|$.

Parallelism and reproducibility. All trial vectors of a generation are built sequentially from one seeded RNG *before* evaluation. Only the evaluations run in parallel, so the result is the same for every thread count. A `ParallelEvaluator<T,R>` does the evaluations:

- each input becomes a `std::packaged_task` in a queue;
- n worker threads consume the queue;
- the caller collects the results, in input order, through the tasks' futures.

Code 6.8: Mutation and crossover of one generation
`cpp/src/optimization/nongradopt/DifferentialEvolution.cpp:132-172`

```
const double F = p.mutation_min + (p.mutation_max - p.mutation_min) * unit(rng);

// Build all trial vectors first (sequential RNG => thread-count independent).
for (Index j = 0; j < np; ++j) {
    VectorXd mutant(n);
    switch (p.strategy) {
    case DEStrategy::Rand1: {
        const auto r = pick_distinct<3>(rng, np, j);
        mutant = population.col(r[0]) + F * (population.col(r[1]) - population.col(r[2]));
        break;
    }
    case DEStrategy::Best1: {
        const auto r = pick_distinct<2>(rng, np, j);
        mutant = population.col(best) + F * (population.col(r[0]) - population.col(r[1]));
        break;
    }
    case DEStrategy::CurrentToBest1: {
        const auto r = pick_distinct<2>(rng, np, j);
        mutant = population.col(j) + F * (population.col(best) - population.col(j)) +
                 F * (population.col(r[0]) - population.col(r[1]));
        break;
    }
    case DEStrategy::Rand2: {
        const auto r = pick_distinct<5>(rng, np, j);
        mutant = population.col(r[0]) + F * (population.col(r[1]) - population.col(r[2])) +
                 F * (population.col(r[3]) - population.col(r[4]));
        break;
    }
    }

    // Binomial crossover; coordinate j_rand always comes from the mutant.
    const Index j_rand = any_coordinate(rng);
    for (Index i = 0; i < n; ++i) {
        double v = (i == j_rand || unit(rng) < p.crossover) ? mutant(i) : population(i, j);
        // Bounce back halfway between the parent and the violated bound.
        if (v < box.lower(i)) {
            v = 0.5 * (box.lower(i) + population(i, j));
        } else if (v > box.upper(i)) {
            v = 0.5 * (box.upper(i) + population(i, j));
        }
        trials(i, j) = v;
```

Code 6.9: ParallelEvaluator: job queue, consumers, futures
`cpp/src/optimization/nongradopt/parallel_evaluator.hpp:34-77`

```cpp
[[nodiscard]] std::vector<TResult> evaluate(const std::vector<T>& inputs) const {
    std::queue<std::packaged_task<TResult()>> jobs;
    std::vector<std::future<TResult>> futures;
    futures.reserve(inputs.size());
    for (const T& input : inputs) {
        std::packaged_task<TResult()> job([this, &input] { return objective_(input); });
        futures.push_back(job.get_future());
        jobs.push(std::move(job));
    }

    std::mutex mutex;  // guards jobs
    auto consume = [&] {
        for (;;) {
            std::packaged_task<TResult()> job;
            {
                const std::lock_guard lock(mutex);
                if (jobs.empty()) {
                    return;
                }
                job = std::move(jobs.front());
                jobs.pop();
            }
            job();  // stores the result, or the exception, in the job's promise
        }
    };

    // Declared after jobs and mutex: on any exit the consumers are joined
    // (and finish the queue) before what they use is destroyed.
    std::vector<nabia::detail::JoiningThread> consumers;
    if (num_threads_ == 1) {
        consume();
    } else {
        consumers.reserve(static_cast<std::size_t>(num_threads_));
        for (int t = 0; t < num_threads_; ++t) {
            consumers.emplace_back(consume);
        }
    }
    std::vector<TResult> results;
    results.reserve(inputs.size());
    for (std::future<TResult>& f : futures) {
        results.push_back(f.get());  // waits; rethrows the job's exception
    }
    return results;
}
```

Code

parameters and defaults — `cpp/include/nabia/optimization/nongradopt/DifferentialEvolution.h:24--40`
full algorithm — `cpp/src/optimization/nongradopt/DifferentialEvolution.cpp:50--193`

Chapter 7

Risk measures

For portfolio weights x on assets with return covariance Σ and mean μ, write $\sigma(x) = \sqrt{x^\top \Sigma x}$ and take the loss $L = -r^\top x$ with $r \sim \mathcal{N}(\mu, \Sigma)$. Then $L \sim \mathcal{N}(-\mu^\top x, \sigma(x)^2)$. The three measures are:

$$\begin{aligned}
\text{Volatility:} \quad & R(x) = \sigma(x), && \nabla R = \Sigma x/\sigma(x), \\
\mathrm{VaR}_\alpha: \quad & R(x) = -\mu^\top x + z_\alpha\, \sigma(x), && \nabla R = -\mu + z_\alpha \Sigma x/\sigma(x), \\
\mathrm{ES}_\alpha: \quad & R(x) = -\mu^\top x + \frac{\varphi(z_\alpha)}{1-\alpha}\, \sigma(x), && \nabla R = -\mu + \frac{\varphi(z_\alpha)}{1-\alpha} \Sigma x/\sigma(x),
\end{aligned}$$

with $z_\alpha = \Phi^{-1}(\alpha)$.

Proposition 7.0.1 (Gaussian expected shortfall)**.** *If $L \sim \mathcal{N}(m, s^2)$, then* $\mathrm{ES}_\alpha = \mathbb{E}[L \mid L \geq \mathrm{VaR}_\alpha] = m + s\,\varphi(z_\alpha)/(1-\alpha)$.

Proof. Write $L = m + sZ$. The event is $Z \geq z_\alpha$, which has probability $1-\alpha$. Since $\varphi'(z) = -z\varphi(z)$, we get $\int_{z_\alpha}^{\infty} z\varphi(z)\, dz = \varphi(z_\alpha)$, so $\mathbb{E}[Z \mid Z \geq z_\alpha] = \varphi(z_\alpha)/(1-\alpha)$. □

Proposition 7.0.2 (Euler decomposition)**.** *Each measure is positively homogeneous of degree one, $R(cx) = cR(x)$ for $c > 0$. Hence $R(x) = \sum_i x_i\, \partial_i R(x)$, and the* risk contributions $\mathrm{RC}_i = x_i \partial_i R$ *add up exactly to the total risk.*

Proof. Differentiate $R(cx) = cR(x)$ with respect to c at $c = 1$. □

At $\sigma = 0$ the gradient of σ is set to zero, which is a valid subgradient of the seminorm. The quantile Φ^{-1} uses Acklam's rational approximation (relative error below $1.2 \cdot 10^{-9}$), refined to full precision by one Halley step on $\Phi(x) - p$:

$$x \leftarrow x - \frac{u}{1 + xu/2}, \qquad u = (\Phi(x) - p)\sqrt{2\pi}\, e^{x^2/2}.$$

Code 7.1: Volatility gradient, the building block of every measure
`cpp/src/risk/IRiskMeasure.cpp:47-55`

```
Eigen::VectorXd IRiskMeasure::portfolio_volatility_gradient(const Eigen::VectorXd& x) const {
    check_dimension(x, "portfolio");
    Eigen::VectorXd sigma_x = covariance_matrix_ * x;
    const double sigma = std::sqrt(std::max(0.0, x.dot(sigma_x)));
    if (sigma == 0.0) {
        return Eigen::VectorXd::Zero(dim());
    }
    return sigma_x / sigma;
}
```

Code 7.2: Gaussian VaR and its gradient
`cpp/src/risk/measures/ValueAtRisk.cpp:27-32`

```cpp
double ValueAtRisk::compute_risk(const Eigen::VectorXd& x) const {
    return -expected_returns_.dot(x) + z_ * portfolio_volatility(x);
}

Eigen::VectorXd ValueAtRisk::compute_marginal_risk_contributions(const Eigen::VectorXd& x) const {
    return -expected_returns_ + z_ * portfolio_volatility_gradient(x);
```

Code

risk measure interface — `cpp/include/nabia/risk/IRiskMeasure.h:18--43`
risk contributions $x \odot \nabla R$ — `cpp/src/risk/IRiskMeasure.cpp:31--33`
volatility — `cpp/src/risk/measures/Volatility.cpp:7--12`
expected shortfall (constant $\varphi(z_\alpha)/(1-\alpha)$) — `cpp/src/risk/measures/ExpectedShortfall.cpp:12--33`
normal quantile: Halley refinement — `cpp/src/stats/normal.cpp:50--53`

Chapter 8

Risk budgeting

Given budgets $b_i > 0$ with $\sum_i b_i = 1$, the *risk budgeting portfolio* is the long-only x with $\mathrm{RC}_i(x)/R(x) = b_i$ for all i. Equal budgets give the equal risk contribution portfolio.

Theorem 8.0.1. *Let R be convex, positively homogeneous of degree one, differentiable on $y > 0$, and positive on $\{y \geq 0,\, y \neq 0\}$. Then*

$$f(y) = R(y) - \sum_i b_i \log y_i$$

has a unique minimizer $y^* > 0$, and $x^* = y^*/\sum_i y_i^*$ is the unique long-only portfolio with $\mathrm{RC}_i(x^*)/R(x^*) = b_i$. Moreover $R(y^*) = 1$.

Proof. Existence. R is continuous and positive on the compact simplex $\Delta = \{y \geq 0, \sum y_i = 1\}$, so $c = \min_\Delta R > 0$. By homogeneity, $R(y) \geq c \sum_i y_i$ for $y \geq 0$. Hence $f(y) \geq \sum_i (c y_i - b_i \log y_i)$. Each term tends to $+\infty$ as $y_i \to 0^+$ or $y_i \to \infty$, so f is coercive on $y > 0$ and attains its minimum.

Uniqueness. $-\sum b_i \log y_i$ is strictly convex and R is convex, so f is strictly convex.

Characterization. At the minimizer, $\partial_i f = \partial_i R(y^*) - b_i/y_i^* = 0$, so $\mathrm{RC}_i(y^*) = y_i^* \partial_i R(y^*) = b_i$. Summing and using Euler gives $R(y^*) = \sum_i b_i = 1$. Let $S = \sum_i y_i^*$. Because $\partial_i R$ is homogeneous of degree zero, $x^* = y^*/S$ has $\mathrm{RC}_i(x^*) = b_i/S$ and $R(x^*) = 1/S$, so $\mathrm{RC}_i/R = b_i$.

Conversely, if x satisfies the budget condition, then $y = x/R(x)$ satisfies $\partial_i f(y) = 0$ and is therefore the unique minimizer. □

For VaR and ES, the condition "$R > 0$ on long-only portfolios" needs $z\,\sigma(x) > \mu^\top x$. The solver checks it at the start and throws otherwise.

Algorithm. The steps follow `cpp/src/risk/RiskBudgeting.cpp:45--111`.

1. **Normalize** the budgets. Empty budgets mean equal risk.

2. **Start** at $y_i^0 \propto b_i/\sigma_i$, scaled so that $R(y^0) = 1$. That scale is the magnitude of the solution by the theorem. The start is the inverse-volatility portfolio for equal budgets, which is exact for uncorrelated assets.

3. **Minimize** f with L-BFGS (memory 20). Points with any $y_i \leq 0$ return $+\infty$, so the log barrier needs no explicit constraint.

4. **Match the tolerance to the budget error.** L-BFGS stops on $\|\nabla f\|_\infty$, but the target is the budget error $e = \max_i |\mathrm{RC}_i/R - b_i|$. Near the solution $R \approx 1$ and $\mathrm{RC}_i - b_i = y_i \nabla_i f$, so $e \lesssim \max_i y_i \, \|\nabla f\|_\infty$. The inner tolerance therefore starts at $\tau/\max_i y_i^0$. If $e > \tau$ afterwards, L-BFGS warm-restarts with tolerance $\tau_{\text{in}} \cdot \frac{1}{2}\tau/e$, at most five times.

5. **Return** $x = y / \sum y$, with risk, contributions, budget error and a convergence flag.

Code 8.1: Objective and gradient, for any risk measure
`cpp/src/risk/RiskBudgeting.cpp:65-73`

```cpp
// f(y) = R(y) - b' log(y),  grad f(y) = dR/dy - b ./ y,  y_i grad_i = RC_i(y) - b_i.
// Points with some y_i <= 0 get +inf, which the line search rejects.
const LBFGS::ValueAndGradient objective = [&](const VectorXd& y, VectorXd& grad) {
    if ((y.array() <= 0.0).any()) {
        grad.setZero();
        return std::numeric_limits<double>::infinity();
    }
    grad = measure.compute_marginal_risk_contributions(y) - b.cwiseQuotient(y);
    return measure.compute_risk(y) - b.dot(VectorXd(y.array().log()));
```

Code 8.2: Tolerance matching on the budget error
`nabia_python/nabia/sentiment_rb/portfolio/backtest.py:32-38`

```python
def solve(cfg, cov, b, d):
    """nabia risk budgeting on D Sigma D: weights D w' and relative risk contributions (= b)."""
    vol = risk.Volatility(d[:, None] * cov * d[None, :])
    P = cfg["portfolio"]
    x = np.asarray(risk.RiskBudgeting(risk.RiskBudgetingParams(budgets=b, tolerance=P.get("
     ↪ solver_tolerance", 1e-8),
                                                               max_iter=P.get("solver_max_iter",
     ↪ 5000)), vol).optimize())
    return d * x, np.asarray(vol.compute_risk_contributions(x)) / vol.compute_risk(x)
```

Code

budget normalization — `cpp/src/risk/RiskBudgeting.cpp:19--30`
starting point — `cpp/src/risk/RiskBudgeting.cpp:34--41`
`RiskBudgeting<Measure>` wrapper — `cpp/include/nabia/risk/RiskBudgeting.h:51--69`

Chapter 9

The walk-forward portfolio engine

`RiskBudgetingPortfolio` backtests a long-only risk budgeting portfolio from a $T \times N$ matrix of simple returns, where row t holds the returns over $(t-1, t]$. Table 9.1 lists its parameters, which are also the JSON keys of the CLI and the Python API.

Table 9.1: Portfolio parameters: engine defaults and the defaults of the AI configuration.

Key	Engine	AI	Meaning
`covariance_window`	60	120	rows W per covariance estimate
`covariance_estimator`	sample	exponential	equal or exponential weights
`ewma_halflife`	20	40	half-life h in rows (exponential)
`budgets`	equal	set by policy	risk budgets per asset
`risk_measure`	volatility	volatility	volatility, `value_at_risk`, `expected_shortfall`
`confidence_level`	0.99	–	VaR/ES level in $(0.5, 1)$
`use_expected_returns`	false	–	VaR/ES: window mean instead of 0
`target_volatility`	none	0.10	annualized target σ^*
`max_leverage`	2.0	3.0	cap $L_{\max}$ on $\sum_i w_i$
`rebalance_frequency`	1	21	trade every k rows
`transaction_cost`	0	0.0005	cost κ per unit traded (5 bp)
`drift_weights`	true	true	weights drift between trades
`initial_capital`	1	10^6	starting equity
`periods_per_year`	252	252	P, for annualization
`solver_tolerance`	10^{-8}	–	budget error target τ
`solver_max_iter`	5000	–	L-BFGS iteration budget
`num_threads`	0 (all)	–	threads for the solves

Code

parameter struct — `cpp/include/nabia/strategy/RiskBudgetingPortfolio.h:31--57`
validation — `cpp/src/strategy/RiskBudgetingPortfolio.cpp:110--136`
the engine — `cpp/src/strategy/RiskBudgetingPortfolio.cpp:138--267`

9.1 Timing and absence of look-ahead

Rebalances happen at the close of rows $t_k = W - 1 + kf$ for $k = 0, \dots, \lfloor (T-W)/f \rfloor$. At rebalance t the engine:

1. estimates moments from rows $t - W + 1, \dots, t$, all known at the close of t;
2. solves for the risk budgeting weights and scales them to the volatility target;
3. trades from the drifted weights to the targets.

The new weights earn the returns of rows $t+1$ onwards. A test confirms there is no look-ahead: overwriting the data from row 200 on leaves every weight up to row 200 bit-identical.

9.2 Covariance estimation

For the window's returns $r_1, \dots, r_W$ (oldest first) and weights $w_k \geq 0$ with $\sum w_k = 1$:

$$\hat{\mu} = \sum_k w_k r_k, \qquad \hat{\Sigma} = \frac{\sum_k w_k (r_k - \hat{\mu})(r_k - \hat{\mu})^\top}{1 - \sum_k w_k^2}.$$

The weights are either equal ($w_k = 1/W$) or exponential: $w_k \propto \lambda^{W-k}$ with $\lambda = 2^{-1/h}$, so the newest return has age 0 and weight 1 before normalization.

Proposition 9.2.1. *For i.i.d. returns with covariance Σ, $\mathbb{E}\hat{\Sigma} = \Sigma$. With equal weights the denominator is $(W-1)/W$, which recovers the usual $1/(W-1)$ estimator.*

Proof. Write $r_k - \hat{\mu} = \sum_j (\delta_{kj} - w_j)(r_j - \mu)$. By independence, $\mathbb{E}\big[(r_k - \hat{\mu})(r_k - \hat{\mu})^\top\big] = \sum_j (\delta_{kj} - w_j)^2 \Sigma = (1 - 2w_k + \sum_j w_j^2)\Sigma$. Weighting by w_k and summing gives $(1 - \sum_j w_j^2)\Sigma$. □

The quantity $1/\sum_k w_k^2$ is the effective number of observations.

Code 9.1: Window moments
`cpp/src/strategy/RiskBudgetingPortfolio.cpp:35-53`

```cpp
WindowMoments estimate(const RiskBudgetingPortfolioParams& p, const MatrixXd& window) {
    const Index W = window.rows();
    VectorXd w = VectorXd::Constant(W, 1.0 / static_cast<double>(W));
    if (p.covariance_estimator == CovarianceEstimator::Exponential) {
        // Weight lambda^age with lambda = 2^(-1 / halflife); the newest row has age 0.
        const double lambda = std::exp2(-1.0 / p.ewma_halflife);
        for (Index i = 0; i < W; ++i) {
            w(i) = std::pow(lambda, static_cast<double>(W - 1 - i));
        }
        w /= w.sum();
    }
    WindowMoments m;
    m.mean = window.transpose() * w;
    const MatrixXd centred = window.rowwise() - m.mean.transpose();
    // Unbiased weighted covariance: divide by 1 - sum(w^2) (= (W - 1) / W for equal weights).
    m.covariance = centred.transpose() * w.asDiagonal() * centred / (1.0 - w.squaredNorm());
    m.covariance = 0.5 * (m.covariance + m.covariance.transpose());
    return m;
}
```

9.3 Allocation, budget schedule and volatility targeting

Budget schedule. The budgets in force at rebalance row t are the last *budget change* with first row at or before t. A schedule row dated d takes effect from the first returns row dated on or after d (a binary search). This is how time-varying budgets (from a rule, or from the sentiment pipeline of Part VII) enter one continuous backtest. Without a schedule, the parameter budgets apply.

Volatility targeting. The risk budgeting weights x (summing to 1) are scaled by the leverage

$$L = \min\Big(L_{\max}, \; \frac{\sigma^*}{\sqrt{x^\top \hat{\Sigma} x \cdot P}}\Big),$$

so the predicted annualized volatility is σ^* unless the cap binds. Any weight not invested, $1 - \sum_i w_i$, is cash earning zero.

Code 9.2: Schedule dates to first rows
`cpp/include/nabia/strategy/RiskBudgetingPortfolio.h:166-168`

```
for (Eigen::Index s = 0; s < budget_schedule.rows(); ++s) {
    const auto first = std::lower_bound(index.begin(), index.end(), budget_schedule.time(s));
    schedule.push_back({static_cast<Eigen::Index>(first - index.begin()), budget_schedule.values().row(
    ↪ s).transpose()});
```

Code 9.3: Budgets in force at a rebalance
`cpp/src/strategy/RiskBudgetingPortfolio.cpp:164-174`

```
// Budgets in force at a rebalance row: the last change at or before it.
auto budgets_at = [&](Index row) -> const VectorXd& {
    const VectorXd* b = &default_budgets;
    for (const BudgetChange& change : schedule) {
        if (change.first_row > row) {
            break;
        }
        b = &change.budgets;
    }
    return *b;
};
```

Code 9.4: Target weights of one rebalance date
`cpp/src/strategy/RiskBudgetingPortfolio.cpp:92-106`

```
Target solve_target(const RiskBudgetingPortfolioParams& p, const VectorXd& budgets, const MatrixXd&
    ↪ window,
                    const std::optional<VectorXd>& warm_start) {
    const WindowMoments m = estimate(p, window);
    const RiskBudgetingResult rb = allocate(p, budgets, m, warm_start);
    Target target;
    if (p.target_volatility) {
        const double ppy = static_cast<double>(p.periods_per_year);
        const double predicted = std::sqrt(std::max(0.0, rb.weights.dot(m.covariance * rb.weights)) *
    ↪ ppy);
        target.leverage = predicted > 0.0 ? std::min(p.max_leverage, *p.target_volatility / predicted)
    ↪ : p.max_leverage;
    }
    target.weights = target.leverage * rb.weights;
    target.contributions = rb.relative_contributions;
    target.converged = rb.converged;
    return target;
}
```

Code

risk measure selection — `cpp/src/strategy/RiskBudgetingPortfolio.cpp:67--79`

9.4 Phase 1: parallel solves

The targets of all rebalance dates depend only on their own windows, so they are solved first, in parallel. Each solve warm-starts from the previous date's unlevered weights, which cuts iterations sharply because consecutive windows differ by a few rows. Warm starts make solves depend on each other, so dates are grouped into *fixed* blocks of 8 consecutive rebalances:

- one block is one task;
- warm starts happen only within a block;
- the block boundaries do not depend on the thread count.

Results are therefore bit-identical for any number of threads. With 50 assets, daily rebalancing and 10 years of data (2,271 solves), a backtest takes 375 ms on one thread and 53 ms on eight.

Code 9.5: Phase 1

`cpp/src/strategy/RiskBudgetingPortfolio.cpp:192-208`

```
// Phase 1 (parallel): target weights of every rebalance date. Each task
// solves one block of consecutive dates, warm-starting within the block.
std::vector<Target> targets(static_cast<std::size_t>(n_rebalances));
const Index n_blocks = (n_rebalances + kWarmStartBlock - 1) / kWarmStartBlock;
const int threads = nabia::detail::resolve_threads(p.num_threads, n_blocks);
nabia::detail::parallel_for(n_blocks, threads, [&](std::int64_t block) {
    std::optional<VectorXd> warm_start;
    const Index first = block * kWarmStartBlock;
    const Index last = std::min(n_rebalances, first + kWarmStartBlock);
    for (Index k = first; k < last; ++k) {
        const Index t = c.rebalance_rows[static_cast<std::size_t>(k)];
        Target target = solve_target(p, budgets_at(t), returns.middleRows(t - W + 1, W), warm_start);
        warm_start = target.weights / target.leverage;  // the unlevered risk budgeting weights
        targets[static_cast<std::size_t>(k)] = std::move(target);
    }
});
c.threads_used = threads;
```

Code 9.6: Thread pool helper

`cpp/src/common/parallel_for.hpp:29-59`

```
void parallel_for(std::int64_t count, int threads, const Task& task) {
    std::atomic<std::int64_t> next{0};
    std::mutex error_mutex;
    std::exception_ptr error;
    std::int64_t error_index = count;

    auto work = [&] {
        for (std::int64_t i = next++; i < count; i = next++) {
            try {
                task(i);
            } catch (...) {
                const std::lock_guard lock(error_mutex);
                if (i < error_index) {
                    error_index = i;
                    error = std::current_exception();
                }
            }
        }
    };
    {
        std::vector<JoiningThread> workers;
        workers.reserve(static_cast<std::size_t>(std::max(0, threads - 1)));
        for (int t = 1; t < threads; ++t) {
            workers.emplace_back(work);
        }
        work();
    }  // joining the workers makes all their writes visible here
    if (error) {
        std::rethrow_exception(error);
    }
}
```

Code

thread count (0 = hardware concurrency, capped by tasks) — `cpp/src/common/parallel_for.hpp:18--22`

9.5 Phase 2: drift, trading and accounting

Phase 2 walks through time. With held weights w_t and asset returns r_t, each row does the following:

$$
\begin{aligned}
&g_t = w_t^\top r_t && \text{gross return,}\\
&w \leftarrow w_t \odot (1 + r_t)/(1 + g_t) && \text{drift,}\\
&\tau_t = \|w^{\text{target}} - w\|_1, \quad c_t = \kappa\, \tau_t && \text{turnover and cost (rebalance rows),}\\
&n_t = (1 + g_t)(1 - c_t) - 1, \quad E_t = E_{t-1}(1 + n_t) && \text{net return and equity.}
\end{aligned}
$$

Drift. Holding i is worth $w_i E$ and grows to $w_i E(1 + r_i)$. Cash earns nothing, so equity grows to $E(1 + g)$. The new weight is the ratio of the two.

Costs. They are paid out of equity at the close, after the period's return, which gives $E(1 + g)(1 - c)$. The first trade buys the whole portfolio from cash, so its turnover is $\sum_i w_i$.

Checks. The engine stops with an error if $1 + g_t \le 0$ (the portfolio lost all its capital). These identities are checked to 10^{-15} in the tests.

Code 9.7: Phase 2
cpp/src/strategy/RiskBudgetingPortfolio.cpp:210-254

```cpp
// Phase 2 (sequential): drift, trade and account through time.
VectorXd w = VectorXd::Zero(N);  // current weights (fractions of equity)
double equity = p.initial_capital;
const double ppy = static_cast<double>(p.periods_per_year);
Index next = 0;  // next rebalance

for (Index t = 0; t < T; ++t) {
    // 1. Earn this period's returns with the weights set at the previous close.
    c.weights.row(t) = w.transpose();
    const VectorXd r = returns.row(t).transpose();
    const double gross = w.dot(r);
    if (!(1.0 + gross > 0.0)) {
        throw std::invalid_argument("RiskBudgetingPortfolio: the portfolio lost all its capital at row
    ↪ " +
                                    std::to_string(t));
    }
    if (p.drift_weights) {
        // Each holding grows by (1 + r_i), equity by (1 + gross); cash is the remainder.
        w = w.cwiseProduct((VectorXd::Ones(N) + r)) / (1.0 + gross);
    }

    // 2. Rebalance at the close, with the data known at t.
    double cost = 0.0;
    if (next < n_rebalances && c.rebalance_rows[static_cast<std::size_t>(next)] == t) {
        const Target& target = targets[static_cast<std::size_t>(next)];
        const double traded = (target.weights - w).lpNorm<1>();
        cost = p.transaction_cost * traded;
        w = target.weights;

        c.target_weights.row(next) = target.weights.transpose();
        c.risk_contributions.row(next) = target.contributions.transpose();
        c.leverage(next) = target.leverage;
        c.all_converged = c.all_converged && target.converged;
        c.turnover(t) = traded;
        ++next;
    }

    // 3. Accounting: costs are paid out of the equity at the close.
    const double net = (1.0 + gross) * (1.0 - cost) - 1.0;
    const double new_equity = equity * (1.0 + net);
    c.gross_returns(t) = gross;
    c.costs(t) = cost;
    c.net_returns(t) = net;
```

```
    c.pnl(t) = new_equity - equity;
    c.equity(t) = new_equity;
    equity = new_equity;
```

Chapter 10

Performance metrics

The metrics use the net returns $n_1, \dots, n_T$ from the first trade on, with P periods per year and equity E starting at 1.

Metric	Definition
Total return	$\prod_t (1 + n_t) - 1$
Annualized return (CAGR)	$(1 + \text{total})^{P/T} - 1$
Annualized volatility	$\text{sd}(n)\sqrt{P}$ (sample standard deviation)
Sharpe ratio	$\bar{n}\,P / (\text{sd}(n)\sqrt{P})$ (zero risk-free rate)
Sortino ratio	$\bar{n}\,P / \left(\sqrt{\frac{1}{T}\sum_t \min(n_t, 0)^2}\,\sqrt{P}\right)$
Maximum drawdown	$\max_t \left(1 - E_t / \max_{s \le t} E_s\right)$
Max drawdown duration	longest run of periods below a previous peak
Calmar ratio	annualized return / maximum drawdown
Hit rate	fraction of periods with $n_t > 0$
Best / worst period	$\max_t n_t$, $\min_t n_t$
Turnover	average τ per rebalance; annualized $\sum_t \tau_t \, P/T$
Total costs	$\sum_t c_t$

Code 10.1: Performance metrics
`cpp/src/strategy/PerformanceMetrics.cpp:31-67`

```cpp
PerformanceMetrics compute_performance(const Eigen::VectorXd& returns, int periods_per_year) {
    if (returns.size() == 0 || !returns.allFinite()) {
        throw std::invalid_argument("compute_performance: returns must be non-empty and finite");
    }
    if (periods_per_year <= 0) {
        throw std::invalid_argument("compute_performance: periods_per_year must be positive");
    }
    const auto n = static_cast<double>(returns.size());
    const double ppy = static_cast<double>(periods_per_year);
    PerformanceMetrics m;
    m.periods = returns.size();

    const Eigen::VectorXd equity = equity_curve(returns, 1.0);
    m.total_return = equity(equity.size() - 1) - 1.0;
    m.annualized_return = m.total_return > -1.0 ? std::pow(1.0 + m.total_return, ppy / n) - 1.0 : -1.0;

    const double mean = returns.mean();
    const double var = returns.size() > 1 ? (returns.array() - mean).square().sum() / (n - 1.0) : 0.0;
    m.annualized_volatility = std::sqrt(var * ppy);
    m.sharpe_ratio = m.annualized_volatility > 0.0 ? mean * ppy / m.annualized_volatility : 0.0;
    const double downside = std::sqrt(returns.cwiseMin(0.0).squaredNorm() / n) * std::sqrt(ppy);
    m.sortino_ratio = downside > 0.0 ? mean * ppy / downside : 0.0;

    const Eigen::VectorXd dd = drawdown_series(equity, 1.0);
    m.max_drawdown = dd.maxCoeff();
    Eigen::Index run = 0;
    for (Eigen::Index t = 0; t < dd.size(); ++t) {
        run = dd(t) > 0.0 ? run + 1 : 0;
```

```
        m.max_drawdown_duration = std::max(m.max_drawdown_duration, run);
    }
    m.calmar_ratio = m.max_drawdown > 0.0 ? m.annualized_return / m.max_drawdown : 0.0;

    m.hit_rate = static_cast<double>((returns.array() > 0.0).count()) / n;
    m.best_period = returns.maxCoeff();
    m.worst_period = returns.minCoeff();
    return m;
}
```

Code

drawdown series — `cpp/src/strategy/PerformanceMetrics.cpp:21--29`

Chapter 11

Elastic net factor regression

The regression module regresses a target series (a fund, a strategy, an asset) on candidate factors. It chooses the factors, the penalty and the mix, and measures their uncertainty. Typical uses are replicating a fund with tradable factors, explaining a strategy's returns, and hedging. Chapter 16 has every proof. This section states the results and maps them to the code.

11.1 The elastic net

For a design $X \in \mathbb{R}^{n\times p}$ and a target y, with $\ell_1 = \lambda\alpha$ and $\ell_2 = \lambda(1-\alpha)$, the solver minimizes the glmnet objective

$$\frac{1}{2n}\|y - b_0\mathbf{1} - Xb\|^2 + \ell_1\|b\|_1 + \frac{\ell_2}{2}\|b\|_2^2.$$

$\alpha = 1$ is the lasso, $\alpha = 0$ ridge, and $\lambda = 0$ least squares.

- **Intercept.** It is unpenalized, and its optimum is $b_0 = \bar{y} - \bar{x}^\top b$. The solver works on centred data.
- **Standardization.** Each centred column is divided by its population standard deviation s_j, and the coefficients are reported as $b_j = \beta_j/s_j$. The penalty therefore charges a factor in proportion to its volatility, so units and leverage do not change the selection. Constant columns get $b_j = 0$.

Proposition 11.1.1 (Coordinate update)**.** *With working columns z_j, $v_j = \|z_j\|^2/n$ and residual r, the exact minimizer over β_j with the others fixed is*

$$\beta_j \leftarrow \frac{\mathcal{S}(\rho_j, \ell_1)}{v_j + \ell_2}, \qquad \rho_j = \tfrac{1}{n} z_j^\top r + v_j\beta_j, \qquad \mathcal{S}(z,t) = \operatorname{sign}(z)\max(|z| - t, 0).$$

Proof. In β_j alone the objective is $\frac{v_j+\ell_2}{2}\beta_j^2 - \rho_j\beta_j + \ell_1|\beta_j|$ plus a constant. Its subgradient condition $0 \in (v_j + \ell_2)\beta_j - \rho_j + \ell_1\partial|\beta_j|$ has the stated solution in each of the cases $\rho_j > \ell_1$, $\rho_j < -\ell_1$ and $|\rho_j| \le \ell_1$. □

Each update costs $O(n)$: the residual is corrected by $-z_j\Delta_j$. Cyclic coordinate descent converges because the non-smooth term is separable (Tseng, 2001).

- **Active set.** After a full sweep, the solver sweeps only the non-zero coefficients until they settle, then checks with a full sweep.
- **Stopping rule.** The solver stops when a full sweep moves no fitted value by more than a fraction τ of the target's variance: $\max_j v_j\Delta_j^2 < \tau\|\tilde{y}\|^2/n$.

Proposition 11.1.2 ($\lambda_{\max}$)**.** *For $\alpha > 0$, $\beta = 0$ is optimal if and only if $\lambda \ge \lambda_{\max} = \max_j |z_j^\top \tilde{y}|/(n\alpha)$.*

Proof. At $\beta = 0$ the residual is $\tilde{y}$, and the optimality condition of an inactive coefficient is $|z_j^\top r|/n \le \ell_1$. □

- **Floor on α.** As in glmnet, α is floored at 10^{-3}, so ridge also gets a finite path.
- **Rounding.** The value is multiplied by $1 + 10^{-12}$, so rounding cannot leave a 10^{-17} coefficient at $\lambda_{\max}$.
- **Grid.** The automatic grid is $\lambda_i = \lambda_{\max} r^{i/(L-1)}$. It is solved from the largest λ down, each fit warm-started from the previous one.

Degrees of freedom. With active set A and locally constant signs, the active coefficients are an affine function of y: $\beta_A = (Z_A^\top Z_A + n\ell_2 I)^{-1}(Z_A^\top \tilde{y} - n\ell_1 s_A)$. By Stein's lemma, the degrees of freedom are the trace of the hat matrix:

$$\mathrm{df} = 1 + \mathrm{tr}\big(Z_A(Z_A^\top Z_A + n\ell_2 I)^{-1} Z_A^\top\big) = 1 + \sum_i \frac{d_i}{d_i + n\ell_2},$$

where the d_i are the eigenvalues of $Z_A^\top Z_A$ and the 1 counts the intercept. The lasso gives $|A| + 1$ (Zou, Hastie and Tibshirani, 2007).

Grouping. For standardized factors i, j with correlation ρ and coefficients of the same sign, the optimality conditions give $|\beta_i - \beta_j| \le \|\tilde{y}\|\sqrt{2(1-\rho)}/(\sqrt{n}\,\ell_2)$. Near-duplicate factors share a loading instead of one of them taking it arbitrarily. This is why $\alpha < 1$ helps with substitutes such as the S&P 500 and the Nasdaq 100.

Code 11.1: One coordinate descent sweep
`cpp/src/regression/ElasticNet.cpp:124-143`

```cpp
double sweep(bool active_only) {
    const auto n = static_cast<double>(s_.X.rows());
    double max_change = 0.0;
    for (Index j = 0; j < beta_.size(); ++j) {
        const double vj = s_.v(j);
        if (vj == 0.0 || (active_only && beta_(j) == 0.0)) {
            continue;
        }
        const double old = beta_(j);
        const double z = s_.X.col(j).dot(residual_) / n + vj * old;
        const double updated = soft_threshold(z, l1_) / (vj + l2_);
        const double delta = updated - old;
        if (delta != 0.0) {
            residual_ -= delta * s_.X.col(j);
            beta_(j) = updated;
            max_change = std::max(max_change, vj * delta * delta);
        }
    }
    return max_change;
}
```

Code 11.2: $\lambda_{\max}$ with the same arithmetic as the first update
`cpp/src/regression/ElasticNet.cpp:211-227`

```cpp
double lambda_max(const MatrixXd& X, const VectorXd& y, double alpha, bool fit_intercept, bool
    ↪ standardize) {
    validate_data(X, y);
    if (!(alpha >= 0.0 && alpha <= 1.0)) {
        throw std::invalid_argument("lambda_max: alpha must be in [0, 1]");
    }
    if (X.cols() == 0) {
        return 0.0;
    }
    const Standardized s = standardize_data(X, y, fit_intercept, standardize);
```

```
    // Same arithmetic as the first coordinate update, plus a few ulps so that
    // lambda * alpha >= |gradient| survives the rounding of the division.
    double gradient = 0.0;
    for (Index j = 0; j < X.cols(); ++j) {
        gradient = std::max(gradient, std::abs(s.X.col(j).dot(s.y) / static_cast<double>(X.rows())));
    }
    return gradient / std::max(alpha, kMinAlphaForLambdaMax) * (1.0 + 1e-12);
}
```

Code

parameters — `cpp/include/nabia/regression/ElasticNet.h:11--18`
centring and scaling — `cpp/src/regression/ElasticNet.cpp:48--77`
solver loop: full and active-set sweeps — `cpp/src/regression/ElasticNet.cpp:90--150`
path with warm starts — `cpp/src/regression/ElasticNet.cpp:251--275`
effective degrees of freedom — `cpp/src/regression/ElasticNet.cpp:153--175`

11.2 The regression engine

`RegressionEngine` runs the following steps (`cpp/src/regression/RegressionEngine.cpp:472--555`):

1. **Split** the T rows in time order. The first $n = \lfloor fT + 10^{-9} \rfloor$ rows train, and the rest test. The test set is an extrapolation. With $f = 1$ the engine is in sample.
2. For each candidate subset S of $k_{\min}..k_{\max}$ factors:
   (a) choose (α, λ) by time-series cross-validation on the training rows;
   (b) fit on all training rows, warm-started down the grid;
   (c) score by the test MSE, or in sample by BIC $= n \log \mathrm{MSE} + \mathrm{df} \log n$ or AIC $= n \log \mathrm{MSE} + 2\,\mathrm{df}$, which are $-2 \log \hat{L}$ under Gaussian errors plus the penalty, with constants dropped.
3. **Keep** the lowest score. Optionally refit on all rows, and bootstrap.

Cross-validation. The training rows are cut into contiguous blocks with boundaries $\lfloor kn/B \rfloor$, and g rows are left out on each side of a validation block.

- *Blocked K-fold* ($B = K$) trains on the rows before and after the validation block.
- *Walk-forward* ($B = K + 1$) trains only on the rows before it.

The estimate is the mean fold MSE, and its standard error is $\sqrt{\widehat{\mathrm{Var}}_k(m_k)/K}$. The cell with the lowest mean wins. The one-standard-error rule then moves, within that α, to the largest λ whose error is within one standard error of the minimum, which gives a sparser model that is statistically as good.

Subset search.

- *Exhaustive* evaluates $\sum_k \binom{N}{k}$ subsets (1 470 for 14 factors and sizes 1–4).
- *Forward* adds, greedily, the factor that most improves the score, which takes $\sum_{k \le k_{\max}} (N - k + 1)$ evaluations.
- *Auto* chooses Exhaustive below `max_exhaustive_subsets`.

The test MSE of the winner of many subsets is optimistic, because a factor can win by noise. The bootstrap below exposes such factors, and a later holdout gives a clean figure.

Moving block bootstrap. Each of B samples:

1. concatenates training-row blocks of length ℓ with uniform starts until there are n rows;
2. refits the selected subset at the selected (α, λ);
3. records the coefficients, the intercept and the test MSE.

Blocks keep the serial dependence of returns, such as volatility clustering (Künsch, 1989). The summaries are:

- the mean and standard error;
- percentile intervals (linear interpolation at $q(B-1)$);
- selection frequencies, the share of samples with a non-zero coefficient;
- the spread of the test MSE.

The intervals are conditional on the selected model, so they do not include selection uncertainty.

Determinism. Subsets and bootstrap samples run on the thread pool of §9. Each task writes its own slot, ties go to the first index, and sample b seeds its own `mt19937_64` from `seed_seq(seed, b)`. Results are bit-identical for any thread count.

Code 11.3: Contiguous validation folds with a gap
`cpp/src/regression/RegressionEngine.cpp:71-94`

```
std::vector<Fold> make_folds(const RegressionEngineParams& p, Index n) {
    const Index K = p.validation_folds;
    const Index blocks = p.validation == ValidationScheme::BlockedKFold ? K : K + 1;
    const auto bound = [&](Index i) { return i * n / blocks; };
    std::vector<Fold> folds;
    const Index first = p.validation == ValidationScheme::BlockedKFold ? 0 : 1;
    for (Index k = first; k < blocks; ++k) {
        const Index begin = bound(k);
        const Index end = bound(k + 1);
        Fold f;
        f.validation = range(begin, end);
        f.train = range(0, begin - p.validation_gap);
        if (p.validation == ValidationScheme::BlockedKFold) {
            const std::vector<Index> after = range(end + p.validation_gap, n);
            f.train.insert(f.train.end(), after.begin(), after.end());
        }
        if (f.validation.empty() || f.train.size() < 2) {
            throw std::invalid_argument("RegressionEngine: " + std::to_string(n) +
                                        " training rows are too few for the validation folds and gap");
        }
        folds.push_back(std::move(f));
    }
    return folds;
}
```

Code 11.4: Cross-validation error, standard error and the 1-SE rule
`cpp/src/regression/RegressionEngine.cpp:144-191`

```
if (pb.tune) {
    MatrixXd sum = MatrixXd::Zero(A, L);
    MatrixXd sum_sq = MatrixXd::Zero(A, L);
    for (const Fold& fold : pb.folds) {
        const MatrixXd Xf = X(fold.train, all);
        const VectorXd yf = pb.y_train(fold.train);
        const MatrixXd Xv = X(fold.validation, all);
        const VectorXd yv = pb.y_train(fold.validation);
        for (Index a = 0; a < A; ++a) {
            const auto ai = static_cast<std::size_t>(a);
            const std::vector<ElasticNetFit> path =
```

```
                elastic_net_path(Xf, yf, solver_settings(p, p.alphas[ai]), grids[ai]);
            for (Index l = 0; l < L; ++l) {
                const ElasticNetFit& fit = path[static_cast<std::size_t>(l)];
                const double m = mse(yv, fit.predict(Xv));
                sum(a, l) += m;
                sum_sq(a, l) += m * m;
                e.converged = e.converged && fit.converged;
            }
        }
    }
    const auto K = static_cast<double>(pb.folds.size());
    const MatrixXd mean = sum / K;
    MatrixXd se = MatrixXd::Zero(A, L);
    if (pb.folds.size() > 1) {
        const MatrixXd var = ((sum_sq / K - mean.cwiseProduct(mean)) * (K / (K - 1.0))).cwiseMax(0.0);
        se = (var / K).cwiseSqrt();
    }
    for (Index a = 0; a < A; ++a) {
        for (Index l = 0; l < L; ++l) {
            if (mean(a, l) < mean(best_a, best_l)) {
                best_a = a;
                best_l = l;
            }
        }
    }
    if (p.one_standard_error_rule) {
        // Lambdas decrease along the row: the first one within the band is the largest.
        const double threshold = mean(best_a, best_l) + se(best_a, best_l);
        for (Index l = 0; l <= best_l; ++l) {
            if (mean(best_a, l) <= threshold) {
                best_l = l;
                break;
            }
        }
    }
    e.validation_mse = mean(best_a, best_l);
    e.validation_se = se(best_a, best_l);
```

Code 11.5: One moving block bootstrap sample
cpp/src/regression/RegressionEngine.cpp:364-378

```
// One generator per sample, seeded by (seed, b): independent of the thread count.
std::seed_seq seq{static_cast<std::uint32_t>(p.bootstrap_seed), static_cast<std::uint32_t>(p.
    ↪ bootstrap_seed >> 32),
                  static_cast<std::uint32_t>(b), static_cast<std::uint32_t>(static_cast<std::uint64_t>(
    ↪ b) >> 32)};
std::mt19937_64 rng(seq);
std::uniform_int_distribution<Index> start(0, n - block);
std::vector<Index> rows;
rows.reserve(static_cast<std::size_t>(n));
while (static_cast<Index>(rows.size()) < n) {
    const Index s0 = start(rng);
    for (Index i = 0; i < block && static_cast<Index>(rows.size()) < n; ++i) {
        rows.push_back(s0 + i);
    }
}
const ElasticNetFit fit =
    elastic_net_path(X(rows, all), pb.y_train(rows), settings, {best.lambda}).front();
```

Code

parameters (defaults and meaning) — cpp/include/nabia/regression/RegressionEngine.h:39--80
engine class, index-free and DataFrame interfaces — cpp/include/nabia/regression/RegressionEngine.h:183--222
evaluation of one subset — cpp/src/regression/RegressionEngine.cpp:127--231
BIC, AIC, MSE — cpp/src/regression/RegressionEngine.cpp:106--118
exhaustive search — cpp/src/regression/RegressionEngine.cpp:284--308
forward search — cpp/src/regression/RegressionEngine.cpp:310--331
bootstrap and its summaries — cpp/src/regression/RegressionEngine.cpp:345--417

Part IV

Derivations

Chapter 12

Simple exponential smoothing

This note documents `nabia::forecast::SimpleExponentialSmoothing`. We define the recursion and prove that it is an exponentially weighted average whose weights sum to one. We compute its mean lag and derive the span convention $\alpha = 2/(N+1)$. We then give the statistical model behind it, the innovations state-space model ETS(A,N,N), and derive the forecast distribution, including the variance $\sigma^2(1+(h-1)\alpha^2)$. We also prove its equivalence with ARIMA(0,1,1) and with the steady-state Kalman filter of the local level model. Finally, we describe the implementation: initialization, variance estimate, prediction intervals and complexity.

12.1 The recursion

Let $y_0, \dots, y_{T-1}$ be an equally spaced series and $\alpha \in (0, 1]$ the smoothing factor. Simple exponential smoothing (SES) [3] maintains a *level* ℓ_t, updated as

$$\ell_t = \alpha\, y_t + (1-\alpha)\, \ell_{t-1}, \qquad t = 0, \dots, T-1, \tag{12.1}$$

from an initial level ℓ_{-1}. Equivalently, in *error-correction form*,

$$e_t = y_t - \ell_{t-1}, \qquad \ell_t = \ell_{t-1} + \alpha\, e_t, \tag{12.2}$$

where e_t is the one-step-ahead forecast error: ℓ_{t-1} is the forecast of y_t made at time $t-1$. Every forecast made at the end of the sample equals the final level,

$$\hat{y}_{T-1+h|T-1} = \ell_{T-1}, \qquad h = 1, 2, \dots$$

SES is therefore suited to series without trend or seasonality: it estimates the current level and extrapolates it flat. The value of α sets the trade-off between reacting to recent data ($\alpha \to 1$, the naive forecast $\ell_t = y_t$) and smoothing out noise ($\alpha \to 0$, an almost constant level).

12.1.1 Exponential weights

Proposition 12.1.1 (Exponentially weighted average)**.** *For $t \geq 0$,*

$$\ell_t = \sum_{j=0}^{t} \alpha(1-\alpha)^j\, y_{t-j} + (1-\alpha)^{t+1}\, \ell_{-1}, \tag{12.3}$$

and the weights add up to one: $\sum_{j=0}^{t} \alpha(1-\alpha)^j + (1-\alpha)^{t+1} = 1$.

Proof. By induction on t. For $t = 0$, (12.1) gives $\ell_0 = \alpha y_0 + (1-\alpha)\ell_{-1}$, which is (12.3). If (12.3) holds for $t-1$, then

$$\ell_t = \alpha y_t + (1-\alpha)\Big[\sum_{j=0}^{t-1} \alpha(1-\alpha)^j y_{t-1-j} + (1-\alpha)^t \ell_{-1}\Big] = \alpha y_t + \sum_{j=1}^{t} \alpha(1-\alpha)^j y_{t-j} + (1-\alpha)^{t+1}\ell_{-1},$$

which is (12.3) for t. For the sum, the geometric series gives $\sum_{j=0}^{t} \alpha(1-\alpha)^j = \alpha \frac{1-(1-\alpha)^{t+1}}{1-(1-\alpha)} = 1-(1-\alpha)^{t+1}$. □

So the level is a weighted average of past observations, with weights that decay geometrically with age. The weight on the initial level ℓ_{-1} vanishes as t grows, so the choice of initialization matters only for the first $O(1/\alpha)$ steps.

Proposition 12.1.2 (Mean lag and the span convention)**.** *In the limit $t \to \infty$, the weights $w_j = \alpha(1-\alpha)^j$ have mean age $\bar{\jmath} = \sum_j j\, w_j = (1-\alpha)/\alpha$. A simple moving average over the last N points has mean age $(N-1)/2$. The two match when $\alpha = 2/(N+1)$.*

Proof. Differentiating the geometric series $\sum_{j\geq 0} q^j = 1/(1-q)$ gives $\sum_{j\geq 0} jq^{j-1} = 1/(1-q)^2$. With $q = 1-\alpha$, $\sum_j j\,\alpha q^j = \alpha q/(1-q)^2 = (1-\alpha)/\alpha$. The moving average puts weight $1/N$ on ages $0, \ldots, N-1$, so its mean age is $(N-1)/2$. Setting $(1-\alpha)/\alpha = (N-1)/2$ and solving gives $\alpha = 2/(N+1)$. □

This is the convention behind `EWMA::alpha_from_span` in `nabia/signal`. The mean lag is also the delay with which the level follows a linear trend: if $y_t = a + bt$, the steady-state level is $\ell_t = a + b(t - \bar{\jmath})$. This lag is why trend strategies built on SES or EWMA signals react late, and why trend models (local linear trend in the Kalman filter note) extrapolate the slope instead.

12.2 The statistical model: ETS(A,N,N)

On its own, (12.1) is only an algorithm. To attach uncertainty to the forecasts we need a statistical model whose optimal one-step predictor is SES. The standard choice is the innovations state-space model with additive error, no trend and no seasonality, ETS(A,N,N) [13]:

$$y_t = \ell_{t-1} + \varepsilon_t, \qquad \ell_t = \ell_{t-1} + \alpha\,\varepsilon_t, \qquad \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2). \tag{12.4}$$

A single error source drives both equations (hence "innovations" form). Given ℓ_{t-1}, the conditional mean of y_t is ℓ_{t-1}, and $\varepsilon_t = y_t - \ell_{t-1} = e_t$. Substituting into the state equation gives exactly (12.2). So the SES levels are the model's states, and the SES errors are its innovations.

Theorem 12.2.1 (Forecast distribution)**.** *Under* (12.4)*, conditionally on the information up to time $T-1$ (that is, on ℓ_{T-1}), for every horizon $h \geq 1$,*

$$y_{T-1+h} \sim \mathcal{N}\big(\ell_{T-1},\ \sigma^2\,[1 + (h-1)\alpha^2]\big).$$

Proof. Iterating the state equation from $T-1$, $\ell_{T-2+h} = \ell_{T-1} + \alpha \sum_{j=1}^{h-1} \varepsilon_{T-1+j}$. Then

$$y_{T-1+h} = \ell_{T-2+h} + \varepsilon_{T-1+h} = \ell_{T-1} + \alpha \sum_{j=1}^{h-1} \varepsilon_{T-1+j} + \varepsilon_{T-1+h}.$$

The future innovations are independent of ℓ_{T-1} and of each other, with mean 0 and variance σ^2. The right-hand side is therefore Gaussian with mean ℓ_{T-1} and variance $\alpha^2(h-1)\sigma^2 + \sigma^2$. □

The point forecast is flat, but the uncertainty grows linearly in h. Each future shock moves the level permanently by $\alpha\varepsilon$, so the model is a random walk observed with noise. The parameter α sets how much of each shock is permanent ($\alpha\varepsilon_t$ enters the level) and how much is transitory.

12.2.1 Equivalence with ARIMA(0,1,1)

Proposition 12.2.2. *Under* (12.4)*, the differences* $\Delta y_t = y_t - y_{t-1}$ *follow the MA(1) process* $\Delta y_t = \varepsilon_t - (1-\alpha)\,\varepsilon_{t-1}$*, so* y *is ARIMA(0,1,1) with moving-average coefficient* $\theta = -(1-\alpha)$*.*

Proof. From (12.4), $y_t - y_{t-1} = (\ell_{t-1} + \varepsilon_t) - (\ell_{t-2} + \varepsilon_{t-1}) = (\ell_{t-1} - \ell_{t-2}) + \varepsilon_t - \varepsilon_{t-1} = \alpha\varepsilon_{t-1} + \varepsilon_t - \varepsilon_{t-1}$. □

The MA polynomial $1-(1-\alpha)z$ has its root at $1/(1-\alpha)$, outside the unit circle for $\alpha \in (0,2)$. The process is therefore invertible, and ε_t can be recovered from past y. This is what makes the recursion (12.1) forget its initialization (Proposition 12.1.1). `nabia` restricts α to $(0,1]$, where all weights in (12.3) are non-negative and the level is a genuine average.

12.2.2 Equivalence with the Kalman filter of the local level model

The local level (random walk plus noise) model has two independent noise sources:

$$x_t = x_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0,q), \qquad y_t = x_t + v_t, \quad v_t \sim \mathcal{N}(0,r). \tag{12.5}$$

Proposition 12.2.3 (Muth [17])**.** *The Kalman filter for* (12.5) *converges to the recursion* $\hat{x}_t = \hat{x}_{t-1} + K(y_t - \hat{x}_{t-1})$ *with steady-state gain*

$$K = \frac{P}{P+r}, \qquad P = \frac{q + \sqrt{q^2 + 4qr}}{2}.$$

This is SES with $\alpha = K$*. Conversely, the SES with factor* $\alpha \in (0,1)$ *is the steady state of* (12.5) *with signal-to-noise ratio* $q/r = \alpha^2/(1-\alpha)$*.*

Proof. For this model the predicted variance $P_t = \mathrm{Var}(x_t \mid y_{0:t-1})$ obeys the Riccati recursion $P_{t+1} = P_t r/(P_t + r) + q$ (see the Kalman filter note, Section 6). A fixed point satisfies $P(P+r) = Pr + q(P+r)$, that is $P^2 - qP - qr = 0$, whose positive root is the P above. The map $P \mapsto Pr/(P+r) + q$ is increasing and concave with slope $r^2/(P+r)^2 < 1$ at the fixed point, so the iteration converges to it from any $P_0 \geq 0$. The filtered estimate is $\hat{x}_t = \hat{x}_{t-1} + K_t(y_t - \hat{x}_{t-1})$ with $K_t = P_t/(P_t+r) \to K$. For the converse, $K = P/(P+r)$ gives $P = Kr/(1-K)$. Substituting into $P^2 = q(P+r)$ gives $K^2 r^2/(1-K)^2 = q\,r/(1-K)$, that is $q/r = K^2/(1-K)$. □

The `nabia` test suite checks Proposition 12.2.3 numerically. After 300 steps, the local-level Kalman filter and an SES with $\alpha = K$ agree to 10^{-9}.

12.3 Estimation

Innovation variance. Conditionally on the initial level, the likelihood of (12.4) factorizes over the one-step errors (prediction error decomposition): $\log L(\sigma^2) = -\frac{T}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_t e_t^2$. Setting the derivative in σ^2 to zero gives the maximum-likelihood estimate

$$\hat{\sigma}^2 = \frac{\mathrm{SSE}}{T}, \qquad \mathrm{SSE} = \sum_{t=0}^{T-1} e_t^2.$$

Smoothing factor. For fixed σ^2, maximizing the likelihood in α amounts to minimizing $\mathrm{SSE}(\alpha)$. This one-dimensional function is smooth but not necessarily convex. `nabia` leaves the choice to the user: the `forecast_demo` example scans a grid, and `NelderMead` on $[0,1]$ or a golden-section search would also do.

12.4 Implementation

12.4.1 Interface

The class implements the forecaster interface

`IForecaster<SimpleExponentialSmoothingParams>` :

```
SimpleExponentialSmoothing model(0.3);           // or (std::make_unique<Params>(...))
model.fit(y);                                    // Eigen::VectorXd, finite values
Forecast fc = model.forecast(12);                // mean and variance per step
Eigen::VectorXd lo = fc.lower(0.95), hi = fc.upper(0.95);
Eigen::VectorXd one_step = model.fitted_values(); // l_{t-1}, the forecast of y_t
double sse = model.sse();
```

12.4.2 Algorithm

Algorithm 1 SES fit and forecast

Require: $y_0, \dots, y_{T-1}$ finite, $\alpha \in (0, 1]$

$\ell \leftarrow y_0$; SSE $\leftarrow 0$ ▷ $\ell_{-1} = y_0$

for $t = 0, \dots, T-1$ **do**

  $\hat{y}_t \leftarrow \ell$ ▷ one-step forecast, stored in `fitted_values()`

  $e \leftarrow y_t - \ell$; SSE $\leftarrow$ SSE $+ e^2$; $\ell \leftarrow \ell + \alpha e$

end for

$\hat{\sigma}^2 \leftarrow$ SSE$/T$

for $h = 1, \dots, H$ **do** ▷ `forecast(H)`

  $\mu_h \leftarrow \ell$; $v_h \leftarrow \hat{\sigma}^2 \,(1 + (h-1)\alpha^2)$

end for

Initialization. The implementation uses $\ell_{-1} = y_0$, so the first error is $e_0 = 0$ and the first fitted value equals the first observation. By Proposition 12.1.1 the effect of this choice decays like $(1-\alpha)^t$. Because $e_0 = 0$ by construction, $\hat{\sigma}^2 = \text{SSE}/T$ averages $T-1$ informative errors over T terms. This slightly underestimates σ^2 in short samples, a bias of factor $(T-1)/T$.

Prediction intervals. `Forecast::lower(level)` and `upper(level)` return $\mu_h \mp z\sqrt{v_h}$ with $z = \Phi^{-1}((1 + \text{level})/2)$, computed by `nabia::stats::normal_quantile`. These are exact intervals under (12.4) with known parameters. They ignore the estimation error in α and σ^2, which makes them slightly too narrow in small samples.

Complexity and validation. Fitting is a single $O(T)$ pass with $O(T)$ memory (the fitted values), and forecasting is $O(H)$. The constructor rejects $\alpha \notin (0, 1]$, and `fit` rejects an empty or non-finite series. `forecast` before `fit` throws `std::logic_error`.

Correspondence with the code.

Mathematics	`SimpleExponentialSmoothing`
α	`parameters().alpha`
ℓ_{T-1}	private `level_`
$(\ell_{t-1})_t$	`fitted_values()`
SSE	`sse()`
(μ_h, v_h)	`forecast(H).mean`, `.variance`

Tests. `tests/test_exponential_smoothing.cpp` covers several cases: a hand-computed series (levels $10 \to 15 \to 22.5$, SSE $= 325$, variances $325/3$ and $1.25 \cdot 325/3$), $\alpha = 1$ as the naive forecast, invalid α, and forecasting before fitting. The Kalman filter equivalence of Proposition 12.2.3 is tested in `tests/test_forecasters.cpp`.

Chapter 13

The Kalman filter

This note documents `KalmanFilter`, the `nabia` forecaster for linear Gaussian state-space models with a scalar observation. We first prove the Gaussian conditioning lemma and derive the filter from it. We then prove that the Kalman gain minimizes the error covariance among all linear updates, in the Loewner order, and show why the Joseph form of the covariance update is used. We derive the likelihood by the prediction error decomposition, the h-step forecast distribution, the treatment of missing values and the Rauch–Tung–Striebel smoother. For the two built-in models, local level and local linear trend, we derive the steady state, the link with exponential smoothing and the growth of the forecast variance with the horizon. The last section maps the algorithm to the code and discusses numerical choices, complexity and limitations.

13.1 Model

A linear Gaussian state-space model with n states and a scalar observation is

$$\begin{aligned} x_t &= F x_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q), \\ y_t &= H x_t + v_t, \qquad v_t \sim \mathcal{N}(0, R), \end{aligned} \qquad t = 0, \dots, T-1, \tag{13.1}$$

with transition $F \in \mathbb{R}^{n \times n}$, observation row $H \in \mathbb{R}^{1 \times n}$, process noise covariance $Q \succeq 0$ and observation noise variance $R > 0$. The prior on the first state is $x_0 \sim \mathcal{N}(\bar{x}_0, P_0)$. The initial state, the w_t and the v_t are mutually independent and independent over time. The state x_t is hidden; only y_t is observed, and some y_t may be missing.

Notation. For $s \le t$, write $y_{0:s}$ for the observations up to time s and

$$x_{t|s} = \mathbb{E}[x_t \mid y_{0:s}], \qquad P_{t|s} = \mathrm{Cov}(x_t \mid y_{0:s}).$$

With this notation, $x_{0|-1} = \bar{x}_0$ and $P_{0|-1} = P_0$. The *filter* computes $(x_{t|t}, P_{t|t})$, the *one-step predictor* $(x_{t|t-1}, P_{t|t-1})$, the *forecast* $(x_{T-1+h|T-1}, P_{T-1+h|T-1})$ and the *smoother* $(x_{t|T-1}, P_{t|T-1})$.

13.2 The filter

13.2.1 Gaussian conditioning

Lemma 13.2.1 (Conditioning a Gaussian vector)**.** *Let (a, b) be jointly Gaussian with means μ_a, μ_b, covariances Σ_{aa}, Σ_{bb}, cross-covariance $\Sigma_{ab} = \Sigma_{ba}^\top$, and $\Sigma_{bb} \succ 0$. Then*

$$a \mid b \;\sim\; \mathcal{N}(\mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(b - \mu_b),\; \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}).$$

Proof. Let $G = \Sigma_{ab}\Sigma_{bb}^{-1}$ and $c = a - Gb$. Then $\mathrm{Cov}(c, b) = \Sigma_{ab} - G\Sigma_{bb} = 0$. Since (c, b) is a linear function of the Gaussian vector (a, b), it is jointly Gaussian, and zero covariance implies that c and b are independent. Writing $a = c + Gb$, conditionally on b the vector a is c (whose distribution is unchanged by conditioning) plus the constant Gb. Hence $a \mid b$ is Gaussian with mean $\mathbb{E}c + Gb = \mu_a - G\mu_b + Gb$ and covariance $\mathrm{Cov}(c) = \Sigma_{aa} - G\Sigma_{ba} - \Sigma_{ab}G^\top + G\Sigma_{bb}G^\top = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$. □

13.2.2 Prediction and correction

Theorem 13.2.2 (Kalman filter)**.** *For model* (13.1)*, every distribution $x_t \mid y_{0:s}$ is Gaussian, and the moments obey, for $t \geq 0$:*

$$\textit{predict } (t \geq 1)\textit{:} \quad x_{t|t-1} = Fx_{t-1|t-1}, \qquad P_{t|t-1} = FP_{t-1|t-1}F^\top + Q, \tag{13.2}$$

$$\textit{innovation:} \quad e_t = y_t - Hx_{t|t-1}, \qquad S_t = HP_{t|t-1}H^\top + R, \tag{13.3}$$

$$\textit{gain:} \quad K_t = P_{t|t-1}H^\top / S_t, \tag{13.4}$$

$$\textit{correct:} \quad x_{t|t} = x_{t|t-1} + K_t e_t, \qquad P_{t|t} = (I - K_t H)\, P_{t|t-1}. \tag{13.5}$$

Moreover $y_t \mid y_{0:t-1} \sim \mathcal{N}(Hx_{t|t-1}, S_t)$.

Proof. By induction on t. Assume $x_{t-1} \mid y_{0:t-1} \sim \mathcal{N}(x_{t-1|t-1}, P_{t-1|t-1})$. For $t = 0$, the prior plays this role and there is no prediction step.

Prediction. The noise w_t is independent of $(x_{t-1}, y_{0:t-1})$. Given $y_{0:t-1}$, the state $x_t = Fx_{t-1} + w_t$ is an affine function of two independent Gaussian vectors. So $x_t \mid y_{0:t-1}$ is Gaussian with mean $Fx_{t-1|t-1}$ and covariance $FP_{t-1|t-1}F^\top + Q$.

Correction. The noise v_t is independent of $(x_t, y_{0:t-1})$, so given $y_{0:t-1}$ the pair (x_t, y_t) is jointly Gaussian with

$$\mathbb{E}[y_t \mid y_{0:t-1}] = Hx_{t|t-1}, \qquad \mathrm{Var}(y_t \mid y_{0:t-1}) = HP_{t|t-1}H^\top + R = S_t,$$

$$\mathrm{Cov}(x_t, y_t \mid y_{0:t-1}) = P_{t|t-1}H^\top.$$

This already proves the claim on $y_t \mid y_{0:t-1}$. Conditioning additionally on y_t means applying Lemma 13.2.1 to this conditional joint distribution, with $a = x_t$, $b = y_t$ and $\Sigma_{bb} = S_t \geq R > 0$. This gives the mean $x_{t|t-1} + P_{t|t-1}H^\top S_t^{-1} e_t$ and covariance $P_{t|t-1} - P_{t|t-1}H^\top S_t^{-1} HP_{t|t-1} = (I - K_t H)P_{t|t-1}$. □

With a scalar observation, S_t is a positive number. The gain is a vector, and no matrix is ever inverted.

13.2.3 Optimality of the gain and the Joseph form

Theorem 13.2.2 uses Gaussianity. Without it, the filter is still the best *linear* update.

Proposition 13.2.3 (Joseph form, valid for any gain)**.** *Let $\hat{x}^-$ be an estimate of x with error covariance $P^- = \mathrm{Cov}(x - \hat{x}^-)$, and observe $y = Hx + v$ with v zero-mean, variance R, and uncorrelated with $x - \hat{x}^-$. For any gain $G \in \mathbb{R}^n$, the update $\hat{x}^+ = \hat{x}^- + G(y - H\hat{x}^-)$ has error covariance*

$$P^+(G) = (I - GH)\, P^- \,(I - GH)^\top + GRG^\top. \tag{13.6}$$

Proof. The new error is $x - \hat{x}^+ = (x - \hat{x}^-) - G(Hx + v - H\hat{x}^-) = (I - GH)(x - \hat{x}^-) - Gv$. The two terms are uncorrelated, so their covariances add, giving (13.6). □

Theorem 13.2.4 (The Kalman gain is Loewner-optimal)**.** *With $S = HP^-H^\top + R$ and $K = P^-H^\top/S$, for every G,*

$$P^+(G) = (I-KH)P^- + (G-K)\,S\,(G-K)^\top \;\succeq\; P^+(K) = (I-KH)P^-.$$

In particular K minimizes $\operatorname{tr} P^+(G)$ (the mean squared error) and every other weighted mean squared error $\mathbb{E}\|x-\hat{x}^+\|_W^2$, $W \succeq 0$.

Proof. Expanding (13.6), $P^+(G) = P^- - GHP^- - P^-H^\top G^\top + GSG^\top$. Using $P^-H^\top = KS$ and completing the square,

$$P^+(G) = P^- - KSK^\top + (G-K)S(G-K)^\top.$$

Since $KSK^\top = KHP^-$, the first two terms equal $(I-KH)P^-$. The last term is positive semi-definite because $S > 0$, and it vanishes iff $G = K$. For the weighted error, $\mathbb{E}\|e\|_W^2 = \operatorname{tr}(WP^+)$ and $\operatorname{tr}(W(G-K)S(G-K)^\top) \geq 0$. □

Why the implementation uses the Joseph form. The short form $(I-KH)P^-$ equals (13.6) only for the exact gain. In floating point the computed K is slightly off, and the short form then loses symmetry and can even lose positive semi-definiteness: it subtracts two nearly equal matrices. Equation (13.6) is a sum of two positive semi-definite terms, so it stays symmetric positive semi-definite for any K. By Theorem 13.2.4, an error δ in the gain changes it only by $\delta S\delta^\top = O(\|\delta\|^2)$. `nabia` therefore updates the covariance as $P \leftarrow (I-KH)P(I-KH)^\top + RKK^\top$.

13.2.4 Missing observations

If y_t is missing, conditioning on the available data is conditioning on $y_{0:t-1}$ only. Then $x_{t|t} = x_{t|t-1}$ and $P_{t|t} = P_{t|t-1}$: the correction step is skipped. Uncertainty grows through consecutive prediction steps and shrinks again at the next observation. `nabia` treats any non-finite y_t (NaN) as missing.

13.3 Likelihood

Proposition 13.3.1 (Prediction error decomposition)**.** *Let $\mathcal{O} \subseteq \{0,\dots,T-1\}$ be the set of observed times. The log density of the observed values is*

$$\log p\big((y_t)_{t\in\mathcal{O}}\big) = -\frac{1}{2}\sum_{t\in\mathcal{O}}\Big(\log(2\pi) + \log S_t + \frac{e_t^2}{S_t}\Big).$$

Proof. By the chain rule of probability, the joint density of the observed values is the product of the conditional densities $p(y_t \mid \text{observed } y_s,\ s<t)$ over $t \in \mathcal{O}$. By Theorem 13.2.2, applied with the missing values skipped, each factor is $\mathcal{N}(y_t;\, Hx_{t|t-1}, S_t)$. Taking logs gives the result. □

The likelihood therefore comes for free with the filter, in $O(1)$ extra work per step. It is the objective for estimating unknown entries of F, H, Q, R by maximum likelihood.

Remark 13.3.2 (Diffuse prior). With a large P_0 (the `nabia` default is 10^7), the first term contains $\log S_0 \approx \log P_0$. This term reflects the vague prior, not the data. When comparing likelihoods across parameter values with the same P_0, the term is nearly constant and harmless. To compare models with different state dimensions, drop the first n terms (the diffuse likelihood).

13.4 Forecasting

Proposition 13.4.1 (h-step forecast)**.** *For $h \geq 1$, $y_{T-1+h} \mid y_{0:T-1} \sim \mathcal{N}(\mu_h, v_h)$ with*

$$x_{T-1+h|T-1} = F^h x_{T-1|T-1}, \qquad P_{T-1+h|T-1} = F P_{T-2+h|T-1} F^\top + Q,$$
$$\mu_h = H x_{T-1+h|T-1}, \qquad v_h = H P_{T-1+h|T-1} H^\top + R.$$

Proof. Future observations are missing data. Apply the prediction step (13.2) h times without corrections, then take the observation moments as in the proof of Theorem 13.2.2. □

13.5 Smoothing

The filter uses only the past. For analysis (for example, extracting the trend of a historical series), the smoother uses the whole sample.

Theorem 13.5.1 (Rauch–Tung–Striebel smoother [26])**.** *Let $x_{T-1|T-1}, P_{T-1|T-1}$ be the last filtered moments. For $t = T-2, \dots, 0$, with $C_t = P_{t|t} F^\top P_{t+1|t}^{-1}$,*

$$x_{t|T-1} = x_{t|t} + C_t \left(x_{t+1|T-1} - x_{t+1|t}\right), \qquad P_{t|T-1} = P_{t|t} + C_t \left(P_{t+1|T-1} - P_{t+1|t}\right) C_t^\top.$$

Proof. Step 1 (Markov property). Given x_{t+1}, the state x_t is independent of $y_{t+1:T-1}$. Those observations are functions of x_{t+1} and of the noises $w_{t+2:T-1}, v_{t+1:T-1}$, which are independent of x_t and $y_{0:t}$. Hence $p(x_t \mid x_{t+1}, y_{0:T-1}) = p(x_t \mid x_{t+1}, y_{0:t})$.

Step 2. Given $y_{0:t}$, the pair (x_t, x_{t+1}) is jointly Gaussian with $\operatorname{Cov}(x_t, x_{t+1}) = P_{t|t} F^\top$ and $\operatorname{Cov}(x_{t+1}) = P_{t+1|t}$. Lemma 13.2.1 gives $x_t \mid x_{t+1}, y_{0:t} \sim \mathcal{N}(x_{t|t} + C_t(x_{t+1} - x_{t+1|t}),\ P_{t|t} - C_t P_{t+1|t} C_t^\top)$.

Step 3. Average over $x_{t+1} \mid y_{0:T-1} \sim \mathcal{N}(x_{t+1|T-1}, P_{t+1|T-1})$. The law of total expectation gives the mean. The law of total variance gives $P_{t|t} - C_t P_{t+1|t} C_t^\top + C_t P_{t+1|T-1} C_t^\top$, which is the stated covariance. □

`KalmanFilter::smoothed_states()` returns the smoothed means. It computes C_t as $\left(P_{t+1|t}^{-1} F P_{t|t}\right)^\top$ with an $LDL^\top$ solve, using the symmetry of both covariances, rather than forming an inverse.

13.6 Built-in models

13.6.1 Local level

$F = H = 1$, $Q = q$, $R = r$: a random walk observed with noise. The predicted variance follows the scalar Riccati recursion $P_{t+1|t} = P_{t|t-1} r / (P_{t|t-1} + r) + q$.

Proposition 13.6.1 (Steady state)**.** *The predicted variance converges to $P = \frac{1}{2}\left(q + \sqrt{q^2 + 4qr}\right)$ and the gain to $K = P/(P+r)$. The filter then becomes simple exponential smoothing with $\alpha = K$, and $q/r = K^2/(1-K)$.*

Proof. A fixed point solves $P^2 - qP - qr = 0$, whose positive root is the stated P. The map $g(P) = Pr/(P+r) + q$ is increasing with $g'(P) = r^2/(P+r)^2 < 1$ for $P > 0$, and it is concave. Hence the iterates are monotone and bounded, and they converge to the unique positive fixed point from any $P_0 \geq 0$. With $F = H = 1$, (13.5) reads $x_{t|t} = x_{t-1|t-1} + K_t(y_t - x_{t-1|t-1})$, the SES recursion. The relation $q/r = K^2/(1-K)$ follows from substituting $P = Kr/(1-K)$ into $P^2 = q(P+r)$. □

Diffuse start. With $P_0 \gg r$, the first gain is $K_0 = P_0/(P_0 + r) \approx 1$, so $x_{0|0} \approx y_0$ and $P_{0|0} = P_0 r/(P_0 + r) \approx r$. A large initial variance thus amounts to starting from the first observation, as SES does.

13.6.2 Local linear trend

With state $x_t = (\ell_t, b_t)$ (level and slope),

$$F = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad H = \begin{pmatrix} 1 & 0 \end{pmatrix}, \quad Q = \operatorname{diag}(q_\ell, q_b).$$

Proposition 13.6.2 (Trend extrapolation and variance growth)**.** *The point forecast is the straight line* $\mu_h = \ell_{T-1|T-1} + h\, b_{T-1|T-1}$*. The part of the forecast variance due to future slope shocks equals* $q_b \sum_{m=1}^{h-1} m^2 = q_b\,(h-1)h(2h-1)/6 \sim q_b h^3/3$.

Proof. By induction, $F^h = \begin{pmatrix} 1 & h \\ 0 & 1 \end{pmatrix}$, since $F^{h+1} = F^h F = \begin{pmatrix} 1 & h+1 \\ 0 & 1 \end{pmatrix}$. With Proposition 13.4.1, this gives the mean. For the variance, unroll the model from $T-1$: $b_{T-1+k} = b_{T-1} + \sum_{i=1}^{k} w^b_{T-1+i}$, and

$$\ell_{T-1+h} = \ell_{T-1} + \sum_{k=0}^{h-1} b_{T-1+k} + \sum_{j=1}^{h} w^\ell_{T-1+j} = \ell_{T-1} + h\, b_{T-1} + \sum_{i=1}^{h-1} (h-i)\, w^b_{T-1+i} + \sum_{j=1}^{h} w^\ell_{T-1+j}.$$

The future shocks are independent of the current state. The slope shocks contribute $q_b \sum_{i=1}^{h-1} (h-i)^2 = q_b \sum_{m=1}^{h-1} m^2$. □

So a trend model extrapolates the current slope. Its uncertainty grows like h^3, much faster than the linear growth of the local level model, because any error in the slope accumulates over the horizon. A small q_b gives a nearly deterministic trend, and $q_b = 0$ gives a fixed slope estimated from the data.

13.7 Implementation

13.7.1 Interface

```cpp
auto p = std::make_unique<KalmanFilterParams>(
    KalmanFilterParams::local_linear_trend(/*q_level*/ 1e-4, /*q_slope*/ 1e-8, /*R*/ 0.09));
KalmanFilter kf(std::move(p));
kf.fit(y);                                  // NaN = missing
Forecast fc = kf.forecast(10);              // mean and variance (Prop. 4.1)
const Eigen::MatrixXd& x = kf.filtered_states();   // n x T, column t = x_{t|t}
Eigen::MatrixXd xs = kf.smoothed_states();         // n x T, column t = x_{t|T-1}
double ll = kf.log_likelihood();                   // Prop. 3.1
```

To define any other model, fill in the parameter fields:

`transition`	F	`observation_noise`	R
`observation`	H	`initial_state`	$\bar{x}_0$
`process_noise`	Q	`initial_covariance`	P_0

13.7.2 Algorithm

Complexity. Each step costs $O(n^3)$ for the products with F and $O(n^2)$ for the rank-one correction, so fitting is $O(Tn^3)$. The filter stores the predicted and filtered moments ($O(Tn^2)$ memory) because the smoother needs them. Forecasting costs $O(hn^3)$, and smoothing $O(Tn^3)$. For the built-in models $n \le 2$, and the cost is linear in T.

Algorithm 2 `KalmanFilter::fit`

$x \leftarrow \bar{x}_0;\ P \leftarrow P_0;\ \ell \leftarrow 0$
for $t = 0, \dots, T-1$ **do**
  if $t > 0$ **then** $x \leftarrow Fx;\ P \leftarrow FPF^\top + Q$
  end if ▷ predict
  store $x_{t|t-1} = x$, $P_{t|t-1} = P$
  $\hat{y}_t \leftarrow Hx;\ u \leftarrow PH^\top;\ S_t \leftarrow Hu + R$ ▷ u reused by the gain
  if y_t is finite **then**
    $e_t \leftarrow y_t - \hat{y}_t;\ K \leftarrow u/S_t;\ x \leftarrow x + Ke_t$
    $A \leftarrow I - KH;\ P \leftarrow APA^\top + RKK^\top$ ▷ Joseph form
    $\ell \leftarrow \ell - \frac{1}{2}(\log 2\pi + \log S_t + e_t^2/S_t)$
  else $e_t \leftarrow$ NaN ▷ missing: no correction
  end if
  store $x_{t|t} = x$, $P_{t|t} = P$
end for

Validation. The constructor checks that $F, H, Q, \bar{x}_0, P_0$ have consistent dimensions, that Q and P_0 are square and symmetric (to a relative tolerance of 10^{-10}), and that $R > 0$ is finite. It throws `std::invalid_argument` otherwise. `fit` rejects an empty series. Accessors called before `fit` throw `std::logic_error`.

Correspondence with the code.

Mathematics	`KalmanFilter`
$x_{t\|t},\ P_{t\|t}$	`filtered_states().col(t)`, `filtered_covariances()[t]`
$Hx_{t\|t-1}$	`fitted_values()(t)`
$e_t,\ S_t$	`innovations()(t)`, `innovation_variances()(t)`
$\log p(y)$	`log_likelihood()`
$x_{t\|T-1}$	`smoothed_states().col(t)`
$\mu_h,\ v_h$	`forecast(H).mean(h-1)`, `.variance(h-1)`

Tests. `tests/test_forecasters.cpp` checks:

- Proposition 13.6.1: the filter matches SES with the steady-state gain to 10^{-9}.
- Proposition 13.3.1: the log-likelihood equals the log density of $y \sim \mathcal{N}(\bar{x}_0\mathbf{1}, \Sigma)$ with $\Sigma_{ij} = P_0 + q\min(i,j) + r\,\delta_{ij}$, computed directly, to 10^{-10}.
- Proposition 13.6.2: the slope state recovers a true slope of 0.5, and forecasts lie on the extrapolated line.
- Proposition 13.4.1: for the local level, $v_h = P_{T-1|T-1} + hq + r$.
- Missing values: the variance grows at missing steps and shrinks again at the next observation.
- Smoother: with $q = 0$, every smoothed state equals the final filtered estimate.

13.7.3 Limitations and extensions

- Observations are scalar and the system matrices are time-invariant. Vector observations would replace the division by S_t with a Cholesky solve.

- Q, R (and any unknowns in F, H) are inputs. A maximum-likelihood fit can maximize `log_likelihood()` over log-variances, for example with `NelderMead` or, using analytic score recursions, with `LBFGS`.
- The smoother returns means only. Theorem 13.5.1 also gives the smoothed covariances if needed.

Chapter 14

Gaussian process forecasting

This note documents `GaussianProcess` and the kernels in `Kernels.h`. We define Gaussian process regression on the time index and prove that the `nabia` kernels are valid covariance functions. The proofs cover the squared exponential kernel via its power series, the periodic kernel via a warping of time onto a circle, and sums and products via the Schur product theorem. We derive the gradient of every kernel with respect to its log-hyperparameters and the posterior predictive distribution. We also derive the log marginal likelihood and its gradient, using Jacobi's formula for the log-determinant. Further results: how normalizing the data changes the likelihood, the long-horizon behavior of forecasts (mean reversion for decaying kernels, exact periodicity for the periodic kernel), and the Cholesky-based algorithms with their costs. The last section maps everything to the code.

14.1 Model

Definition 14.1.1. A *Gaussian process* $f \sim \mathcal{GP}(0, k)$ on $\mathbb{R}$ is a random function such that, for any finite set of times $t_1, \dots, t_m$, the vector $(f(t_1), \dots, f(t_m))$ is Gaussian with mean zero and covariance $K_{ij} = k(t_i, t_j)$. A function k can serve as a covariance function iff it is symmetric and *positive semi-definite*: every such Gram matrix K is positive semi-definite.

`nabia` places the observations of a series at $t = 0, 1, \dots, T-1$ and models

$$y_t = m + s\,(f(t) + \varepsilon_t), \qquad f \sim \mathcal{GP}(0, k), \qquad \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma_n^2), \tag{14.1}$$

where m is the sample mean and s the sample standard deviation (or $s = 1$ without normalization, Section 14.4.1). We work with the standardized series $z_t = (y_t - m)/s$, so that $z = f + \varepsilon$ with $z \sim \mathcal{N}(0, K)$, $K = K_f + \sigma_n^2 I$, and $(K_f)_{ij} = k(t_i, t_j)$.

The kernel encodes what the series looks like. Its variance sets the amplitude, its length scale sets how fast the function can change, and a period sets which patterns repeat. The data then choose these hyperparameters by maximizing the marginal likelihood (Section 14.4).

14.2 Kernels

All `nabia` kernels are stationary: they depend on $\tau = t - t'$ only. Their hyperparameters $\theta_j > 0$ are exposed as $\log \theta_j$ (see Section 14.6), and `gradient()` returns $\partial k / \partial \log \theta_j = \theta_j \, \partial k / \partial \theta_j$.

14.2.1 Definitions and log-parameter gradients

Squared exponential (`SquaredExponentialKernel`, hyperparameters σ^2, ℓ):

$$k(\tau) = \sigma^2 \exp\Big(-\frac{\tau^2}{2\ell^2}\Big), \qquad \frac{\partial k}{\partial \log \sigma^2} = k, \qquad \frac{\partial k}{\partial \log \ell} = k\,\frac{\tau^2}{\ell^2}.$$

The second derivative follows from $\partial(\tau^2/2\ell^2)/\partial\ell = -\tau^2/\ell^3$, multiplied by ℓ. Sample paths are infinitely differentiable.

Matérn 5/2 (`Matern52Kernel`, σ^2, ℓ): with $s = \sqrt{5}|\tau|/\ell$,

$$k = \sigma^2\Big(1 + s + \frac{s^2}{3}\Big)e^{-s}, \qquad \frac{\partial k}{\partial \log \sigma^2} = k, \qquad \frac{\partial k}{\partial \log \ell} = \sigma^2\,\frac{s^2(1+s)}{3}\,e^{-s}.$$

Derivation. $\frac{dk}{ds} = \sigma^2 e^{-s}\big[(1+\frac{2s}{3}) - (1+s+\frac{s^2}{3})\big] = -\sigma^2 e^{-s}\,\frac{s(1+s)}{3}$, and $\frac{ds}{d\log\ell} = -s$; multiply. Sample paths are twice differentiable. This kernel is usually a more realistic model than the squared exponential for economic and financial series.

Periodic (`PeriodicKernel`, σ^2, ℓ, p): with $u = \pi\tau/p$,

$$k = \sigma^2 \exp\Big(-\frac{2\sin^2 u}{\ell^2}\Big), \quad \frac{\partial k}{\partial \log \sigma^2} = k, \quad \frac{\partial k}{\partial \log \ell} = k\,\frac{4\sin^2 u}{\ell^2}, \quad \frac{\partial k}{\partial \log p} = k\,\frac{2u \sin 2u}{\ell^2}.$$

Derivation of the last one. $\frac{du}{d\log p} = -u$ and $\frac{d\sin^2 u}{du} = \sin 2u$, so $\frac{d\sin^2 u}{d\log p} = -u\sin 2u$. Then multiply by $-2k/\ell^2$.

Sum and product (`SumKernel`, `ProductKernel`): for $k = k_1 + k_2$ the gradient is the concatenation $(\nabla k_1, \nabla k_2)$. For $k = k_1 k_2$ the product rule gives $(k_2 \nabla k_1,\ k_1 \nabla k_2)$. Hyperparameters are ordered as those of k_1 followed by those of k_2, recursively.

14.2.2 Validity

Theorem 14.2.1 (Schur product theorem)**.** *If $A, B \in \mathbb{R}^{m\times m}$ are positive semi-definite, so is their elementwise product $A \circ B$.*

Proof. Write $B = \sum_k \lambda_k u_k u_k^\top$ with $\lambda_k \geq 0$. Then $(A \circ B)_{ij} = \sum_k \lambda_k (u_k)_i A_{ij} (u_k)_j$, so for any x, $x^\top (A \circ B) x = \sum_k \lambda_k \,(x \circ u_k)^\top A\,(x \circ u_k) \geq 0$. □

Corollary 14.2.2. *If k_1, k_2 are valid kernels, so are $k_1 + k_2$, $k_1 k_2$, $c\,k_1$ for $c > 0$, $k_1(\phi(t), \phi(t'))$ for any map ϕ, and pointwise limits of valid kernels.*

Proof. Gram matrices of $k_1 + k_2$ are sums of positive semi-definite matrices. Those of $k_1 k_2$ are Schur products (Theorem 14.2.1). A warped kernel's Gram matrix on $t_1, \ldots, t_m$ is k_1's Gram matrix on $\phi(t_1), \ldots, \phi(t_m)$. The positive semi-definite cone is closed, which covers limits. □

Proposition 14.2.3. *The squared exponential kernel is positive semi-definite, on $\mathbb{R}$ and on $\mathbb{R}^d$ with $\|t - t'\|$ in place of $|t - t'|$.*

Proof. On $\mathbb{R}$, write $e^{-(t-t')^2/2\ell^2} = g(t)\,g(t')\,e^{tt'/\ell^2}$ with $g(t) = e^{-t^2/2\ell^2}$. The power series $e^{tt'/\ell^2} = \sum_{n\geq 0} (tt')^n/(\ell^{2n} n!)$ is a limit of sums of the rank-one kernels $t^n t'^n$, which are positive semi-definite. Multiplying by $g(t)g(t')$ (a diagonal congruence DKD) preserves positive semi-definiteness, and so does scaling by $\sigma^2 > 0$. On $\mathbb{R}^d$, $e^{-\|t-t'\|^2/2\ell^2} = \prod_i e^{-(t_i - t_i')^2/2\ell^2}$ is a product of valid kernels (Corollary 14.2.2). □

Proposition 14.2.4. *The periodic kernel is the squared exponential kernel on $\mathbb{R}^2$, with the same ℓ, applied to the warping $\phi(t) = (\cos(2\pi t/p), \sin(2\pi t/p))$. It is therefore positive semi-definite.*

Proof. $\|\phi(t) - \phi(t')\|^2 = 2 - 2\cos(2\pi(t-t')/p) = 4\sin^2(\pi(t-t')/p)$, so $\exp(-\|\phi(t) - \phi(t')\|^2/2\ell^2) = \exp(-2\sin^2(\pi\tau/p)/\ell^2)$. Conclude with Proposition 14.2.3 and Corollary 14.2.2. □

The Matérn 5/2 kernel is positive semi-definite by Bochner's theorem. Its spectral density is proportional to $(5/\ell^2 + \omega^2)^{-3}$, which is non-negative and integrable [22]. With $\sigma_n^2 > 0$, $K = K_f + \sigma_n^2 I$ is positive *definite* for every kernel above. Its Cholesky factor therefore exists in exact arithmetic. The floor `min_noise_variance` keeps it computable in floating point.

14.3 Posterior prediction

Theorem 14.3.1 (Predictive distribution)**.** *Let t_* be a vector of m new times, $K_* = k(t, t_*) \in \mathbb{R}^{T\times m}$ and $K_{**} = k(t_*, t_*)$. Under* (14.1)*, with $K = K_f + \sigma_n^2 I$,*

$$f(t_*) \mid z \sim \mathcal{N}\big(K_*^\top K^{-1} z,\ K_{**} - K_*^\top K^{-1} K_*\big), \qquad z_* \mid z \sim \mathcal{N}\big(K_*^\top K^{-1} z,\ K_{**} - K_*^\top K^{-1} K_* + \sigma_n^2 I\big).$$

Proof. $(z, f(t_*))$ is jointly Gaussian with zero mean, $\mathrm{Cov}(z) = K$, $\mathrm{Cov}(f(t_*)) = K_{**}$ and $\mathrm{Cov}(z, f(t_*)) = \mathrm{Cov}(f(t), f(t_*)) = K_*$, because the noise is independent of f. The first claim is the conditioning lemma (proved in the Kalman filter note, Lemma 2.1). A new observation $z_* = f(t_*) + \varepsilon_*$ adds independent noise, which adds $\sigma_n^2 I$ to the covariance. □

The predictive mean $K_*^\top \alpha$, with $\alpha = K^{-1} z$, is a weighted sum of kernel functions centred at the data. The variance is the prior variance minus what the data explain. `GaussianProcess::predict` returns, for each t_*, the marginal mean and variance of $y_* = m + s\, z_*$ (noise included), and `forecast(H)` calls it with $t_* = T, \dots, T + H - 1$.

14.3.1 Long-horizon behavior

Proposition 14.3.2 (Mean reversion)**.** *If $k(\tau) \to 0$ as $|\tau| \to \infty$ (squared exponential, Matérn), then as $t_* \to \infty$ the predictive mean of y_* tends to m and its variance to $s^2(k(0) + \sigma_n^2)$.*

Proof. Each entry of K_* is $k(t_i - t_*) \to 0$, while K^{-1} and z are fixed. So the mean $K_*^\top K^{-1} z \to 0$ and the variance $k(0) - K_*^\top K^{-1} K_* + \sigma_n^2 \to k(0) + \sigma_n^2$. Transform back with $y_* = m + s z_*$. □

Proposition 14.3.3 (Periodic extrapolation)**.** *For the periodic kernel with period p, the predictive mean and variance at $t_* + p$ equal those at t_*.*

Proof. $k(t_i, t_* + p) = k(t_i, t_*)$ and $k(t_* + p, t_* + p) = k(t_*, t_*)$, since $\sin^2$ has period π. Hence K_* and K_{**} are unchanged, and so is everything in Theorem 14.3.1. □

So a decaying kernel is a local model. Its forecasts fade towards the mean on the time scale of the length scale, and its intervals widen to the unconditional spread. Long-range structure must be built into the kernel: a periodic component (or a sum with one) repeats seasonality indefinitely, and a product of periodic and decaying kernels lets the pattern drift.

14.4 Marginal likelihood

Integrating out f, $z \sim \mathcal{N}(0, K(\theta))$, so the log marginal likelihood is

$$\log p(z \mid \theta) = -\tfrac{1}{2}\, z^\top K^{-1} z \ -\ \tfrac{1}{2} \log\det K \ -\ \tfrac{T}{2} \log 2\pi. \tag{14.2}$$

The first term rewards fitting the data. The second penalizes complexity: a flexible kernel (short length scales, small noise) can explain many different data sets, so it spreads its probability mass thinly and pays through a larger $\log\det K$. The trade-off is automatic, an Occam's razor that needs no validation set.

Lemma 14.4.1. *For a smooth family of invertible symmetric matrices $K(\theta)$, with $K' = \partial K/\partial\theta$: (i) $\partial K^{-1}/\partial\theta = -K^{-1} K' K^{-1}$; (ii) $\partial \log\det K/\partial\theta = \mathrm{tr}(K^{-1} K')$ when $K \succ 0$.*

Proof. (i) Differentiate $K K^{-1} = I$: $K' K^{-1} + K\, \partial K^{-1} = 0$. (ii) $\det(K + \epsilon K') = \det K\, \det(I + \epsilon K^{-1} K')$, and $\det(I + \epsilon A) = \prod_i (1 + \epsilon \lambda_i(A)) = 1 + \epsilon\, \mathrm{tr}\, A + O(\epsilon^2)$. Hence $\log\det(K + \epsilon K') = \log\det K + \epsilon\, \mathrm{tr}(K^{-1} K') + O(\epsilon^2)$. □

Theorem 14.4.2 (Gradient of the log marginal likelihood)**.** *With* $\alpha = K^{-1}z$*,*

$$\frac{\partial \log p(z \mid \theta)}{\partial \theta_j} = \tfrac{1}{2}\,\alpha^\top \frac{\partial K}{\partial \theta_j}\alpha - \tfrac{1}{2}\operatorname{tr}\Big(K^{-1}\frac{\partial K}{\partial \theta_j}\Big) = \tfrac{1}{2}\operatorname{tr}\Big((\alpha\alpha^\top - K^{-1})\,\frac{\partial K}{\partial \theta_j}\Big).$$

Proof. Differentiate (14.2) with Lemma 14.4.1: $\partial(z^\top K^{-1}z) = -z^\top K^{-1}K'K^{-1}z = -\alpha^\top K'\alpha$ and $\partial \log\det K = \operatorname{tr}(K^{-1}K')$. Finally, $\alpha^\top K'\alpha = \operatorname{tr}(\alpha\alpha^\top K')$. □

For a kernel hyperparameter, $\partial K/\partial \log\theta_j$ is the matrix of `gradient()` entries. For the noise, $\partial K/\partial \log\sigma_n^2 = \sigma_n^2 I$, so $\partial \log p/\partial \log\sigma_n^2 = \frac{1}{2}\sigma_n^2 \operatorname{tr}(\alpha\alpha^\top - K^{-1})$.

14.4.1 Normalization

Proposition 14.4.3. *If* $z = (y - m\mathbf{1})/s$ *with fixed* m, s*, then* $\log p(y) = \log p(z) - T\log s$*.*

Proof. The map $y \mapsto z$ is affine with Jacobian s^{-T}, so $p_y(y) = p_z(z)\,s^{-T}$. □

`nabia` fits on z when `normalize` is true (the default), so that unit variances and noise levels around 10^{-2} are sensible starting points whatever the units of y. The constant $-T\log s$ does not depend on θ, so the optimum is unchanged. `log_marginal_likelihood()` reports $\log p(y)$ in the units of y, using Proposition 14.4.3. Fitted kernel variances and the noise variance are in normalized units: multiply by s^2 for the units of y. The mean m is the sample mean, plugged in rather than estimated jointly. This is standard and harmless when the series has no strong trend.

14.5 Computation

Cholesky factorization. Write $K = LL^\top$ with L lower triangular (cost $T^3/3$). Then:

- $\alpha = L^{-\top}(L^{-1}z)$, two triangular solves costing $O(T^2)$;
- $\log\det K = 2\sum_i \log L_{ii}$, since $\det K = \det(L)^2 = \prod_i L_{ii}^2$;
- for predictions, $v = L^{-1}K_*$ gives $K_*^\top K^{-1}K_* = v^\top v$, so the predictive variances are $\operatorname{diag}(K_{**}) - \sum_i v_i^2$. This costs $O(T^2)$ per new time and never forms K^{-1}.

Subtracting $v^\top v$ can produce tiny negative values in floating point; `predict` clamps the latent variance at 0 before adding the noise.

Gradient. Theorem 14.4.2 needs $W = \alpha\alpha^\top - K^{-1}$. `nabia` forms K^{-1} from the Cholesky factor ($O(T^3)$) and accumulates $-\frac{1}{2}\sum_{ij} W_{ij}\,\partial k(t_i,t_j)/\partial\log\theta$ for all hyperparameters in one pass over the T^2 pairs. One evaluation of the objective and its gradient therefore costs $O(T^3 + T^2 n_\theta)$.

Operation	Cost	Memory
Gram matrix	$O(T^2)$ kernel evaluations	$O(T^2)$
Cholesky, α, $\log\det$	$O(T^3)$	$O(T^2)$
Likelihood gradient	$O(T^3 + T^2 n_\theta)$	$O(T^2)$
Prediction at m times	$O(T^2 m)$	$O(Tm)$

In practice this means a few thousand observations at most. Beyond that, state-space representations of Matérn kernels (which reduce to Kalman filtering) or sparse approximations are needed.

14.6 Hyperparameter optimization

The hyperparameters are $\theta = (\log \theta_1, \ldots, \log \theta_{n_k}, \log \sigma_n^2)$, without the last entry when `optimize_noise` is false. `fit` minimizes $-\log p(z \mid \theta)$ with `LBFGS` (tolerance 10^{-5} on the gradient, at most `max_iter` iterations), starting from the kernel and noise given in the parameters.

- *Why log-parameters.* Positivity becomes unconstrained, and the objective is much better conditioned. Length scales and variances matter on a relative scale, so a unit step in $\log \ell$ means the same thing for $\ell = 0.1$ and $\ell = 100$.
- *Safeguards.* A trial with $|\theta_j| > 20$ (a factor $e^{20} \approx 5 \cdot 10^8$), or whose K fails the Cholesky factorization, returns $+\infty$. The L-BFGS line search rejects such steps and backtracks.
- *Noise floor.* The noise is clamped at `min_noise_variance`. In the clamped region the true gradient in $\log \sigma_n^2$ is zero, but the formula above is not. This only matters if the optimum lies at the floor, which signals noiseless data.
- *Local optima.* The marginal likelihood is not concave and can be multimodal: for example, a short-length-scale explanation that fits the noise against a long-length-scale one with more noise, or periods that are multiples of each other. Start from plausible values, such as a period near the known seasonality. The `nabia` tests start a periodic kernel at $p = 11.5$ and recover $p = 12.0$. Restarting from several initial points is a cheap safeguard.
- The parameter struct keeps the initial kernel. The fitted copy is available through `kernel()`, and `noise_variance()` returns the fitted noise.

14.7 Implementation

14.7.1 Interface

```
auto p = std::make_unique<GaussianProcessParams>();
p->kernel = std::make_unique<SumKernel>(
    std::make_unique<Matern52Kernel>(1.0, 20.0),          // slow trend
    std::make_unique<PeriodicKernel>(0.5, 1.0, 12.0));    // seasonality
p->noise_variance = 1e-2;                                  // initial value
GaussianProcess gp(std::move(p));
gp.fit(y);                                 // equally spaced, finite values
Forecast fc = gp.forecast(24);             // t = T .. T+23, noise included
Forecast at = gp.predict(times);           // any times, e.g. between observations
double lml = gp.log_marginal_likelihood();
```

14.7.2 Algorithm

Validation. The constructor requires a kernel and positive noise variances. `fit` requires at least two finite observations, so the GP does not accept missing values; use the Kalman filter for gappy series. Kernels reject non-positive hyperparameters. A final Cholesky failure, which only happens if the noise is too small for the kernel, throws `std::invalid_argument`.

Correspondence with the code.

Algorithm 3 `GaussianProcess::fit` and `predict`

Require: $y_0, \dots, y_{T-1}$ finite, $T \geq 2$; kernel k; noise σ_n^2

$t \leftarrow (0, \dots, T-1)$; $m \leftarrow \bar{y}$; $s \leftarrow$ sample sd (or 1); $z \leftarrow (y - m)/s$

if `optimize_hyperparameters` **then**

  $\theta^\star \leftarrow$ L-BFGS minimizer of $-\log p(z \mid \theta)$, using Theorem 14.4.2 for the gradient

  set k and σ_n^2 from $\theta^\star$

end if

$L \leftarrow \mathrm{chol}(K_f + \sigma_n^2 I)$; $\alpha \leftarrow L^{-\top} L^{-1} z$ ▷ stored for prediction

predict(t_*):

$K_* \leftarrow k(t, t_*)$; $V \leftarrow L^{-1} K_*$

$\mu \leftarrow m + s\, K_*^\top \alpha$; $v \leftarrow s^2 \big(\max(\mathrm{diag}\, K_{**} - \mathrm{colsum}(V \circ V), 0) + \sigma_n^2 \big)$

Mathematics	`GaussianProcess` / kernels
$k(t,t')$, $\partial k / \partial \log \theta$	`IKernel::operator()`, `gradient()`
$\log \theta$	`log_hyperparameters()`, `set_log_hyperparameters()`
K_f	`IKernel::matrix(t, t)`
m, s	private `offset_`, `scale_`
L, α	private `chol_`, `alpha_`
$\log p(y)$	`log_marginal_likelihood()`
$m + sK_f\alpha$	`fitted_values()`

Tests. `tests/test_forecasters.cpp` checks:

- every kernel's gradient against central finite differences, plus symmetry;
- interpolation of a smooth function, with error below 10^{-3} between the data points;
- that hyperparameter fitting raises the log marginal likelihood by more than 10 nats and finds a plausible length scale;
- Proposition 14.3.3: a periodic kernel recovers period 12.0, extrapolates 24 steps to within 0.1, and the true values fall inside the 95% intervals;
- Proposition 14.3.2: at $t_* = 1000$ the forecast equals m and its variance equals $s^2(k(0) + \sigma_n^2)$, both to 10^{-12};
- rejection of invalid input.

14.7.3 Limitations

Cost is cubic in T. Inputs are equally spaced time indices, although `predict` accepts any times. The mean is constant, the noise Gaussian and homoscedastic, and the kernels stationary. There are no automatic restarts in the hyperparameter search.

Chapter 15

Convex optimization

This document describes the algorithms in the `nabia` module `optimization/convex`: the limited-memory BFGS method (`LBFGS`) for smooth unconstrained problems, and two solvers for smooth problems with linear equality, inequality and box constraints: the augmented Lagrangian method, also called the method of multipliers (`AugmentedLagrangian`), and sequential quadratic programming (`SQP`). We derive the augmented Lagrangian for equalities and inequalities and the multiplier updates. We then discuss convergence in the convex case and the practical safeguards (penalty update, inexact inner solves, stopping tests), and present L-BFGS as both a standalone solver and the inner solver. Finally we derive SQP: its quadratic subproblem, the damped BFGS Hessian approximation and the ℓ_1 merit function, and compare when each constrained solver is preferable.

15.1 Module overview

All solvers follow the same pattern: construct with a parameter object and the objective (and gradient), then call `optimize(std::optional<Eigen::VectorXd> initial_guess)`. It returns the solution and starts from zero when no guess is given. Diagnostics such as multipliers, residuals and iteration counts are in `result()`. The same solver can be run again from a different starting point. Invalid input (mismatched dimensions, inconsistent constraints, nonpositive tolerances) raises `std::invalid_argument`.

15.2 Problem statement

We consider

$$\min_{x\in\mathbb{R}^n} \; f(x) \quad \text{s.t.} \quad h(x) \coloneqq A_{\text{eq}}x - b_{\text{eq}} = 0, \qquad g(x) \coloneqq A_{\text{in}}x - b_{\text{in}} \le 0, \tag{15.1}$$

with $f : \mathbb{R}^n \to \mathbb{R}$ convex and continuously differentiable, $A_{\text{eq}} \in \mathbb{R}^{p\times n}$, $A_{\text{in}} \in \mathbb{R}^{m\times n}$. Box constraints $\ell \le x \le u$ are a special case of the inequalities (Section 15.8).

Lagrangian and KKT conditions. With multipliers $\lambda \in \mathbb{R}^p$ for the equalities and $\mu \in \mathbb{R}^m$ for the inequalities, the Lagrangian is

$$L(x, \lambda, \mu) = f(x) + \lambda^\top h(x) + \mu^\top g(x).$$

Type	Role
`IOptimizer<T, R>`	Base interface: owns the parameters `T` through a `std::unique_ptr`, declares `optimize(initial_guess)`, which returns the solution, and `result()`, which returns the solver's result object `R`.
`LBFGS`	`IOptimizer<LBFGSParams, LBFGSResult>`; unconstrained smooth minimization (Section 15.6).
`IConvexOptimization<R>`	`IOptimizer<ConvexOptimizationParams, R>`; interface for constrained convex solvers.
`AugmentedLagrangian`	`IConvexOptimization<AugmentedLagrangianResult>`; linearly constrained convex minimization, using `LBFGS` for its subproblems.
`SQP`	`IConvexOptimization<SQPResult>`; linearly constrained smooth minimization with few evaluations of f (Section 15.7); solves its QP subproblems with `AugmentedLagrangian`.
`LinearConstraint`	$A_{\text{eq}}x = b_{\text{eq}}$ and $A_{\text{in}}x \le b_{\text{in}}$ as Eigen matrices and vectors.
`BoxConstraints`	Bounds $\ell \le x \le u$ (Section 15.8).
`Constraints`	`std::variant<BoxConstraints, LinearConstraint>`; held as `std::optional` in `ConvexOptimizationParams`, where empty means unconstrained. `to_linear_constraint` converts any of them to matrix form.

Table 15.1: Types in `nabia/optimization`.

Because the constraints are linear, no further constraint qualification is needed: a point $x^\star$ solves (15.1) if and only if there exist $(\lambda^\star, \mu^\star)$ satisfying the Karush–Kuhn–Tucker conditions

$$\nabla f(x^\star) + A_{\text{eq}}^\top \lambda^\star + A_{\text{in}}^\top \mu^\star = 0 \qquad \text{(stationarity)} \tag{15.2}$$

$$A_{\text{eq}}x^\star = b_{\text{eq}}, \quad A_{\text{in}}x^\star \le b_{\text{in}} \qquad \text{(primal feasibility)} \tag{15.3}$$

$$\mu^\star \ge 0 \qquad \text{(dual feasibility)} \tag{15.4}$$

$$\mu_i^\star \, g_i(x^\star) = 0, \ i = 1, \dots, m \qquad \text{(complementarity)} \tag{15.5}$$

The multipliers have a sensitivity interpretation: $\lambda_i^\star$ and $\mu_i^\star$ are the rates of change of the optimal value with respect to the right-hand sides b_i (with opposite sign).

15.3 From penalties to augmented Lagrangians

15.3.1 The quadratic penalty and its drawback

The simplest way to remove the equality constraints is the quadratic penalty

$$Q_\rho(x) = f(x) + \frac{\rho}{2}\|h(x)\|^2.$$

Its minimizer x_ρ satisfies $\nabla f(x_\rho) + A_{\text{eq}}^\top(\rho\, h(x_\rho)) = 0$. Comparing with (15.2), $\rho\, h(x_\rho)$ plays the role of the multiplier. Since $\lambda^\star$ is generally nonzero, $h(x_\rho) \approx \lambda^\star/\rho$ vanishes only as $\rho \to \infty$. The Hessian of Q_ρ contains the term $\rho\, A_{\text{eq}}^\top A_{\text{eq}}$, so its condition number grows like ρ and the subproblems become increasingly hard to solve accurately.

15.3.2 Shifting the penalty: equality constraints

The augmented Lagrangian adds the penalty to the Lagrangian instead of to f:

$$L_\rho(x,\lambda) = f(x) + \lambda^\top h(x) + \frac{\rho}{2}\|h(x)\|^2, \qquad \nabla_x L_\rho = \nabla f(x) + A_{\text{eq}}^\top(\lambda + \rho\, h(x)). \tag{15.6}$$

If x^+ minimizes $L_\rho(\cdot,\lambda)$, then $\nabla f(x^+) + A_{\text{eq}}^\top(\lambda + \rho h(x^+)) = 0$. This is exactly the stationarity condition (15.2) with the multiplier estimate

$$\lambda^+ = \lambda + \rho\, h(x^+). \tag{15.7}$$

Now $h(x^+) = (\lambda^+ - \lambda)/\rho$, so feasibility only requires the multiplier estimates to converge, not $\rho \to \infty$. If $\lambda = \lambda^\star$, then $x^\star$ already minimizes $L_\rho(\cdot,\lambda^\star)$ for every $\rho > 0$: the penalty is *exact* at the right multiplier.

15.3.3 Inequality constraints

Write each inequality with a slack, $g_i(x) + s_i = 0$, $s_i \geq 0$, and apply (15.6) to the equality $g(x) + s = 0$:

$$\min_{x,\, s\geq 0}\ f(x) + \mu^\top(g(x) + s) + \frac{\rho}{2}\|g(x) + s\|^2.$$

The minimization over s separates by component and has a closed form. For fixed x, the unconstrained minimizer of $\mu_i(g_i + s_i) + \frac{\rho}{2}(g_i + s_i)^2$ is $s_i = -g_i - \mu_i/\rho$; projecting onto $s_i \geq 0$ gives $s_i^\star = \max\{0,\, -g_i - \mu_i/\rho\}$. Substituting back:

- if $\mu_i + \rho g_i \leq 0$ (constraint comfortably inactive), then $g_i + s_i^\star = -\mu_i/\rho$ and the term equals $-\mu_i^2/(2\rho)$;
- if $\mu_i + \rho g_i > 0$, then $s_i^\star = 0$ and the term equals $\mu_i g_i + \frac{\rho}{2} g_i^2 = ((\mu_i + \rho g_i)^2 - \mu_i^2)/(2\rho)$.

Both cases combine into the Powell–Hestenes–Rockafellar (PHR) augmented Lagrangian

$$\boxed{L_\rho(x,\lambda,\mu) = f(x) + \lambda^\top h(x) + \frac{\rho}{2}\|h(x)\|^2 + \frac{1}{2\rho}\sum_{i=1}^{m}\Big(\,[\mu_i + \rho\, g_i(x)]_+^2 - \mu_i^2\Big)} \tag{15.8}$$

with $[t]_+ = \max\{0, t\}$, and gradient

$$\nabla_x L_\rho(x,\lambda,\mu) = \nabla f(x) + A_{\text{eq}}^\top(\lambda + \rho\, h(x)) + A_{\text{in}}^\top\,[\mu + \rho\, g(x)]_+\,. \tag{15.9}$$

Applying (15.7) to $g + s^\star$ gives the inequality update

$$\mu^+ = \mu + \rho(g(x^+) + s^\star) = \Big[\mu + \rho\, g(x^+)\Big]_+\,, \tag{15.10}$$

which keeps $\mu^+ \geq 0$ automatically.

Properties of (15.8):

- $L_\rho(\cdot,\lambda,\mu)$ is convex whenever f is (the extra terms are convex functions of affine maps), and continuously differentiable.
- It is *not* twice differentiable where $\mu_i + \rho g_i(x) = 0$. This is why a quasi-Newton inner solver (L-BFGS) is a good fit: it only uses gradients.
- With the updated multipliers, the gradient (15.9) at x^+ is exactly the stationarity residual of the KKT system: $\nabla_x L_\rho(x^+,\lambda,\mu) = \nabla f(x^+) + A_{\text{eq}}^\top\lambda^+ + A_{\text{in}}^\top\mu^+$. The inner solver's gradient norm therefore measures KKT stationarity directly.

15.4 Convergence in the convex case

Dual interpretation. Let $d(\lambda, \mu) = \inf_x L(x, \lambda, \mu)$ be the (concave) dual function. Rockafellar [25] showed that one step of the method of multipliers, minimizing (15.8) and applying (15.7)–(15.10), is exactly one step of the *proximal point algorithm* applied to the dual:

$$(\lambda^+, \mu^+) = \operatorname*{arg\,max}_{\lambda,\, \mu \geq 0} \Big\{ d(\lambda, \mu) - \frac{1}{2\rho} \|(\lambda, \mu) - (\lambda_k, \mu_k)\|^2 \Big\}.$$

Equivalently, it is a gradient ascent step of length ρ on the smoothed dual, whose gradient is the constraint residual. Consequences for convex problems:

- For *any* fixed $\rho > 0$, if (15.1) has a KKT point, the multipliers converge to a dual solution and every limit point of x_k is primal optimal. Unlike the pure penalty method, ρ does not have to go to infinity.
- This remains true when the subproblems are solved inexactly, provided the errors are summable (e.g. inner tolerances $\varepsilon_k \to 0$ geometrically).
- The rate is linear, and the linear constant improves as ρ grows; with $\rho_k \to \infty$ it is superlinear. Larger ρ makes outer iterations more effective but inner problems more ill-conditioned; the practical rule of Section 15.5 balances both.

For nonconvex f the method still works locally (under second-order sufficiency, for ρ above a threshold), but `nabia` targets the convex case.

15.5 Practical algorithm

Measuring infeasibility. For the inequalities, $g_i \leq 0$ alone is not enough: complementarity (15.5) must hold too. We use the measure of Birgin and Martínez [2]

$$V_k = \max \Big\{ \|h(x_k)\|_\infty,\ \|\max\{g(x_k),\, -\mu_k/\rho_k\}\|_\infty \Big\}, \tag{15.11}$$

evaluated *before* the multiplier update. The i-th inequality term is zero iff $g_i \leq 0$ and either $g_i = 0$ or $\mu_i = 0$. Moreover, $\max\{g, -\mu/\rho\} = (\mu^+ - \mu)/\rho$, so V_k also measures how much the multipliers still move.

Penalty update. If the violation did not decrease enough, $V_k > \tau\, V_{k-1}$ (default $\tau = 0.25$), the multipliers alone are not making fast enough progress and the penalty is increased, $\rho_{k+1} = \min\{\gamma \rho_k, \rho_{\max}\}$ (default $\gamma = 10$). Otherwise ρ is kept.

Inexact inner solves. Early outer iterations use inaccurate multipliers, so solving their subproblems to high accuracy is wasted work. The inner tolerance starts at $\varepsilon_0 = \max\{\texttt{tol},\, 10^{-1}\}$ and shrinks as $\varepsilon_{k+1} = \max\{\texttt{tol},\, 0.1\, \varepsilon_k\}$. Each subproblem is warm started at the previous x_k.

Stopping test. The method stops when both KKT residuals are below the user tolerance:

$$\|\nabla_x L_{\rho_k}(x_{k+1}, \lambda_k, \mu_k)\|_\infty \leq \texttt{tol} \quad \text{and} \quad V_k \leq \texttt{tol}.$$

By the identity at the end of Section 2, the first condition is stationarity (15.2) for the updated multipliers. The multipliers are nonnegative by construction, and V_k covers feasibility and complementarity. So the test certifies an approximate KKT point.

When there are no constraints, (15.8) reduces to f, and the implementation solves a single subproblem directly to the tolerance `tol`.

Algorithm 4 Augmented Lagrangian (PHR) with L-BFGS inner solver

Require: x_0, tolerance `tol`, $\rho_0 > 0$, $\gamma > 1$, $\tau \in (0,1)$, $\rho_{\max}$

$\lambda_0 \leftarrow 0$, $\mu_0 \leftarrow 0$, $\varepsilon_0 \leftarrow \max\{\texttt{tol}, 10^{-1}\}$, $V_{-1} \leftarrow \infty$
for $k = 0, 1, \dots,$ `max_iter` $- 1$ **do**
  $x_{k+1} \leftarrow \arg\min_x L_{\rho_k}(x, \lambda_k, \mu_k)$ from x_k, up to $\|\nabla_x L_{\rho_k}\|_\infty \le \varepsilon_k$ ▷ Alg. 5
  $V_k \leftarrow \max\{\|h(x_{k+1})\|_\infty,\ \|\max\{g(x_{k+1}), -\mu_k/\rho_k\}\|_\infty\}$
  $\lambda_{k+1} \leftarrow \lambda_k + \rho_k\, h(x_{k+1})$
  $\mu_{k+1} \leftarrow [\mu_k + \rho_k\, g(x_{k+1})]_+$
  if $\|\nabla_x L_{\rho_k}(x_{k+1}, \lambda_k, \mu_k)\|_\infty \le \texttt{tol}$ **and** $V_k \le \texttt{tol}$ **then**
    return $(x_{k+1}, \lambda_{k+1}, \mu_{k+1})$ ▷ approximate KKT point
  end if
  if $V_k > \tau V_{k-1}$ **then**
    $\rho_{k+1} \leftarrow \min\{\gamma\rho_k, \rho_{\max}\}$
  else
    $\rho_{k+1} \leftarrow \rho_k$
  end if
  $\varepsilon_{k+1} \leftarrow \max\{\texttt{tol},\ 0.1\,\varepsilon_k\}$
end for

15.6 The inner solver: L-BFGS

Each subproblem $\min_x L_\rho(x, \lambda, \mu)$ is smooth and unconstrained. `nabia` solves it with limited-memory BFGS [18, 15], which is also exposed as the standalone optimizer `LBFGS`.

Quasi-Newton direction. BFGS keeps an approximation H_k of the inverse Hessian and moves along $d_k = -H_k \nabla f(x_k)$. With $s_k = x_{k+1} - x_k$, $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$ and $\rho_k = 1/(y_k^\top s_k)$, the update is

$$H_{k+1} = (I - \rho_k s_k y_k^\top)\, H_k\, (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top .$$

L-BFGS never forms H_k. It stores the last M pairs (s_j, y_j) and computes $H_k \nabla f$ with the two-loop recursion (Algorithm 5) in $O(Mn)$ time and memory. The initial matrix is $H_k^0 = \gamma_k I$ with $\gamma_k = s_{k-1}^\top y_{k-1} / y_{k-1}^\top y_{k-1}$, which gives the step a meaningful scale.

Safeguards.

- A pair is stored only if $s^\top y > 10^{-12}\|s\|\|y\|$. This keeps H_k positive definite and d_k a descent direction. For convex f the curvature condition $s^\top y \ge 0$ always holds.
- If d_k is not a descent direction (for example due to round-off), the memory is cleared and a normalized steepest-descent step is taken.
- The step length t is chosen by backtracking ($t \leftarrow 0.5t$) from $t = 1$ until the Armijo condition $f(x_k + t d_k) \le f(x_k) + c_1 t\, \nabla f(x_k)^\top d_k$ holds (default $c_1 = 10^{-4}$). Non-finite trial values are rejected.
- *Approximate Wolfe acceptance.* Close to a minimizer, the decrease $c_1 t\,|\nabla f^\top d|$ that Armijo demands falls below the rounding error in f. The test then stalls even though the gradient, which is still accurate, would allow progress. Following Hager and Zhang [7], a step is also accepted when f is flat to within rounding, $f(x_k + t d_k) \le f(x_k) + \epsilon_f |f(x_k)|$, and the directional derivative satisfies $\sigma\, \nabla f(x_k)^\top d_k \le \nabla f(x_k + t d_k)^\top d_k \le (2c_1 - 1)\, \nabla f(x_k)^\top d_k$ (defaults $\epsilon_f = 10^{-10}$, $\sigma = 0.9$). Without this rule the risk budgeting solver, for example, stalls at an accuracy of about $3 \cdot 10^{-8}$ on problems with hundreds of assets.

Algorithm 5 L-BFGS with Armijo backtracking

Require: x_0, memory M, tolerance ε

for $k = 0, 1, \dots$ **do**
  if $\|\nabla f(x_k)\|_\infty \le \varepsilon$ **then return** x_k
  end if
  $q \leftarrow \nabla f(x_k)$
  for $j = k-1, \dots, k-M$ **do** ▷ newest to oldest
    $\alpha_j \leftarrow \rho_j s_j^\top q; \quad q \leftarrow q - \alpha_j y_j$
  end for
  $r \leftarrow \gamma_k q$
  for $j = k-M, \dots, k-1$ **do** ▷ oldest to newest
    $\beta \leftarrow \rho_j y_j^\top r; \quad r \leftarrow r + (\alpha_j - \beta) s_j$
  end for
  $d_k \leftarrow -r$; **if** $\nabla f(x_k)^\top d_k \ge 0$ **then** clear memory, $d_k \leftarrow -\nabla f(x_k)/\max\{1, \|\nabla f(x_k)\|\}$
  $t \leftarrow 1$; **while** $f(x_k + t d_k) > f(x_k) + c_1 t \nabla f(x_k)^\top d_k$ **do** $t \leftarrow t/2$
  $x_{k+1} \leftarrow x_k + t d_k$; store (s_k, y_k) if $s_k^\top y_k > 0$, dropping the oldest pair beyond M
end for

15.7 Sequential quadratic programming

The augmented Lagrangian method needs many evaluations of f: each outer iteration is a full L-BFGS run. Sequential quadratic programming (SQP) instead builds a quadratic model of the problem at every iterate. It solves that model, which costs no evaluations of f, and takes one step. When f is expensive and n is moderate (SQP stores a dense $n \times n$ matrix), this is usually far cheaper overall.

15.7.1 The QP subproblem

SQP is Newton's method applied to the KKT conditions. For (15.1), linearizing the KKT system at the current iterate x_k gives the optimality conditions of the quadratic program

$$\min_{d \in \mathbb{R}^n} \ \nabla f(x_k)^\top d + \tfrac{1}{2}\, d^\top B_k d \quad \text{s.t.} \quad A_{\text{eq}}(x_k + d) = b_{\text{eq}}, \qquad A_{\text{in}}(x_k + d) \le b_{\text{in}}, \tag{15.12}$$

with $B_k \approx \nabla^2_{xx} L(x_k, \lambda, \mu)$. Because the constraints are linear, two simplifications follow:

- The linearized constraints are the constraints themselves. The QP is feasible whenever (15.1) is, and a full step d lands on a feasible point, even from an infeasible x_k.
- The constraints have zero curvature, so $\nabla^2_{xx} L = \nabla^2 f$. B_k only has to approximate the Hessian of f, and the multipliers do not enter it.

The KKT conditions of (15.12) are

$$\nabla f(x_k) + B_k d + A_{\text{eq}}^\top \lambda + A_{\text{in}}^\top \mu = 0, \quad \mu \ge 0, \quad \mu_i (g_i(x_k) + (A_{\text{in}} d)_i) = 0,$$

plus feasibility. The QP multipliers (λ, μ) are therefore estimates of the multipliers of (15.1). If $d = 0$ solves the QP, then x_k is a KKT point of the original problem.

15.7.2 Damped BFGS Hessian approximation

With $s_k = x_{k+1} - x_k$ and $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$, the BFGS update keeps B_k positive definite only if $s_k^\top y_k > 0$. That always holds for strictly convex f, but can fail otherwise. Powell's

damping [21] replaces y_k by

$$r_k = \theta_k y_k + (1-\theta_k) B_k s_k, \qquad \theta_k = \begin{cases} 1 & \text{if } s_k^\top y_k \geq 0.2\, s_k^\top B_k s_k, \\ \dfrac{0.8\, s_k^\top B_k s_k}{s_k^\top B_k s_k - s_k^\top y_k} & \text{otherwise,} \end{cases}$$

which guarantees $s_k^\top r_k \geq 0.2\, s_k^\top B_k s_k > 0$, and updates

$$B_{k+1} = B_k - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{r_k r_k^\top}{s_k^\top r_k}.$$

$B_0 = I$ is rescaled to $(y_0^\top y_0 / s_0^\top y_0)\, I$ after the first step, as in L-BFGS. Positive definiteness makes every QP strictly convex, so its solution is unique.

15.7.3 Merit function and line search

The full step can increase f or fail to make progress, so the step length t is chosen on the ℓ_1 merit function [8]

$$\phi_\nu(x) = f(x) + \nu\, V_1(x), \qquad V_1(x) = \|h(x)\|_1 + \|[g(x)]_+\|_1.$$

For ν larger than every multiplier, ϕ_ν is an *exact* penalty: its minimizers are the solutions of (15.1).

The QP step is a descent direction. Since the constraints are linear and d satisfies them, $V_1(x_k + td) \leq (1-t) V_1(x_k)$ for $t \in [0,1]$. The directional derivative is therefore $D\phi_\nu(x_k; d) \leq \nabla f^\top d - \nu V_1(x_k)$. From the QP optimality conditions, $A_{\text{eq}} d = -h$, and complementarity gives $\mu^\top A_{\text{in}} d = -\mu^\top g$. Hence

$$\nabla f^\top d = -d^\top B_k d + \lambda^\top h + \mu^\top g \leq -d^\top B_k d + \|(\lambda, \mu)\|_\infty V_1(x_k),$$

using $\mu \geq 0$ and $\mu^\top g \leq \mu^\top [g]_+$. So

$$D\phi_\nu(x_k; d) \leq -d^\top B_k d - (\nu - \|(\lambda,\mu)\|_\infty)\, V_1(x_k) < 0$$

whenever $d \neq 0$ and $\nu > \|(\lambda,\mu)\|_\infty$. The implementation keeps $\nu \leftarrow \max\{\nu, \|(\lambda,\mu)\|_\infty + \delta\}$ with $\delta = 10^{-3}$ and backtracks from $t = 1$ until the Armijo condition $\phi_\nu(x_k + td) \leq \phi_\nu(x_k) + c_1 t\, (\nabla f^\top d - \nu V_1(x_k))$ holds.

Convergence. Every accepted step decreases the merit function, which makes the method robust from poor (even infeasible) starting points. Under standard assumptions (bounded B_k and multipliers) the iterates approach KKT points. Near a solution that satisfies the standard second-order conditions, the full step $t = 1$ is eventually accepted. Convergence is then superlinear with a BFGS approximation, and quadratic with the exact Hessian [19].

Stopping test. At each iterate the QP multipliers give the KKT residuals

$$\|\nabla f(x_k) + A_{\text{eq}}^\top \lambda + A_{\text{in}}^\top \mu\|_\infty, \qquad \max\{\|h(x_k)\|_\infty, \|[g(x_k)]_+\|_\infty\}, \qquad \|\mu \odot g(x_k)\|_\infty,$$

that is, stationarity (which equals $\|B_k d\|_\infty$), feasibility and complementarity. SQP stops when all three are below `tolerance`.

Solving the QP. Without constraints, (15.12) is solved directly by a Cholesky factorization, $d = -B_k^{-1} \nabla f$. Otherwise it is a strictly convex QP with linear constraints, which `AugmentedLagrangian` solves to a tolerance of 10^{-10}. This costs no evaluations of f, and also gives the multipliers.

Algorithm 6 SQP with damped BFGS and ℓ_1 merit line search

Require: x_0 (may be infeasible), tolerance `tol`, $\nu_0 > 0$

$B \leftarrow I$
for $k = 0, 1, \ldots,$ `max_iter` $- 1$ **do**
  $(d, \lambda, \mu) \leftarrow$ solution and multipliers of the QP (15.12) at x_k
  if stationarity, feasibility and complementarity at x_k are $\le$ `tol` **then return** (x_k, λ, μ)
  end if
  $\nu \leftarrow \max\{\nu, \|(\lambda, \mu)\|_\infty + \delta\}$
  $t \leftarrow 1$; **while** $\phi_\nu(x_k + td) > \phi_\nu(x_k) + c_1 t(\nabla f^\top d - \nu V_1(x_k))$ **do** $t \leftarrow t/2$
  if the line search fails **then** reset $B \leftarrow I$ and retry (stop if B already was I)
  $x_{k+1} \leftarrow x_k + td$; $s \leftarrow x_{k+1} - x_k$; $y \leftarrow \nabla f(x_{k+1}) - \nabla f(x_k)$
  $B \leftarrow$ damped BFGS update of B with (s, y) ▷ rescale B first, at $k = 0$
end for

15.7.4 Choosing a constrained solver

	Augmented Lagrangian	SQP
Memory	$O(Mn)$ (L-BFGS pairs)	$O(n^2)$ (dense B)
Evaluations of f	many (one L-BFGS run per outer step)	few (one QP per step)
Work per step	cheap	a QP solve
Local rate	linear	superlinear
Best for	large n, cheap f	moderate n, expensive f

Table 15.2: Trade-offs between the two constrained solvers.

On the 5-variable simplex problem of the test suite, with a non-quadratic f, SQP converges in 12 iterations using 15 evaluations of f. The augmented Lagrangian needs 89 L-BFGS iterations to reach the same point.

15.8 Box constraints

`BoxConstraints` $\ell \le x \le u$ are turned into rows of $A_{\text{in}} x \le b_{\text{in}}$. Every finite u_i gives the row $e_i^\top x \le u_i$, and every finite ℓ_i gives $-e_i^\top x \le -\ell_i$. Infinite bounds produce no row, and an empty bound vector means that side is unbounded. In `result().mu_ineq` the upper-bound multipliers come first (in index order), followed by the lower-bound multipliers. Bounds with $\ell_i > u_i$ are rejected with `std::invalid_argument`. The conversion is the public function `to_linear_constraint` (`Constraints.h`), which both constrained solvers use, so their multiplier vectors follow the same row order.

15.9 Using the implementation

15.9.1 Augmented Lagrangian

```
#include <nabia/optimization/convex/AugmentedLagrangian.h>

// min |x - c|^2  s.t.  sum(x) = 1, x >= 0   (projection onto the simplex)
Eigen::VectorXd c(3);  c << 1.0, -1.0, 0.5;

LinearConstraint lc;
lc.A_eq   = Eigen::MatrixXd::Ones(1, 3);      lc.b_eq   = Eigen::VectorXd::Ones(1);
```

```
lc.A_ineq = -Eigen::MatrixXd::Identity(3, 3); lc.b_ineq = Eigen::VectorXd::Zero(3);

auto params = std::make_unique<ConvexOptimizationParams>(
    ConvexOptimizationParams{3, /*tolerance*/ 1e-8, /*max_iter*/ 100, lc});

AugmentedLagrangian solver(
    std::move(params),
    [&](const Eigen::VectorXd& x) { return (x - c).squaredNorm(); },
    [&](const Eigen::VectorXd& x) -> Eigen::VectorXd { return 2.0 * (x - c); });

Eigen::VectorXd x = solver.optimize(std::nullopt);  // x = (0.75, 0, 0.25)
const auto& r = solver.result();   // r.lambda_eq = (0.5), r.mu_ineq = (0, 2.5, 0)
```

The KKT conditions can be checked by hand: $2(x - c) + \lambda \mathbf{1} - \mu = 0$ gives $\lambda = 0.5$ from the first component (where $\mu_1 = 0$) and $\mu_2 = 2(0 + 1) + 0.5 = 2.5$ for the active bound $x_2 = 0$.

Option	Default	Meaning
`initial_penalty`	10	ρ_0
`penalty_growth`	10	γ, factor applied to ρ when progress stalls
`max_penalty`	10^{10}	$\rho_{\max}$
`required_decrease`	0.25	τ in the test $V_k > \tau V_{k-1}$
`initial_inner_tolerance`	10^{-1}	ε_0 (never below `tolerance`)
`max_inner_iter`	1000	L-BFGS iterations per subproblem
`lbfgs_memory`	10	M, number of stored pairs

Table 15.3: Fields of `AugmentedLagrangianOptions`.

The result type `AugmentedLagrangianResult` reports `x`, `lambda_eq`, `mu_ineq`, `objective`, `constraint_violation` (V_k), `stationarity`, the final `penalty`, the outer and total inner iteration counts, and `converged`.

15.9.2 Sequential quadratic programming

`SQP` has the same constructor and `optimize` signature as `AugmentedLagrangian`, so switching between them is a one-word change:

```
#include <nabia/optimization/convex/SQP.h>

SQP solver(std::move(params), f, grad_f);
Eigen::VectorXd x = solver.optimize(x0);           // x0 may be infeasible
const auto& r = solver.result();   // r.lambda_eq, r.mu_ineq, r.evaluations, ...
```

`SQPOptions` holds `qp_tolerance` (10^{-10}), `qp_max_iter` (200), the line-search constants `merit_c1` (10^{-4}), `backtrack` (0.5) and `max_line_search` (40), and the merit penalty settings `initial_merit_penalty` ($\nu_0 = 1$) and `merit_penalty_margin` ($\delta = 10^{-3}$). `SQPResult` reports the solution, the multipliers, the three KKT residuals, and the iteration and evaluation counts.

15.9.3 L-BFGS on its own

```
#include <nabia/optimization/convex/LBFGS.h>

// Rosenbrock, using a single callback for value and gradient.
auto fg = [](const Eigen::VectorXd& z, Eigen::VectorXd& grad) {
    const double x = z(0), y = z(1);
    grad(0) = -2.0 * (1.0 - x) - 400.0 * x * (y - x * x);
    grad(1) = 200.0 * (y - x * x);
    return (1.0 - x) * (1.0 - x) + 100.0 * (y - x * x) * (y - x * x);
};
Eigen::VectorXd x0(2);  x0 << -1.2, 1.0;
```

```
LBFGS solver(std::make_unique<LBFGSParams>(LBFGSParams{2}), fg);
Eigen::VectorXd x = solver.optimize(x0);          // x = (1, 1)
if (!solver.result().converged()) { /* inspect solver.result().status */ }
```

`LBFGSParams` holds `dim`, `tolerance` (default 10^{-8}), `max_iter` (1000), `memory` (10) and the line-search constants `armijo_c1`, `backtrack`, `max_line_search`, `flat_tolerance` (ϵ_f) and `wolfe_c2` (σ). `LBFGSResult::status` is `Converged`, `MaxIterations` or `LineSearchFailed`.

15.9.4 Scaling advice

The method is invariant to neither the scaling of f nor that of the constraint rows. If the rows of $A_{\text{eq}}, A_{\text{in}}$ have very different norms, or f takes values far from $O(1)$, normalize them first, since $\rho_0 = 10$ and the tolerances assume a well-scaled problem. If the solver ends with `converged == false` and a large `penalty`, the problem may be infeasible or badly scaled.

Chapter 16

Elastic net factor regression: derivations

This note documents the regression module of `nabia`: the elastic net solver (`ElasticNet`) and the `RegressionEngine`. The engine regresses a target series on a set of candidate factors. It chooses how many factors to use and which ones, the penalty λ and the mix α, and measures the uncertainty of the result.

We first derive the elastic net from its optimality conditions:

- the coordinate descent update and soft thresholding;
- the smallest penalty that zeroes every coefficient, $\lambda_{\max}$;
- the role of the intercept and of standardization;
- the grouping effect that makes the elastic net stable with correlated factors;
- the effective degrees of freedom.

We then describe the engine:

- the chronological train/test split;
- time-series cross-validation (blocked K-fold and walk-forward, with a gap);
- the one-standard-error rule;
- the in-sample criteria (BIC, AIC) used when there is no test set;
- exhaustive and forward subset search;
- the moving block bootstrap.

The last sections map each step to the code, show how to use it from C++ and from Python, and list the tests and the limitations.

16.1 Problem and notation

We observe a target y_t (for example a fund's daily return) and N candidate factors $x_{t1}, \dots, x_{tN}$ (market, rate, commodity returns) for $t = 1, \dots, T$, in time order. We look for a sparse linear model

$$y_t = b_0 + \sum_{j \in S} b_j x_{tj} + \varepsilon_t, \qquad S \subseteq \{1, \dots, N\}, \quad k_{\min} \le |S| \le k_{\max},$$

that *extrapolates*, that is, predicts y well on dates after those used to fit it. When only in-sample explanation is wanted, the same machinery chooses the model by an information criterion instead.

Throughout, $X \in \mathbb{R}^{n\times p}$ is the design of the rows and columns at hand (a training window, a fold, a subset), x_j its j-th column, $\|\cdot\|$ the Euclidean norm and $\mathcal{S}(z,t) = \operatorname{sign}(z)\max(|z|-t,0)$ the soft-thresholding operator.

16.2 The elastic net

16.2.1 Objective

For $\lambda \geq 0$ and $\alpha \in [0,1]$, the elastic net in the glmnet parametrization solves

$$\min_{b_0,\,b} \underbrace{\frac{1}{2n}\|y - b_0\mathbf{1} - Xb\|^2}_{\text{fit}} + \lambda\Big(\alpha\|b\|_1 + \frac{1-\alpha}{2}\|b\|_2^2\Big). \tag{16.1}$$

- $\alpha = 1$ is the *lasso*: the ℓ_1 penalty sets coefficients exactly to zero, so it selects factors.
- $\alpha = 0$ is *ridge* regression: the ℓ_2 penalty shrinks coefficients but never zeroes them.
- $\lambda = 0$ is ordinary least squares.

We write $\ell_1 = \lambda\alpha$ and $\ell_2 = \lambda(1-\alpha)$ for the two penalty weights. The intercept is never penalized.

16.2.2 Intercept and standardization

Proposition 16.2.1 (Intercept)**.** *For any b, the optimal intercept in* (16.1) *is $b_0 = \bar{y} - \bar{x}^\top b$, where $\bar{y}$ and $\bar{x}$ are the means of y and of the columns of X. Substituting it leaves problem* (16.1) *without intercept on the centred data $\tilde{y} = y - \bar{y}\mathbf{1}$, $\tilde{X} = X - \mathbf{1}\bar{x}^\top$.*

Proof. The objective is a convex quadratic in b_0 with derivative $-\frac{1}{n}\mathbf{1}^\top(y - b_0\mathbf{1} - Xb) = -(\bar{y} - b_0 - \bar{x}^\top b)$, which vanishes at the stated value. Then $y - b_0\mathbf{1} - Xb = \tilde{y} - \tilde{X}b$. □

Standardization. Write $s_j^2 = \frac{1}{n}\|\tilde{x}_j\|^2$ (the population variance of factor j). With `standardize`, the solver works on $z_j = \tilde{x}_j / s_j$ and finds coefficients β_j for those columns. The coefficients in original units are

$$b_j = \beta_j / s_j, \qquad b_0 = \bar{y} - \bar{x}^\top b.$$

Equivalently, the penalty in original units is $\lambda(\alpha\sum_j s_j|b_j| + \frac{1-\alpha}{2}\sum_j s_j^2 b_j^2)$: each factor is charged in proportion to its volatility. A factor's units, or its leverage (a 2x version of an index), therefore do not change whether it is selected. Without `standardize`, the penalty applies to b directly.

Constant factors. A column whose centred mean square is below $10^{-24}\max(1, \frac{1}{n}\|x_j\|^2)$ is constant up to rounding. It is set to zero, and its coefficient is fixed at 0. The relative threshold is needed because centring a constant column leaves rounding noise of the size of its magnitude.

In the working units, let $v_j = \frac{1}{n}\|z_j\|^2$. Then $v_j = 1$ with standardization, $v_j = s_j^2$ without it, and $v_j = 0$ for a constant column. The problem the solver works on is

$$\min_{\beta}\; F(\beta) = \frac{1}{2n}\|\tilde{y} - Z\beta\|^2 + \ell_1\|\beta\|_1 + \frac{\ell_2}{2}\|\beta\|_2^2. \tag{16.2}$$

16.2.3 Optimality conditions

F is convex, as the sum of a convex quadratic and two norms. With $\ell_2 > 0$ it is strictly convex, so the minimizer is unique. A point β minimizes F if and only if $0 \in \partial F(\beta)$, that is, with residual $r = \tilde{y} - Z\beta$:

$$\frac{1}{n} z_j^\top r - \ell_2 \beta_j \begin{cases} = \ell_1 \operatorname{sign}(\beta_j) & \beta_j \neq 0, \\ \in [-\ell_1, \ell_1] & \beta_j = 0. \end{cases} \tag{16.3}$$

An active factor has its correlation with the residual pinned at $\pm\ell_1$, after the ridge shrinkage. An inactive factor has a correlation with the residual below the threshold. The tests check (16.3) directly on solutions.

16.2.4 The coordinate update

Proposition 16.2.2 (Soft thresholding)**.** *Fix every coefficient but β_j, with $v_j > 0$, and let r be the current residual. The minimizer of F over β_j is*

$$\beta_j^{new} = \frac{\mathcal{S}(\rho_j, \ell_1)}{v_j + \ell_2}, \qquad \rho_j = \frac{1}{n} z_j^\top r + v_j \beta_j^{old}.$$

Proof. Let $r^{(j)} = r + z_j \beta_j^{\text{old}}$ be the partial residual, the residual without factor j. As a function of β_j, up to constants,

$$F = \frac{1}{2n} \|r^{(j)} - z_j \beta_j\|^2 + \ell_1 |\beta_j| + \frac{\ell_2}{2} \beta_j^2 = \frac{v_j + \ell_2}{2} \beta_j^2 - \rho_j \beta_j + \ell_1 |\beta_j| + \text{const},$$

because $\frac{1}{n} z_j^\top r^{(j)} = \frac{1}{n} z_j^\top r + v_j \beta_j^{\text{old}} = \rho_j$. This is a scalar problem $\min_u \frac{a}{2} u^2 - \rho u + \ell_1 |u|$ with $a > 0$. Its subgradient condition is $au - \rho + \ell_1 \partial |u| \ni 0$:

- if $u > 0$, then $u = (\rho - \ell_1)/a$, which needs $\rho > \ell_1$;
- if $u < 0$, then $u = (\rho + \ell_1)/a$, which needs $\rho < -\ell_1$;
- otherwise $u = 0$ is optimal, because $|\rho| \leq \ell_1$ means $0 \in -\rho + [-\ell_1, \ell_1]$.

In every case $u = \mathcal{S}(\rho, \ell_1)/a$. □

After the update the residual is corrected in $O(n)$: $r \leftarrow r - z_j(\beta_j^{\text{new}} - \beta_j^{\text{old}})$. A sweep over all factors costs $O(np)$, and no matrix is ever formed or inverted.

Convergence. The non-smooth part of F is separable across coordinates, and the smooth part is a convex quadratic. Cyclic coordinate descent therefore converges to a minimizer (Tseng, 2001). Each update cannot increase F, because it minimizes F exactly along one coordinate.

Stopping rule. A change Δ_j of coordinate j moves the fitted values by $z_j \Delta_j$, whose mean square is $v_j \Delta_j^2$. The solver stops when, over a full sweep,

$$\max_j v_j \Delta_j^2 < \tau \cdot \tfrac{1}{n} \|\tilde{y}\|^2,$$

that is, when no coordinate moves the fit by more than a fraction τ (`tolerance`, default 10^{-7}) of the target's variance. The rule is the same in standardized and in raw units, and does not depend on the scale of y.

Active-set iterations. After a full sweep, the solver sweeps only the non-zero coefficients until they settle. Then it checks with a new full sweep. It stops when a *full* sweep meets the rule, so a factor that should enter is never missed. Inactive factors usually stay at zero, so most sweeps cost $O(n|A|)$ instead of $O(np)$.

16.2.5 The smallest penalty that zeroes everything

Proposition 16.2.3 ($\lambda_{\max}$)**.** *For $\alpha > 0$, $\beta = 0$ solves* (16.2) *if and only if*

$$\lambda \geq \lambda_{\max} = \frac{\max_j |z_j^\top \tilde{y}|/n}{\alpha}.$$

Proof. At $\beta = 0$, $r = \tilde{y}$ and every coefficient is inactive. By (16.3), 0 is optimal if and only if $|\frac{1}{n} z_j^\top \tilde{y}| \leq \ell_1 = \lambda\alpha$ for every j. □

For ridge ($\alpha = 0$) no finite λ zeroes the coefficients. As in glmnet, α is floored at 10^{-3} in this formula, so ridge also gets a finite path that starts strongly shrunk.

The code computes $\lambda_{\max}$ with the same arithmetic as the first coordinate update and multiplies it by $1 + 10^{-12}$. Without that factor, rounding in the division can leave $\lambda\alpha$ a few ulps below $|\rho_j|$, which leaves a coefficient of order 10^{-17} instead of an exact zero.

16.2.6 Lambda grid and warm starts

The automatic grid has L (`n_lambdas`) values, log-spaced from $\lambda_{\max}$ down to $r\lambda_{\max}$ (`lambda_min_ratio`):

$$\lambda_i = \lambda_{\max}\, r^{\,i/(L-1)}, \qquad i = 0, \ldots, L-1.$$

The path is solved in this decreasing order, and each solution starts from the previous one (a *warm start*). The first solution is $\beta = 0$ exactly. Each later solution is close to the previous one, so it takes a few sweeps. A user-supplied grid is sorted in decreasing order for the same reason.

16.2.7 Grouping effect: why the elastic net for correlated factors

The lasso picks one factor out of a group of highly correlated ones, somewhat arbitrarily, and may switch between them from sample to sample. The ridge term stops this.

Proposition 16.2.4 (Grouping)**.** *Let the factors be standardized ($\frac{1}{n}\|z_j\|^2 = 1$), let $\ell_2 > 0$, and let factors i, j have correlation ρ_{ij} and coefficients of the same sign. Then*

$$|\beta_i - \beta_j| \leq \frac{\|\tilde{y}\|}{\sqrt{n}\,\ell_2}\sqrt{2(1-\rho_{ij})}.$$

Proof. Both coefficients satisfy (16.3) with the same sign s: $\frac{1}{n} z_i^\top r - \ell_2\beta_i = \ell_1 s = \frac{1}{n} z_j^\top r - \ell_2\beta_j$. Hence $\ell_2(\beta_i - \beta_j) = \frac{1}{n}(z_i - z_j)^\top r$, and by Cauchy–Schwarz $|\beta_i - \beta_j| \leq \frac{1}{n\ell_2}\|z_i - z_j\|\|r\|$. Here $\|z_i - z_j\|^2 = 2n(1-\rho_{ij})$. Also $F(\hat{\beta}) \leq F(0)$ gives $\|r\| \leq \|\tilde{y}\|$. □

As $\rho_{ij} \to 1$, the two coefficients are forced together, and they share the loading instead of one taking it all. In the factor replication example (S&P 500 against Nasdaq 100, correlation 0.93), $\alpha < 1$ keeps the split stable across bootstrap samples.

16.2.8 Effective degrees of freedom

The information criteria of §16.3.4 need the number of parameters an elastic net fit really uses. Penalized coefficients count for less than one.

Proposition 16.2.5 (Degrees of freedom)**.** *Let A be the active set of the solution and Z_A its columns. For a y at which the active set and the signs are locally constant,*

$$\mathrm{df} = \mathbb{1}[\textit{intercept}] + \operatorname{tr}\big(Z_A(Z_A^\top Z_A + n\ell_2 I)^{-1} Z_A^\top\big) = \mathbb{1}[\textit{intercept}] + \sum_i \frac{d_i}{d_i + n\ell_2},$$

where the d_i are the eigenvalues of $Z_A^\top Z_A$.

Proof. For a fit $\hat{y} = g(y)$, Stein's lemma gives the unbiased degrees of freedom $\mathrm{df} = \sum_t \partial \hat{y}_t / \partial y_t = \mathrm{tr}(\partial g / \partial y)$ (Efron, 2004). On the active set, (16.3) with fixed signs s_A reads $\frac{1}{n} Z_A^\top (\tilde{y} - Z_A \beta_A) - \ell_2 \beta_A = \ell_1 s_A$, so

$$\beta_A = (Z_A^\top Z_A + n\ell_2 I)^{-1} (Z_A^\top \tilde{y} - n\ell_1 s_A).$$

The second term does not depend on y locally. Hence $\partial(Z_A\beta_A)/\partial\tilde{y} = H = Z_A(Z_A^\top Z_A + n\ell_2 I)^{-1} Z_A^\top$. With an intercept, $\hat{y} = \bar{y}\mathbf{1} + Z_A\beta_A$ and $\tilde{y} = (I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top)y$. The centred columns are orthogonal to $\mathbf{1}$, so the trace is $1 + \mathrm{tr}\, H$. Diagonalizing $Z_A^\top Z_A = UDU^\top$ gives $\mathrm{tr}\, H = \mathrm{tr}\left(D(D + n\ell_2 I)^{-1}\right)$. □

Special cases:

- the lasso ($\ell_2 = 0$) with Z_A of full rank gives $\mathrm{df} = |A|$ (+1), the result of Zou, Hastie and Tibshirani (2007);
- ridge gives the familiar $\sum d_i/(d_i + n\ell_2)$;
- OLS gives the number of columns.

Eigenvalues below $10^{-10} \max_i d_i$ are treated as zero, which handles a rank-deficient active set.

16.3 The regression engine

16.3.1 Overview

Given $y \in \mathbb{R}^T$, $X \in \mathbb{R}^{T \times N}$ and the parameters, the engine runs these steps:

1. **Split** the rows in time order into training and test sets (§16.3.2).
2. For every candidate subset S (§16.3.5):
   (a) choose (α, λ) by time-series cross-validation on the training rows (§16.3.3);
   (b) fit the elastic net on all training rows at that (α, λ);
   (c) score it by the test MSE, or by an in-sample criterion when there is no test set (§16.3.4).
3. **Keep** the subset with the lowest score. Optionally refit it on every row (§16.3.6) and bootstrap it (§16.3.7).

16.3.2 Chronological split

The training set is the first

$$n = \min\left(T, \lfloor fT + 10^{-9} \rfloor\right)$$

rows, with $f =$ `train_fraction`. The test set is the remaining $T - n$ rows. The 10^{-9} makes 0.7×10 count as 7 despite rounding. Rows are never shuffled.

- The test set lies after every training row, so its error measures *extrapolation*: the model faces a period it has never seen, with its own regimes and correlations.
- With $f = 1$ there is no test set, and the engine works *in sample*: subsets are compared by `in_sample_criterion`.

16.3.3 Choosing λ and α: time-series cross-validation

Grid. For each α_a of `alphas`, the engine builds a λ grid $\lambda_{a,0} > \cdots > \lambda_{a,L-1}$. It is the automatic grid from $\lambda_{\max}$ of the subset's training rows, or the user's `lambdas`. The grid differs by α, because $\lambda_{\max} \propto 1/\alpha$, so cells are compared by position. If the grid has a single point, there is nothing to choose, and cross-validation is skipped.

Folds. The n training rows are cut into contiguous blocks with boundaries $\lfloor kn/B \rfloor$. With gap g (`validation_gap`):

Blocked K-fold ($B = K$ blocks). Fold k validates on block k and trains on every row before the block minus g, and every row after it plus g.

Walk-forward ($B = K + 1$ blocks). Fold $k = 1, \ldots, K$ validates on block k and trains on the rows before it minus g. Training always precedes validation, as in live use. The cost is smaller training sets in the first folds.

The gap removes the rows next to the validation block from training. Returns are serially dependent, through overlapping horizons or volatility clustering, so a training row adjacent to a validation row leaks information about it. A fold with no validation rows, or with fewer than 2 training rows, is an error.

Estimate and standard error. For every fold k and grid cell (a, i), the path for α_a is fitted on the fold's training rows. Its validation MSE is $m_k(a, i)$. With K folds:

$$\mathrm{CV}(a,i) = \frac{1}{K}\sum_k m_k(a,i), \qquad \mathrm{SE}(a,i) = \sqrt{\frac{1}{K}\cdot\frac{1}{K-1}\sum_k \big(m_k(a,i) - \mathrm{CV}(a,i)\big)^2}.$$

These are the mean of the fold errors and its standard error, treating the folds as independent replicates. The chosen cell is $(a^*, i^*) = \arg\min \mathrm{CV}$, with ties going to the first cell, so the result is deterministic.

One-standard-error rule. With `one_standard_error_rule`, the engine stays on row a^* and takes the *largest* λ whose error is within one standard error of the minimum:

$$i^{**} = \min\{\, i \le i^* :\ \mathrm{CV}(a^*, i) \le \mathrm{CV}(a^*, i^*) + \mathrm{SE}(a^*, i^*)\,\}.$$

The CV minimum is a noisy estimate. A model that is statistically as good but more strongly penalized (sparser, more stable) is preferred (Hastie, Tibshirani and Friedman, 2009, §7.10).

Final fit of the subset. The subset is refitted on all n training rows along the grid of α_{a^*}, from $\lambda_{a^*,0}$ down to the chosen value. This warm-starts it exactly as during validation.

16.3.4 Scores

Out of sample ($T > n$). The score is the test MSE, $\frac{1}{T-n}\sum_{t>n}(y_t - \hat{y}_t)^2$. The engine also reports the test R^2:

$$R^2_{\text{test}} = 1 - \frac{\sum_{t>n}(y_t - \hat{y}_t)^2}{\sum_{t>n}(y_t - \bar{y}_{\text{train}})^2}.$$

It is measured against the *training* mean, the forecast available without the model (Campbell and Thompson, 2008). It is negative when the model does worse than that mean.

In sample ($T = n$). The score is an information criterion on the training rows. Under Gaussian errors, the log-likelihood maximized over the noise variance is $-2\log\hat{L} = n\log(2\pi\,\mathrm{MSE}) + n$. Dropping the constants, which are the same for every model:

$$\mathrm{BIC} = n\log\mathrm{MSE} + \mathrm{df}\log n, \qquad \mathrm{AIC} = n\log\mathrm{MSE} + 2\,\mathrm{df},$$

with the effective df of Proposition 16.2.5.

- BIC is consistent: with a true sparse model it selects it with probability $\to 1$.
- AIC targets prediction error and admits more factors.
- `MSE` alone never pays for a factor. Among nested least-squares fits it always prefers the largest subset. It is there for comparison.

16.3.5 Subset search

With $k_{\min}$ = `min_factors` and $k_{\max}$ = `max_factors` (0 meaning N):

Exhaustive evaluates every subset of each size, in lexicographic order, which is $\sum_{k=k_{\min}}^{k_{\max}} \binom{N}{k}$ subsets. For each size it keeps the subset with the lowest score. For example, $N = 14$ and sizes 1–4 give $14 + 91 + 364 + 1001 = 1470$ subsets.

Forward is greedy. Starting from the empty set, step $k = 1, \ldots, k_{\max}$ adds the remaining factor whose addition gives the lowest score. That costs $\sum_{k=1}^{k_{\max}} (N - k + 1)$ evaluations, which is linear in N for a fixed $k_{\max}$. The best subset of size k is recorded for $k \geq k_{\min}$. Forward search can miss the best subset when factors only help together, but it reaches it in the common case of nested good models.

Auto uses Exhaustive if the count above is at most `max_exhaustive_subsets` (default 10 000), and Forward otherwise. The count is computed in floating point and saturates, so it never overflows.

The selected model is the best of the per-size winners (`best_by_size`), by score. Every evaluation is independent, so the subsets of one size (exhaustive) or one step (forward) are evaluated in parallel (§16.3.8).

Remark 16.3.1 (Selection on the test set). Choosing among many subsets by their test error makes the winner's test error an *optimistic* estimate of its error on new data, because the minimum of many noisy errors is biased down. A factor can win by noise. The bootstrap of §16.3.7 helps detect this: a spurious factor's interval contains zero. For a clean final number, keep a later period out of the engine entirely.

16.3.6 Refit on the full sample

With `refit_on_full_sample` and a test set, the chosen factors are refitted on all T rows at the chosen (α, λ). The reported coefficients and intercept are then the refit, which uses the most recent data. The predictions, residuals and scores remain those of the training-rows model, so the test rows stay out of sample.

16.3.7 Moving block bootstrap

With B = `bootstrap_samples` > 0, the engine estimates the sampling distribution of the selected model's coefficients. Each sample $b = 1, \ldots, B$:

1. draws block starts s uniformly in $\{0, \ldots, n - \ell\}$, with ℓ = `bootstrap_block_size` $\leq n$, and concatenates the blocks of rows $s, \ldots, s + \ell - 1$ until there are n rows (the last block is truncated);
2. refits the elastic net on those rows, for the selected factors, at the selected (α, λ);
3. records the coefficients, the intercept and the MSE of that model on the test rows.

With $\ell = 1$ this is the ordinary i.i.d. bootstrap. Daily returns are not independent: their volatility clusters. Blocks keep the dependence within ℓ consecutive rows, and the moving block bootstrap is consistent for weakly dependent series when $\ell \to \infty$ and $\ell/n \to 0$ (Künsch, 1989). A block of about a month (20 rows) is a reasonable choice for daily data.

Summaries. For each selected factor, with draws $c_1, \ldots, c_B$:

- the mean, and the standard error $\sqrt{\frac{1}{B-1}\sum_b (c_b - \bar{c})^2}$;
- the percentile interval at level γ (`confidence_level`): the empirical quantiles at $q = (1-\gamma)/2$ and $1-q$. With the sorted draws $c_{(0)} \le \cdots \le c_{(B-1)}$, the quantile at q interpolates linearly at position $q(B-1)$;
- the *selection frequency*, the fraction of samples with $c_b \neq 0$. It is meaningful for $\alpha > 0$; for ridge every coefficient is non-zero, so it is always 1;
- the intercept's mean and standard error, and the mean and percentile interval of the test MSE.

What the bootstrap does not cover. The resamples refit the *selected* subset at the *selected* (α, λ). The intervals are conditional on that choice: they measure estimation noise, not the uncertainty of selecting the factors. Redoing the whole search on every resample would cost B times the search.

Reproducibility. Sample b uses its own Mersenne Twister, seeded with `std::seed_seq` from the 64-bit `bootstrap_seed` and b. The draws do not depend on which thread runs which sample.

16.3.8 Parallelism and determinism

Subset evaluations and bootstrap samples run on `nabia::detail::parallel_for` with `num_threads` threads (0 = all cores). Every task writes only its own slot. Selections use a sequential argmin with first-index tie-breaking, and random draws are seeded per sample. Results are therefore bit-identical for every thread count, and a test checks this.

16.3.9 Complexity

One elastic net fit along an L-point path costs about $O(\text{sweeps} \cdot n|S|)$. One subset evaluation costs $|\text{alphas}| \times (K+1)$ path fits. The search multiplies that by the number of subsets, and the bootstrap adds B single fits. In the factor replication example (14 factors, sizes 1–4 exhaustive, 3 alphas × 30 lambdas, 5 folds, about 2 600 training rows, 500 bootstrap samples), the whole example script, with its other variants, runs in about 4 s on a laptop.

16.4 Implementation

16.4.1 Code map

Paths are relative to the C++ project, `cpp/`.

Concept	Code
Parameters, fit, class	`include/nabia/regression/ElasticNet.h`: `ElasticNetParams`, `ElasticNetFit`, `ElasticNet`
Centring, scaling (§16.2.2)	`src/regression/ElasticNet.cpp`: `standardize_data`
Coordinate descent (Prop. 16.2.2)	`CoordinateDescent::solve`, `CoordinateDescent::sweep`, `soft_threshold`
$\lambda_{\max}$, grid (Prop. 16.2.3)	`nabia::regression::lambda_max`, `lambda_grid`
Path and warm starts	`nabia::regression::elastic_net_path`
Degrees of freedom (Prop. 16.2.5)	`degrees_of_freedom`
Back to original units	`make_fit`
Engine parameters and results	`include/nabia/regression/RegressionEngine.h`
Split, search, final model	`src/regression/RegressionEngine.cpp`: `run_regression_engine`
Folds (§16.3.3)	`make_folds`
CV, 1-SE rule, scores	`evaluate`, `in_sample_criterion`, `r_squared`
Subset search (§16.3.5)	`exhaustive_search`, `forward_search`, `count_subsets`
Bootstrap (§16.3.7)	`bootstrap`, `percentile`
Python bindings	`bindings/src/bind_regression.cpp`
pandas interface	`bindings/nabia_python_bridge/frames.py`: `regress`
Tests	`tests/test_regression.cpp`, `../nabia_python/tests/bridge/test_regression.py`

16.4.2 Parameters

Field (`RegressionEngineParams`)	Default	Meaning
`min_factors`, `max_factors`	1, 0	subset sizes $k_{\min}..k_{\max}$; $k_{\max} = 0$ means all
`search`	`Auto`	`Exhaustive`, `Forward` or `Auto`
`max_exhaustive_subsets`	10 000	Auto: largest exhaustive search
`train_fraction`	0.7	f; 1 = in sample
`in_sample_criterion`	`BIC`	`BIC`, `AIC` or `MSE` (when $f = 1$)
`refit_on_full_sample`	false	report coefficients refitted on all rows
`alphas`	0, .25, .5, .75, 1	mixes α to try
`lambdas`	empty	fixed grid; empty = automatic
`n_lambdas`, `lambda_min_ratio`	50, 10^{-3}	automatic grid
`validation`	`BlockedKFold`	or `WalkForward`
`validation_folds`, `validation_gap`	5, 0	K and g
`one_standard_error_rule`	false	sparser model within 1 SE
`bootstrap_samples`, `bootstrap_block_size`	0, 1	B (0 = off) and ℓ
`confidence_level`, `bootstrap_seed`	0.95, 42	intervals and seed
`fit_intercept`, `standardize`	true, true	see §16.2.2
`tolerance`, `max_iter`	10^{-7}, 10^{5}	solver stopping rule and sweep cap
`num_threads`	0	0 = all cores

Invalid values throw `std::invalid_argument` at construction, with a message naming the field. Python raises `ValueError`.

16.4.3 Usage from C++

```
#include "nabia/regression/RegressionEngine.h"

auto params = std::make_unique<RegressionEngineParams>();
params->min_factors = 1;
params->max_factors = 4;
params->search = SubsetSearch::Exhaustive;
params->train_fraction = 0.7;                // last 30% of the rows out of sample
params->alphas = {0.0, 0.5, 1.0};
params->n_lambdas = 30;
```

```
params->validation = ValidationScheme::BlockedKFold;
params->bootstrap_samples = 500;
params->bootstrap_block_size = 20;           // keep about a month of dependence

RegressionEngine engine(std::move(params));
// Index-free: y is T, X is T x N (rows = time), names optional.
RegressionComputation r = engine.run(y, X, names);
r.factor_names; r.coefficients; r.alpha; r.lambda;  // the selected model
r.best.test_r2; r.bootstrap->lower; r.bootstrap->upper;

// With an index: target and factors must share it (align with drop_missing first).
RegressionResult<Date> rr = engine.run(target_series, factor_frame);
rr.predictions; rr.residuals; rr.test_start;

// A single elastic net, or a whole path.
ElasticNet en(std::make_unique<ElasticNetParams>(ElasticNetParams{.lambda = 1e-4, .alpha = 0.5}));
en.fit(X, y);
const ElasticNetFit& fit = en.result();      // coefficients in original units
auto grid = nabia::regression::lambda_grid(nabia::regression::lambda_max(X, y, 1.0), 60, 1e-4);
auto path = nabia::regression::elastic_net_path(X, y, ElasticNetParams{.alpha = 1.0}, grid);
```

16.4.4 Usage from Python

The module `nabia_python_bridge.regression` exposes the same classes, with `lambda` spelled `lambda_`. Arrays are `float64` numpy arrays, and the computation runs with the GIL released. `frames.regress` adds pandas alignment and labels:

```
import numpy as np
from nabia_python_bridge import regression, frames

# numpy: y (T,), X (T, N)
r = regression.RegressionEngine(min_factors=1, max_factors=4, train_fraction=0.7,
                                search=regression.SubsetSearch.Exhaustive,
                                alphas=[0.0, 0.5, 1.0], n_lambdas=30,
                                bootstrap_samples=500, bootstrap_block_size=20).run(y, X, names)
r.factor_names, r.coefficients, r.alpha, r.lambda_
r.best.test_r2, r.validation_curve.mse, r.bootstrap.lower, r.bootstrap.upper

# In sample, chosen by BIC
regression.RegressionEngine(max_factors=5, train_fraction=1.0,
                            in_sample_criterion=regression.InSampleCriterion.BIC).run(y, X, names)

# pandas: aligned on the common index, rows with a missing value dropped
out = frames.regress(fund_returns, factor_returns, min_factors=1, max_factors=4,
                     validation="WalkForward", bootstrap_samples=500, bootstrap_block_size=20)
out.coefficients        # Series indexed by factor name
out.bootstrap           # DataFrame: mean, std_error, lower, upper, selection_frequency
out.best_by_size        # DataFrame: one row per subset size
out.predictions, out.residuals, out.test_start

# The elastic net itself
lmax = regression.lambda_max(X, y, alpha=1.0)
path = regression.elastic_net_path(X, y, regression.ElasticNetParams(alpha=1.0),
                                   regression.lambda_grid(lmax, 60, 1e-4))
```

16.4.5 Worked example: recovering a known factor mix

A former example of the project (factor replication, since removed) built a "fund" from real daily returns (2012–2026):

$$\text{fund}_t = 0.0001 + 0.50\,\text{NDX}_t + 0.30\,\text{Gold}_t + 0.20\,10\text{y}_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}\big(0, (0.02/\sqrt{252})^2\big).$$

It then lets the engine choose 1 to 4 factors out of 14 candidates. The candidates include close substitutes: the S&P 500 ($\rho = 0.93$ with the Nasdaq), silver, and 5y/30y bonds. With a 70/30 split, 3 alphas, 30 lambdas, 5 blocked folds and 500 bootstrap samples in 20-day blocks:

Factor	True	Estimate	95% bootstrap interval
Nasdaq 100	0.50	0.497	[0.493, 0.500]
Gold	0.30	0.300	[0.295, 0.305]
10y notes	0.20	0.193	[0.175, 0.212]
Copper (not in the fund)	0	0.003	[−0.002, 0.007]

- The test R^2 is 0.981, and the out-of-sample tracking error is 1.98%/yr against the 2% noise.
- Copper wins the fourth slot on the test set by noise (the remark of §16.3.5), and its interval contains zero.

The script plots the lasso path, the best R^2 by subset size, the validation curve with its standard errors, the bootstrap intervals against the true weights, and the cumulative fund against its replication.

16.5 Verification

`tests/test_regression.cpp` (Catch2) checks:

- $\lambda = 0$ reproduces OLS;
- $\alpha = 0$ reproduces the ridge closed form $(\tilde{X}^\top \tilde{X} + n\lambda I)^{-1} \tilde{X}^\top \tilde{y}$;
- solutions satisfy the optimality conditions (16.3);
- $\lambda_{\max}$ zeroes every coefficient exactly, the next point of the grid does not, and the grid runs from $\lambda_{\max}$ to $r\lambda_{\max}$;
- constant factors get a zero coefficient;
- exhaustive and forward search recover the true factors out of sample, and Auto switches to Forward above the limit;
- in-sample BIC rejects noise factors;
- a single-point grid skips validation;
- walk-forward validation works with the 1-SE rule;
- bootstrap intervals cover the true coefficients;
- results, bootstrap draws included, are identical for 1 and 4 threads;
- refitting keeps the out-of-sample predictions;
- the `DataFrame` interface keeps names and the index;
- invalid parameters and data throw.

`nabia_python/tests/bridge/test_regression.py` repeats the key checks through Python. The suites also run under AddressSanitizer.

16.6 Limitations

- The model is linear with constant coefficients. Time-varying exposures need rolling windows, or a state-space model such as the Kalman filter of the forecasting module.
- The bootstrap is conditional on the selected subset and hyperparameters (§16.3.7).
- Exhaustive search grows as $\binom{N}{k}$. Forward search is greedy.
- The test error of the selected model is optimistic when many subsets are compared.

References

- J. Friedman, T. Hastie, R. Tibshirani (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software* 33(1).
- H. Zou, T. Hastie (2005). Regularization and variable selection via the elastic net. *JRSS B* 67(2).
- H. Zou, T. Hastie, R. Tibshirani (2007). On the "degrees of freedom" of the lasso. *Annals of Statistics* 35(5).
- P. Tseng (2001). Convergence of a block coordinate descent method for nondifferentiable minimization. *JOTA* 109(3).
- B. Efron (2004). The estimation of prediction error: covariance penalties and cross-validation. *JASA* 99(467).
- H. Künsch (1989). The jackknife and the bootstrap for general stationary observations. *Annals of Statistics* 17(3).
- J. Campbell, S. Thompson (2008). Predicting excess stock returns out of sample. *Review of Financial Studies* 21(4).
- T. Hastie, R. Tibshirani, J. Friedman (2009). *The Elements of Statistical Learning*, 2nd ed., Springer.

Part V

Application, Python bindings and packaging

Chapter 17

Command-line application

`nabia_rb_backtest --prices prices.csv --config config.json --output out.csv`:

- **Prices** come from a CSV with ISO dates and one column per asset. Empty cells are missing values.
- **Alignment**: dates where any asset is missing are dropped, and the drop is reported.
- **Returns**: simple returns are computed after alignment.
- **Parameters** come from a JSON file with the keys of Table 9.1, plus an optional `start` and `end`. Unknown keys are rejected, so a typo fails loudly. Budgets can be a list or an object keyed by asset name.
- **Output**: a CSV row per date with the weights, the risk contributions on rebalance dates, turnover, costs, returns, PnL, equity and drawdown, and an optional metrics CSV.

Code

entry point — `cpp/apps/rb_backtest/main.cpp:94--139`
JSON parser — `cpp/apps/rb_backtest/config.cpp:68--163`
CSV reader — `cpp/apps/rb_backtest/csv_io.cpp:156--214`
CSV writer — `cpp/apps/rb_backtest/csv_io.cpp:216--259`

Chapter 18

Backtest bindings for the Python package

The module `nabia_python_bridge._rb_backtest` reuses the application's pipeline and JSON parser, so the command line and Python accept the same parameters with the same error messages. The `nabia` package of the Python project wraps it in pandas objects (`nabia.backtest`, `nabia.latest_weights`; Chapter 21). Data crosses the boundary in simple forms:

- dates as `int64` days since 1970-01-01 (numpy `datetime64[D]`);
- prices as a `float64` matrix;
- parameters as JSON text.

The GIL is released during the computation, so the engine's threads run in parallel. `latest_weights` solves only the last rebalance: the target weights to hold now.

Code 18.1: Shared preparation: parse, align, window, returns, budgets
`cpp/bindings/src/rb_backtest_module.cpp:65-96`

```
Prepared prepare(const DaysArray& days, const std::vector<std::string>& assets, const MatrixXd& prices,
                 const std::string& config_json) {
    if (prices.rows() != days.size() || prices.cols() != static_cast<Index>(assets.size())) {
        throw std::invalid_argument("prices must be (dates x assets): got " + std::to_string(prices.
    ↪ rows()) + " x " +
                                    std::to_string(prices.cols()) + " for " + std::to_string(days.size
    ↪ ()) +
                                    " dates and " + std::to_string(assets.size()) + " assets");
    }
    Prepared p;
    p.config = nabia::app::parse_config(config_json);
    DataFrame<Date> px(to_dates(days), assets, prices);
    const Index read = px.rows();
    px = px.drop_missing();
    p.dropped_dates = read - px.rows();
    if (px.rows() >= 2 && (p.config.start || p.config.end)) {
        px = px.between(p.config.start.value_or(px.time(0)), p.config.end.value_or(px.time(px.rows() -
    ↪ 1)));
    }
    if (px.rows() < 2) {
        throw std::invalid_argument("need at least 2 dates with a price for every asset");
    }
    for (Index t = 0; t < px.rows(); ++t) {
        for (Index j = 0; j < px.cols(); ++j) {
            if (!(px.values()(t, j) > 0.0)) {
                throw std::invalid_argument("prices must be positive for returns: " + px.columns()[
    ↪ static_cast<std::size_t>(j)] +
                                            " is " + std::to_string(px.values()(t, j)) + " on " +
                                            nabia::app::format_date(px.time(t)));
            }
        }
```

```
    }
    p.returns = px.pct_change();
    p.config.params.budgets = nabia::app::resolve_budgets(p.config, assets);
    return p;
}
```

Code

`backtest` binding — `cpp/bindings/src/rb_backtest_module.cpp:119--160`
`latest_weights` binding — `cpp/bindings/src/rb_backtest_module.cpp:162--189`
module definition — `cpp/bindings/src/rb_backtest_module.cpp:193--206`

Chapter 19

The full library in Python: `nabia_python_bridge`

The pybind11 package `nabia_python_bridge` exposes *every* public class and function of the C++ library, for research and scripting from Python. Its sources are in `bindings`: the C++ of the modules in `src`, and the Python files of the package (re-exports and the pandas interface `frames`) in `nabia_python_bridge`.

19.1 Modules

Module	Contents
`stats`	`mean`, `variance`, `stddev`, `autocorrelation`; `normal_pdf/cdf/quantile` (vectorized)
`signal`	`EWMA`, `ButterworthBandPass` (streaming `process`, batch `apply`), `moving_average`, `exponential_moving_average`, `diff`
`forecast`	`KalmanFilter` (local level, local linear trend, custom), `GaussianProcess` with squared exponential, Matérn 5/2, periodic, sum and product kernels (`k1 + k2`, `k1 * k2`), `SimpleExponentialSmoothing`, `Forecast`
`optimization`	`LBFGS`, `AugmentedLagrangian`, `SQP`, `NelderMead`, `DifferentialEvolution` with Python objectives; `BoxConstraints`, `LinearConstraint`
`risk`	`Volatility`, `ValueAtRisk`, `ExpectedShortfall`, `RiskBudgeting`
`strategy`	`RiskBudgetingPortfolio`, `compute_performance`, `equity_curve`, `drawdown_series`
`regression`	`ElasticNet`, `elastic_net_path`, `lambda_max`, `lambda_grid`, `RegressionEngine` (§11)
`frames`	pandas interface: `backtest`, `regress`, `performance`, `metrics_dict`

The `TimeSeries` and `DataFrame` containers are not bound, because pandas does their job in Python. Where an engine takes indexed data, `frames` maps the pandas index to row positions and back.

19.2 Design

Parameters as keyword arguments. Every parameter struct is built from keyword arguments, and a model accepts the fields directly: `signal.EWMA(alpha=0.2)`. Each keyword is assigned through the bound attribute, so the attribute's type checks apply, and an unknown name raises `AttributeError` instead of being silently ignored. `lambda` is a Python keyword, so it is spelled `lambda_`.

Arrays. Vectors and matrices are `float64` numpy arrays. Matrices have one row per observation, as in the C++ code. Results are copies, so they stay valid after the next run.

Errors. `std::invalid_argument` becomes `ValueError`, and "not fitted" (`std::logic_error`) becomes `RuntimeError`.

GIL. Fits, backtests, regressions and optimizations release the GIL, so the engines' own threads run in parallel. A Python objective is wrapped in a `PyCallable`. Copies of it share one pointer, and every call, and the final release, takes the GIL. C++ can therefore call it from any thread. It is only faster with threads if the objective itself releases the GIL (numpy on large arrays). A Python exception raised in an objective propagates out of `optimize()`.

Templates. `RiskBudgeting<Measure>` is a template. The binding wraps its three instantiations in one `AnyRiskBudgeting` type (a `std::variant`), so Python sees a single `RiskBudgeting` class that accepts any of the three measures.

Kernels are polymorphic and owned by `unique_ptr` in C++. In Python they are shared objects. `GaussianProcessParams` clones the kernel when it is set, so fitting never modifies the user's kernel.

Readable results. The Python package re-exports each submodule and gives every result struct a `__repr__` listing its fields.

Code 19.1: Parameter structs from keyword arguments
`cpp/bindings/src/bridge.hpp:27-36`

```
std::unique_ptr<T> from_kwargs(const py::kwargs& kwargs) {
    auto value = std::make_unique<T>();
    {
        const py::object self = py::cast(value.get(), py::return_value_policy::reference);
        for (const auto& [name, item] : kwargs) {
            py::setattr(self, name, item);
        }
    }
    return value;
}
```

Code 19.2: A Python callable usable from any C++ thread
`cpp/bindings/src/bridge.hpp:61-77`

```
class PyCallable {
public:
    explicit PyCallable(py::function f)
        : f_(new py::function(std::move(f)), [](py::function* p) {
              const py::gil_scoped_acquire gil;
              delete p;
          }) {}

    template <typename R, typename... Args>
    R call(const Args&... args) const {
        const py::gil_scoped_acquire gil;
        return (*f_)(args...).template cast<R>();
    }

private:
    std::shared_ptr<py::function> f_;
};
```

19.3 pandas interface

- `frames.backtest(returns_df, params=None, budget_schedule=None, **fields)` runs `RiskBudgetingPortfolio`. Budgets may be a mapping by column name, and the schedule is a DataFrame indexed by date. It returns weights, target weights, risk contributions, leverage, a performance frame and the metrics, all labelled.
- `frames.regress(target, factors_df, params=None, **fields)` aligns the target and the factors on their common index and drops rows with a missing value. It then runs the

regression engine and returns the coefficients by factor name, predictions and residuals by date, the first out-of-sample date, the best subset of each size and the bootstrap table.

- In both, enum fields also accept member names (`risk_measure="ValueAtRisk"`, `validation="WalkForward"`), and the given parameters object is copied, never modified.

Code 19.3: `frames.regress`: alignment, engine and labelled results
`cpp/bindings/nabia_python_bridge/frames.py:125-177`

```
def regress(target: pd.Series, factors: pd.DataFrame, params: Any = None, **overrides: Any) ->
    ↪ RegressionFrames:
    """Elastic net regression of ‘target‘ on ‘factors‘ with factor, lambda and alpha selection.

    The two are aligned on their common index and rows with a missing value
    are dropped. Parameters are a ‘‘regression.RegressionEngineParams‘‘ and/or
    its fields as keyword arguments, e.g. ‘‘min_factors=1, max_factors=3,
    train_fraction=0.7, bootstrap_samples=500‘‘.
    """
    if not isinstance(factors, pd.DataFrame):
        raise TypeError("factors must be a pandas DataFrame")
    data = factors.join(pd.Series(target, name="__target__"), how="inner").dropna()
    names = [str(c) for c in factors.columns]
    p = _make_params(_regression.RegressionEngineParams, params, overrides)
    y = data.pop("__target__").to_numpy(dtype=float)
    c = _regression.RegressionEngine(p).run(y, data.to_numpy(dtype=float), names)

    index = data.index
    sizes = pd.DataFrame([
        {
            "size": len(e.factors),
            "factors": [names[j] for j in e.factors],
            "alpha": e.alpha,
            "lambda": e.lambda_,
            "nonzero": e.nonzero,
            "validation_mse": e.validation_mse,
            "train_mse": e.train_mse,
            "train_r2": e.train_r2,
            "test_mse": e.test_mse,
            "test_r2": e.test_r2,
            "criterion": e.criterion,
            "score": e.score,
        }
        for e in c.best_by_size
    ]).set_index("size")
    boot = None
    if c.bootstrap is not None:
        b = c.bootstrap
        boot = pd.DataFrame(
            {"mean": b.mean, "std_error": b.std_error, "lower": b.lower, "upper": b.upper,
             "selection_frequency": b.selection_frequency},
            index=pd.Index(c.factor_names, name="factor"))
    return RegressionFrames(
        coefficients=pd.Series(c.coefficients, index=pd.Index(c.factor_names, name="factor"), name="
    ↪ coefficient"),
        intercept=c.intercept,
        alpha=c.alpha,
        lambda_=c.lambda_,
        predictions=pd.Series(c.predictions, index=index, name="predictions"),
        residuals=pd.Series(c.residuals, index=index, name="residuals"),
        test_start=index[c.train_rows] if c.test_rows > 0 else None,
        best_by_size=sizes,
        bootstrap=boot,
        computation=c,
    )
```

Code

module definition — `cpp/bindings/src/module.cpp:6--26`
`RiskBudgeting` over any measure — `cpp/bindings/src/bind_risk.cpp:21--57`
Python objectives for the optimizers — `cpp/bindings/src/bind_optimization.cpp:24--26`
budget schedule from rows and budgets — `cpp/bindings/src/bind_strategy.cpp:20--39`

regression bindings — cpp/bindings/src/bind_regression.cpp:14--177
re-export and __repr__ — cpp/bindings/nabia_python_bridge/_reexport.py:46--54
parameters plus overrides — cpp/bindings/nabia_python_bridge/frames.py:19--29
frames.backtest — cpp/bindings/nabia_python_bridge/frames.py:56--100

19.4 Development build and tests

- **Development build.** The preset python-bridge builds _core and _rb_backtest in RelWithDebInfo, with the interpreter of ../nabia_python/.venv. The modules are written next to the package's Python files, in the folder bindings/nabia_python_bridge, so a script run with PYTHONPATH=cpp/bindings imports them. Attaching CLion's debugger to that Python process stops in the C++ code.
- **Wheel.** pyproject.toml builds the package with scikit-build-core for each Python version and platform (Chapter 20).
- **Tests.** The bridge tests are Python tests, so they live in the Python project, in nabia_python/tests/bridge. The wheel scripts run them on every wheel they build. They check each module against closed forms and numpy, and the binding semantics:
  - Python objectives called from differential evolution's threads;
  - exceptions propagating out of objectives;
  - unknown parameter names rejected;
  - kernel composition;
  - frame alignment and budget schedules.

Chapter 20

Wheels, cibuildwheel and GitHub Actions

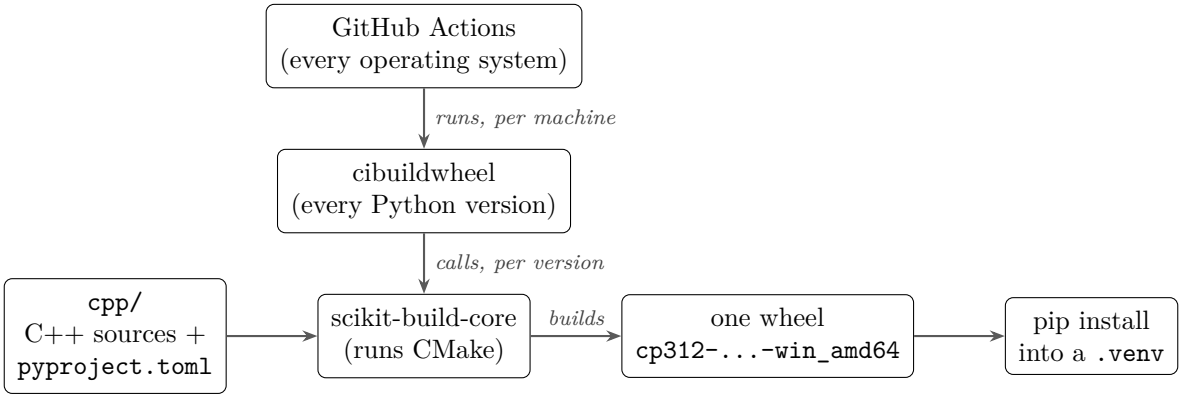
nabia's numerical library is C++; its users work in Python. The bridge between the two is a *wheel*: a pre-compiled Python package. This tutorial explains, from the ground up and with nabia's own files as the running example, what a wheel is and how pip chooses and installs one, how a wheel is built from a CMake project, how cibuildwheel builds one per Python version and platform, and how GitHub Actions runs all of it on Linux, macOS and Windows machines. The last part shows how to write new jobs, how to run, monitor and debug workflows, and how to publish. The files shown are *extracted from the repository when this document is built*, so they are always the ones in use.

20.0.1 The problem, and the map

A Python package written in Python runs anywhere Python runs. `nabia_python_bridge` is not: its heart is `_core`, a shared library compiled from C++ (a `.so` on Linux and macOS, a `.pyd` on Windows). Compiled code is tied to

- an **operating system** (Linux, macOS, Windows) and its C/C++ runtime;
- a **processor architecture** (x86_64 for Intel and AMD, arm64/aarch64 for Apple silicon and ARM servers);
- a **Python version**: an extension module is compiled against the C API of one CPython release (3.12, 3.13, ...), whose binary interface (ABI) changes between them.

So "making nabia installable" means producing one binary package per combination, testing each, and putting them where pip can find them. Here is how the pieces fit:



File	Role
`cpp/pyproject.toml`	what the package is and how to build it (scikit-build-core) and how to build all variants (cibuildwheel)
`cpp/bindings/CMakeLists.txt`	the extension modules and what goes into the wheel
`cpp/scripts/build_wheels.sh, .ps1`	all wheels of *this* machine (macOS/Linux, Windows)
`.github/workflows/wheels.yml`	all wheels of *all* machines, in GitHub's cloud
`nabia_python/scripts/setup_env.sh, .ps1`	installs a wheel into the Python project

20.1 Wheels

20.1.1 Anatomy of a wheel

A wheel is a **zip file** with the extension `.whl`. Look inside one:

```
$ python -m zipfile -l dist/nabia_python_bridge-0.1.0-cp314-cp314-macosx_13_0_arm64.whl
nabia_python_bridge/__init__.py
nabia_python_bridge/frames.py                       ... and the other .py files
nabia_python_bridge/_core.cpython-314-darwin.so     the whole C++ library, compiled
nabia_python_bridge/_rb_backtest.cpython-314-darwin.so
nabia_python_bridge-0.1.0.dist-info/METADATA        name, version, dependencies, README
nabia_python_bridge-0.1.0.dist-info/WHEEL           wheel format and tags
nabia_python_bridge-0.1.0.dist-info/RECORD          every file with its SHA-256 hash and size
```

- The files are laid out *exactly as they will be installed*: installing is unzipping into `site-packages`. Nothing is compiled at install time.
- `METADATA` lists the dependencies (`Requires-Dist: numpy>=1.24`), which pip installs too.
- `WHEEL` says `Root-Is-Purelib: false` (it contains compiled code) and repeats the tag.
- `RECORD` lets pip verify the files and uninstall them later.

The filename is the compatibility contract

`nabia_python_bridge-0.1.0-cp314-cp314-macosx_13_0_arm64.whl`

Part	Name	Meaning
`nabia_python_bridge`	distribution	the package name (`project.name`)
`0.1.0`	version	`project.version`
`cp314`	Python tag	which Python implementation and version can run it: CPython 3.14
`cp314`	ABI tag	which binary interface the compiled code uses. `abi3` would mean the stable ABI (one wheel for many versions); pybind11 modules use the full API, so it is the version itself
`macosx_13_0_arm64`	platform tag	operating system, its minimum version, and the architecture

Platform tags you will meet:

`manylinux_2_28_x86_64, _aarch64` Linux with glibc 2.28 or newer (Debian 10+, Ubuntu 20.04+, RHEL 8+). "Many" Linux distributions, because the wheel only uses libraries every one of them has, in versions at least that old. It is built inside an old Linux container on purpose: code compiled against an old glibc runs on newer ones, not the reverse.

`macosx_13_0_arm64, _x86_64` macOS 13 or newer, on Apple silicon or Intel. The number comes from `MACOSX_DEPLOYMENT_TARGET` at build time; it tells the compiler which macOS APIs it may use.

`win_amd64`, `win_arm64` 64-bit Windows on Intel/AMD, or on ARM.

`any` no compiled code. The Python project `nabia` would produce `py3-none-any` wheels: one file for every platform.

How pip chooses

pip computes the list of tags the running interpreter supports, most specific first, and picks the first wheel that matches. See the list with `python -m pip debug --verbose`. If no wheel matches, pip falls back to the **sdist** (source distribution, a `.tar.gz` of the sources) and compiles it, which needs CMake and a C++20 compiler on that machine. That is why binary packages publish one wheel per platform and Python version: users then never compile.

20.1.2 Building one wheel

The build interface: `[build-system]`

pip does not know CMake. Since PEP 517, a project declares a *build backend* that pip calls in a fresh, isolated environment:

Code 20.1: `cpp/pyproject.toml`: which backend builds the wheel

```
[build-system]
requires = ["scikit-build-core>=0.10", "pybind11>=2.13"]
build-backend = "scikit_build_core.build"
```

1. pip (or `python -m build`, or `uv build`) creates a temporary virtual environment and installs `requires` into it: scikit-build-core and pybind11.
2. It calls the backend's `build_wheel()`.
3. scikit-build-core configures and builds the CMake project, runs the CMake install step into a staging folder, and zips that folder into a wheel with its `dist-info`.

What the package is: `[project]`

Code 20.2: `cpp/pyproject.toml`: package metadata (becomes `METADATA`)

```
[project]
name = "nabia_python_bridge"
version = "0.1.0"
description = "Python bindings of the nabia C++ numerical library"
readme = "bindings/README.md"
requires-python = ">=3.10"
dependencies = ["numpy>=1.24"]
```

`dependencies` are installed by pip with the wheel. Optional groups are installed only on request, for example with `pip install "nabia_python_bridge[test]"`:

Code 20.3: `cpp/pyproject.toml`: optional dependency groups

```
[project.optional-dependencies]
pandas = ["pandas>=2.0"]
test = ["pytest>=7", "pandas>=2.0"]
examples = ["pandas>=2.0", "matplotlib>=3.7", "yfinance>=0.2"]
```

How CMake is driven: `[tool.scikit-build]`

Code 20.4: `cpp/pyproject.toml`: scikit-build-core settings

```
[tool.scikit-build]
sdist.exclude = ["build/", "cmake-build-*/", ".idea/", "doc/", "compile_commands.json", "bindings/
    ↪ nabia_python_bridge/_*.so"]
# This C++ project (cpp/) is the CMake source; only the Python bindings are built.
cmake.source-dir = "."
cmake.build-type = "Release"
# CMake installs the whole package (bindings/nabia_python_bridge/*.py, _core and _rb_backtest), so a
# development build's modules or __pycache__ left in the source tree never end up in the wheel.
wheel.packages = []
# Only the install rules of the bindings (COMPONENT python in bindings/CMakeLists.txt): dependencies
# fetched at configure time (Eigen on CI runners) have install rules of their own.
install.components = ["python"]
build-dir = "build/{wheel_tag}"
```

- `cmake.source-dir` is the folder with the top `CMakeLists.txt`: `cpp/` itself.
- `wheel.packages = []`: scikit-build-core copies no Python folder by itself; CMake installs everything (below). This keeps development leftovers out of the wheel.
- `build-dir` keeps a build folder per wheel tag, so rebuilding is incremental.
- `sdist.exclude` keeps build folders and documentation out of the source package.

Code 20.5: `cpp/pyproject.toml`: CMake options for wheel builds

```
[tool.scikit-build.cmake.define]
NABIA_BUILD_PYTHON_BRIDGE = "ON"
NABIA_BUILD_APPS = "OFF"
NABIA_BUILD_TESTS = "OFF"
NABIA_BUILD_EXAMPLES = "OFF"
# nabia, iir1 and rb_backtest_core are static libraries linked into the extension modules.
CMAKE_POSITION_INDEPENDENT_CODE = "ON"
```

These are ordinary CMake cache variables, as with `-D` on the command line: only the bindings are built (no tests, apps, examples or PDFs), and static libraries are compiled as position-independent code so they can be linked into a shared module on Linux.

What goes into the wheel: CMake install rules

Code 20.6: `cpp/bindings/CMakeLists.txt`: the modules and the install rules

```
# Python bindings: the C++ side of the nabia_python_bridge wheel (pybind11).
#
#   nabia_python_bridge._core          the whole library (stats, signal, forecast, optimization, risk,
#                                      strategy, regression)
#   nabia_python_bridge._rb_backtest   JSON-configured risk budgeting backtests (apps/rb_backtest),
#                                      used by nabia.portfolio in the Python project
#
# Wheel build (pyproject.toml of this C++ project, scikit-build-core): both modules and the package's
# .py files are installed into the wheel. Development build (CLion, preset python-bridge): the modules
# are written next to the .py files in bindings/nabia_python_bridge/, so the package can be imported
# with PYTHONPATH=cpp/bindings, e.g. to debug the C++ side from a Python script.

find_package(Python 3.10 REQUIRED COMPONENTS Interpreter Development.Module)

# pybind11: from the Python environment (pip install pybind11), else fetched.
execute_process(
    COMMAND "${Python_EXECUTABLE}" -m pybind11 --cmakedir
    OUTPUT_VARIABLE pybind11_cmake_dir
    OUTPUT_STRIP_TRAILING_WHITESPACE
    ERROR_QUIET)
```

```
find_package(pybind11 2.13 CONFIG QUIET HINTS "${pybind11_cmake_dir}")
if(NOT pybind11_FOUND)
    message(STATUS "pybind11 not found, fetching it")
    include(FetchContent)
    FetchContent_Declare(pybind11
        GIT_REPOSITORY https://github.com/pybind/pybind11.git
        GIT_TAG        v3.0.1
        GIT_SHALLOW    TRUE
        SYSTEM)
    FetchContent_MakeAvailable(pybind11)
endif()

pybind11_add_module(_core
    src/module.cpp
    src/bind_stats.cpp
    src/bind_signal.cpp
    src/bind_forecast.cpp
    src/bind_optimization.cpp
    src/bind_risk.cpp
    src/bind_strategy.cpp
    src/bind_regression.cpp)
target_link_libraries(_core PRIVATE nabia::nabia)
nabia_set_warnings(_core)

pybind11_add_module(_rb_backtest src/rb_backtest_module.cpp)
target_link_libraries(_rb_backtest PRIVATE rb_backtest_core)
nabia_set_warnings(_rb_backtest)

if(SKBUILD)
    # Component "python": pyproject.toml installs only this component into the wheel, so the install
    # rules of fetched dependencies (Eigen's headers, ...) never add files to it.
    install(TARGETS _core _rb_backtest LIBRARY DESTINATION nabia_python_bridge COMPONENT python)
    install(DIRECTORY nabia_python_bridge/ DESTINATION nabia_python_bridge COMPONENT python
        FILES_MATCHING PATTERN "*.py" PATTERN "__pycache__" EXCLUDE)
else()
    set_target_properties(_core _rb_backtest PROPERTIES
        LIBRARY_OUTPUT_DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR}/nabia_python_bridge)
endif()
```

`SKBUILD` is defined only when scikit-build-core runs CMake. Then the two modules and the `.py` files are `install`ed to `nabia_python_bridge/`, which becomes that folder inside the wheel. They belong to the install *component* `python`, and `install.components = ["python"]` in `pyproject.toml` installs only that component: dependencies that CMake downloads have install rules of their own (Eigen installs its 600 headers), which would otherwise land in every wheel built where Eigen is not installed, as on CI runners. In a normal CMake build (CLion, preset `python-bridge`) the modules are instead written next to the `.py` files, for debugging from the source tree.

Static linking. nabia, iir1 and the app's parser are static libraries, and Eigen is header-only, so all of their code is *inside* `_core`: the wheel needs only the system C++ runtime. A wheel that links a shared library (say `libfoo.so`) must carry a copy; cibuildwheel's *repair* step (§20.2.2) does that.

Try it

```
nabia_python/.venv/bin/python -m pip wheel ./cpp --no-deps -w dist     # one wheel, this Python
MACOSX_DEPLOYMENT_TARGET=13.0 nabia_python/.venv/bin/python -m pip wheel ./cpp --no-deps -w dist
uv build --wheel --python 3.12 --out-dir dist cpp                # with uv, another Python
uv build --sdist --out-dir dist cpp                              # the source package
```

20.1.3 Installing and using a wheel

```
python -m venv .venv                                    # an isolated environment
.venv/bin/pip install --no-index --find-links dist nabia_python_bridge
```

- `-find-links dist` adds a folder of wheels as a source; `-no-index` turns PyPI off, so the package must come from that folder.
- pip picks the matching file, reads its dependencies, installs them, unzips the wheel into `.venv/lib/python3.x/site-packages/`, and keeps its `dist-info` there.
- `import nabia_python_bridge` finds the folder; `__init__.py` imports `_core`, and the operating system's loader maps the `.so` into the process.

Same version, new build. pip skips a package whose version is already installed. After rebuilding the C++ code without changing the version, reinstall explicitly: `pip install --force ↪ -reinstall --no-deps ...`. The setup scripts do that every time:

Code 20.7: `nabia_python/scripts/setup_env.sh`: the wheel, then the project in editable mode

```
#!/usr/bin/env bash
# Sets up nabia_python/.venv for the Python project (the interpreter to select in PyCharm):
#   1. installs the nabia_python_bridge wheel built by the C++ project (../dist, see
#      cpp/scripts/build_wheels.sh) - reinstalled every time, so a rebuilt wheel is picked up;
#   2. installs this nabia package in editable mode, with the market data and test extras (add llm for
    ↪ --live).
#
#   nabia_python/scripts/setup_env.sh                # uses python3 to create .venv if it does not
    ↪ exist
#   PYTHON=python3.12 nabia_python/scripts/setup_env.sh
#   NABIA_WHEEL_DIR=/path/to/wheels nabia_python/scripts/setup_env.sh
set -euo pipefail

project="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
wheels="${NABIA_WHEEL_DIR:-$(dirname "$project")/dist}"
venv="$project/.venv"

[[ -x "$venv/bin/python" ]] || "${PYTHON:-python3}" -m venv "$venv"
py="$venv/bin/python"
"$py" -m pip install --quiet --upgrade pip

# pip picks the wheel matching this interpreter and platform among those in $wheels.
"$py" -m pip install --force-reinstall --no-deps --no-index --find-links "$wheels" nabia_python_bridge
    ↪ || {
    echo "no nabia_python_bridge wheel for $("$py" -V) in $wheels: run cpp/scripts/build_wheels.sh
    ↪ first" >&2
    exit 1; }
"$py" -m pip install --quiet --find-links "$wheels" -e "$project[marketdata,test]"

"$py" -c 'import nabia, nabia_python_bridge as nb; print("nabia", nabia.__file__); print("
    ↪ nabia_python_bridge", nb.__version__, nb.__file__)'
```

Editable installs. `pip install -e nabia_python` does not copy `nabia_python/nabia`: it installs a pointer to the source folder, so edits are live. That is how `nabia_python` is installed, while the C++ arrives as a finished wheel.

20.2 Many wheels

20.2.1 All Python versions on one machine

On macOS and Linux, `build_wheels.sh` loops over versions with uv, which downloads any missing Python:

Code 20.8: `cpp/scripts/build_wheels.sh`

```
#!/usr/bin/env bash
# Builds one nabia_python_bridge wheel per Python version for this machine, and tests each one
# in a clean virtual environment. uv (https://docs.astral.sh/uv/) downloads missing interpreters.
#
#   cpp/scripts/build_wheels.sh                          # Python 3.10 to 3.14
#   cpp/scripts/build_wheels.sh 3.12 3.13                # only these versions
#   NABIA_WHEEL_TEST=0 cpp/scripts/build_wheels.sh       # build without testing
#   NABIA_WHEEL_BUILD_TYPE=RelWithDebInfo cpp/scripts/build_wheels.sh 3.14   # with debug symbols
#
# Wheels go to dist/ at the repository root (NABIA_WHEEL_DIR overrides it), where the Python
# project installs them from (nabia_python/scripts/setup_env.sh). The tests are the bridge tests of
# nabia_python (nabia_python/tests/bridge). On macOS the wheels target
# MACOSX_DEPLOYMENT_TARGET, 13.0 unless already set. Wheels for Linux and Windows are built by
# cibuildwheel (configured in pyproject.toml, run by .github/workflows/wheels.yml).
set -euo pipefail

cpp="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
root="$(dirname "$cpp")"
tests="$root/nabia_python/tests/bridge"
build_type="${NABIA_WHEEL_BUILD_TYPE:-Release}"
out="${NABIA_WHEEL_DIR:-$root/dist}"
versions=("$@")
[[ ${#versions[@]} -gt 0 ]] || versions=(3.10 3.11 3.12 3.13 3.14)

command -v uv >/dev/null || { echo "uv is required: https://docs.astral.sh/uv/" >&2; exit 1; }
# Use an installed interpreter (Homebrew, python.org...) before a uv-managed one.
export UV_PYTHON_PREFERENCE="${UV_PYTHON_PREFERENCE:-system}"
[[ "$(uname)" == Darwin ]] && export MACOSX_DEPLOYMENT_TARGET="${MACOSX_DEPLOYMENT_TARGET:-13.0}"

mkdir -p "$out"
work="$(mktemp -d)"
trap 'rm -rf "$work"' EXIT

for v in "${versions[@]}"; do
    echo "=== Python $v ==="
    # Finds the interpreter, downloading it with uv when none is installed.
    py="$(uv python find "$v" 2>/dev/null || { uv python install "$v" >&2 && uv python find "$v"; })"
    # A pre-release (an old uv may only know a 3.x alpha) has no numpy wheels and the wrong ABI.
    "$py" -c 'import sys; sys.exit(sys.version_info.releaselevel != "final")' || {
        echo "$py is a pre-release Python; update uv (uv self update) or install Python $v" >&2; exit
    ↪ 1; }
    echo "$py ($("$py" -c 'import platform; print(platform.python_version())'))"
    # Isolated build: uv installs scikit-build-core and pybind11 for this interpreter.
    uv build --wheel --python "$py" --out-dir "$work/$v" -C cmake.build-type="$build_type" "$cpp"
    wheel="$(ls "$work/$v"/*.whl)"

    if [[ "${NABIA_WHEEL_TEST:-1}" != 0 ]]; then
        # Install in a fresh environment and run the tests from outside the source tree,
        # so they import the installed package.
        uv venv --quiet --python "$py" "$work/venv-$v"
        uv pip install --quiet --python "$work/venv-$v/bin/python" "$wheel[test]"
        (cd "$work" && "$work/venv-$v/bin/python" -m pytest -q -p no:cacheprovider "$tests")
    fi
    cp "$wheel" "$out/"
done

echo "=== wheels in $out ==="
ls -1 "$out"/*.whl
```

The pattern is the same for every wheel: *build in isolation, install into a clean environment, run the tests against the installed copy, keep it only if they pass.* Testing the installed wheel, from outside the source tree, is the point: it catches a file missing from the wheel, which tests run on the sources would never see.

20.2.2 cibuildwheel

cibuildwheel automates exactly that loop, for every platform, and adds what portable wheels need. For each *build identifier* — `cp312-manylinux_x86_64`, `cp313-macosx_arm64`, `cp314-win_amd64`,

. . . — it:

1. gets that Python (on Linux, inside a `manylinux` Docker container; on macOS and Windows, official installers it downloads);
2. builds the wheel through the backend, like `pip wheel`;
3. **repairs** it: on Linux `auditwheel` checks it only uses allowed system libraries and copies others in, then sets the `manylinux` tag; on macOS `delocate` does the same for `.dylib`s; on Windows `delvewheel` copies the C++ runtime `msvcp140.dll`, which Python does not ship, into `nabia_python_bridge.libs/` under a unique name, so the wheel works on a PC without the Visual C++ Redistributable;
4. installs it in a fresh environment with `test-extras` and runs `test-command`;
5. moves it to the output folder.

Configuration

Code 20.9: `cpp/pyproject.toml`: which wheels, and how to test them

```
# Wheels for every supported Python and platform (run by .github/workflows/wheels.yml):
#   pipx run cibuildwheel==4.3.4 --platform linux cpp     (from the repository root; Linux needs Docker)
# Each wheel is tested with the bridge tests of nabia_python ({project} is the repository root).
[tool.cibuildwheel]
build = "cp310-* cp311-* cp312-* cp313-* cp314-*"
skip = "*-musllinux_* *-win32 *-manylinux_i686"
test-extras = ["test"]
test-command = "pytest -q -p no:cacheprovider {project}/nabia_python/tests/bridge"
```

- `build` and `skip` are glob patterns over identifiers. Here: CPython 3.10–3.14; no musl Linux, no 32-bit.
- `{project}` is the folder cibuildwheel was started in (the repository root, so the Python project's tests are reachable); `{package}` would be `cpp/`.
- Any setting can be overridden by an environment variable: `CIBW_BUILD`, `CIBW_ARCHS`, `CIBW_TEST_SKIP`, . . . The workflow and the PowerShell script use that.

Code 20.10: Linux: the container image

```
[tool.cibuildwheel.linux]
# GCC 12+ for C++20; Eigen and iir1 are fetched at configure time.
manylinux-x86_64-image = "manylinux_2_28"
manylinux-aarch64-image = "manylinux_2_28"
```

Code 20.11: macOS: the oldest supported version

```
[tool.cibuildwheel.macos]
environment = { MACOSX_DEPLOYMENT_TARGET = "13.0" }
```

Running it yourself

```
pipx run cibuildwheel==4.3.4 --print-build-identifiers --platform linux cpp   # what would be built
pipx run cibuildwheel==4.3.4 --platform linux cpp       # Linux wheels, needs Docker running
pipx run cibuildwheel==4.3.4 --platform macos cpp       # on a Mac
CIBW_BUILD="cp312-*" pipx run cibuildwheel==4.3.4 --platform linux cpp   # one version only
```

On Windows, `build_wheels.ps1` wraps the same call: it checks Visual Studio, Git and Python, installs cibuildwheel in a private environment and runs it from the repository root.

Code 20.12: `cpp/scripts/build_wheels.ps1`

```
<#
.SYNOPSIS
    Builds nabia_python_bridge wheels for Windows (win_amd64 by default) with cibuildwheel, and tests
    each one with the bridge tests of nabia_python (nabia_python/tests/bridge).

.DESCRIPTION
    The Windows counterpart of build_wheels.sh. cibuildwheel downloads every CPython it needs, builds
    the wheel from this C++ project (cpp/pyproject.toml, scikit-build-core, MSVC), installs it in a
    clean environment and runs the tests. The wheels end up in dist\ at the repository root, where
    nabia_python\scripts\setup_env.ps1 installs them from.

    Needs, once:
      - Visual Studio 2022 Build Tools with the "Desktop development with C++" workload (C++20);
      - Git (CMake fetches Eigen and iir1);
      - Python 3.10 or newer with the py launcher (python.org installer), to run cibuildwheel.
    CMake is optional: scikit-build-core downloads it when it is not installed.

.PARAMETER Versions
    CPython versions to build, e.g. 3.12, 3.13. Default: 3.10 to 3.14.

.PARAMETER Arch
    AMD64 (Intel and AMD 64-bit, the default) or ARM64 (on an ARM machine).

.PARAMETER OutDir
    Where the wheels go. Default: dist\ at the repository root.

.PARAMETER NoTest
    Build without running the tests on each wheel.

.PARAMETER PythonExe
    Python used to run cibuildwheel. Default: the py launcher (py -3), else python on the PATH.

.EXAMPLE
    cpp\scripts\build_wheels.ps1
.EXAMPLE
    cpp\scripts\build_wheels.ps1 -Versions 3.12,3.13
.EXAMPLE
    cpp\scripts\build_wheels.ps1 -Versions 3.12 -NoTest
#>
[CmdletBinding()]
param(
    [string[]] $Versions = @("3.10", "3.11", "3.12", "3.13", "3.14"),
    [ValidateSet("AMD64", "ARM64")] [string] $Arch = "AMD64",
    [string] $OutDir = "",
    [switch] $NoTest,
    [string] $PythonExe = ""
)
$ErrorActionPreference = "Stop"

$cpp = Split-Path -Parent $PSScriptRoot
$root = Split-Path -Parent $cpp
if (-not $OutDir) { $OutDir = Join-Path $root "dist" }
$CibuildwheelVersion = "4.3.4"

# --- Prerequisites -----------------------------------------------------------
$hostArgs = @()
if ($PythonExe) {
    $hostExe = $PythonExe
} elseif (Get-Command py -ErrorAction SilentlyContinue) {
    $hostExe = "py"
    $hostArgs = @("-3")
} elseif (Get-Command python -ErrorAction SilentlyContinue) {
    $hostExe = "python"
} else {
    throw "Python was not found: install Python 3.10+ from python.org."
}
if (-not (Get-Command git -ErrorAction SilentlyContinue)) {
    throw "git was not found: install Git for Windows (CMake fetches Eigen and iir1 with it)."
```

```
}
$vswhere = Join-Path ${env:ProgramFiles(x86)} "Microsoft Visual Studio\Installer\vswhere.exe"
if (Test-Path $vswhere) {
    $component = if ($Arch -eq "ARM64") { "Microsoft.VisualStudio.Component.VC.Tools.ARM64" } else { "
    ↪ Microsoft.VisualStudio.Component.VC.Tools.x86.x64" }
    $vs = & $vswhere -latest -products * -requires $component -property installationPath
    if (-not $vs) {
        throw "Visual Studio has no C++ compiler: add the 'Desktop development with C++' workload."
    }
    Write-Host "MSVC: $vs"
} else {
    Write-Warning "vswhere.exe not found; the build fails if Visual Studio 2022 C++ tools are missing."
}

# --- cibuildwheel, in its own virtual environment ---------------------------------------------
$tools = Join-Path ([System.IO.Path]::GetTempPath()) "nabia-cibuildwheel"
if (-not (Test-Path (Join-Path $tools "Scripts\python.exe"))) {
    & $hostExe @hostArgs -m venv $tools
    if ($LASTEXITCODE -ne 0) { throw "could not create $tools" }
}
& (Join-Path $tools "Scripts\python.exe") -m pip install --quiet --upgrade pip "cibuildwheel==
    ↪ $CibuildwheelVersion"
if ($LASTEXITCODE -ne 0) { throw "could not install cibuildwheel" }

# --- Build ------------------------------------------------------------------------------------
# cp312-* etc.; the other settings (skips, tests, test extras) are in cpp\pyproject.toml.
$env:CIBW_BUILD = ($Versions | ForEach-Object { "cp" + ($_ -replace "\.", "") + "-*" }) -join " "
$env:CIBW_ARCHS = $Arch
if ($NoTest) { $env:CIBW_TEST_SKIP = "*" } else { Remove-Item Env:CIBW_TEST_SKIP -ErrorAction
    ↪ SilentlyContinue }

Write-Host "Building $($env:CIBW_BUILD) for $Arch into $OutDir"
# cibuildwheel runs from the repository root: {project} in the test command is the root, so the
# tests of python\tests\bridge are found; "cpp" is the package folder with pyproject.toml.
Push-Location $root
try {
    & (Join-Path $tools "Scripts\cibuildwheel.exe") --platform windows --output-dir $OutDir cpp
    if ($LASTEXITCODE -ne 0) { throw "cibuildwheel failed (exit code $LASTEXITCODE)" }
} finally {
    Pop-Location
}

Write-Host "=== wheels in $OutDir ==="
Get-ChildItem (Join-Path $OutDir "*win*.whl") | ForEach-Object { Write-Host $_.FullName }
```

20.3 GitHub Actions

20.3.1 Concepts

GitHub Actions runs commands on machines in GitHub's cloud when something happens in the repository. The vocabulary:

Workflow a YAML file in `.github/workflows/`. GitHub reads every file there. It must be at the repository root (not in `cpp/`).

Event what starts it: a push, a tag, a pull request, a schedule, or a manual start (`workflow_dispatch`). A manual start requires the workflow to exist on the default branch.

Run one execution of a workflow, with a number and a page (Actions tab).

Job a unit of work on *one fresh machine*. Jobs run in parallel unless one `needs` another. Nothing on disk survives from one job to the next.

Runner the machine: `ubuntu-24.04`, `ubuntu-24.04-arm`, `macos-14` (Apple silicon), `windows-latest`, ... A fresh virtual machine each time.

Step one command (`run:`) or one reusable **action** (`uses:`) inside a job, executed in order. If a step fails, the job stops and fails.

Action a packaged step published in a repository, used as `owner/repo@version`: `actions/checkout@v5` clones the repository, `pypa/cibuildwheel@v4.3.4` runs cibuildwheel, `actions/upload-artifact@v4` saves files.

Artifact files a job uploads to the run, to download later or in another job: the way to pass wheels from the build jobs to the collect job, and to you.

Matrix one job definition expanded into several jobs, one per combination of values.

YAML and expressions

YAML is indentation-based: two spaces per level, `key: value`, lists with `-` , and `{ a: 1, b: 2 }` as a one-line map. Text between `${{` and `}}` is an *expression* GitHub evaluates before running anything:

Expression	Value
`${{ matrix.os }}`	the current matrix value
`${{ github.ref }}`	what triggered the run: `refs/heads/main`, `refs/tags/v0.1.0`
`${{ github.token }}`	a temporary token for this run, with the job's `permissions`
`${{ secrets.NAME }}`	a secret set in Settings → Secrets and variables → Actions
`${{ needs.collect.result }}`	`success`, `failure`, `cancelled` or `skipped`
`startsWith(a, b)`, `!cancelled()`	functions and status checks, used in `if:`

Inside `run:` scripts, the same information is in environment variables (`$GITHUB_REF_NAME`, `$GITHUB_REPOSITORY`, ...).

20.3.2 nabia's workflow, piece by piece

Name and triggers

Code 20.13: `.github/workflows/wheels.yml`: when it runs

```
# Builds and tests nabia_python_bridge wheels for CPython 3.10-3.14 on Linux, macOS and Windows
# with cibuildwheel (settings in cpp/pyproject.toml), testing each with nabia_python/tests/bridge. Runs
    ↪ on
# demand, on version tags (v*), and on pull requests that touch the C++ project or the bridge tests.
#
#   wheels          one job per platform; artifact wheels-<os>-<arch>
#   collect         every wheel plus the sdist in one artifact, all-wheels
#   release         on a v* tag: a GitHub Release with all of them attached
#   windows-script  runs cpp/scripts/build_wheels.ps1 and nabia_python/scripts/setup_env.ps1 on Windows
name: wheels

on:
  workflow_dispatch:
  push:
    tags: ["v*"]
  pull_request:
    paths: ["cpp/**", "nabia_python/tests/bridge/**", "nabia_python/scripts/**", ".github/workflows/
    ↪ wheels.yml"]
```

- `workflow_dispatch`: the "Run workflow" button and `gh workflow run wheels.yml`.
- `push: tags: ["v*"]`: pushing a tag such as `v0.1.0`. Pushing commits to `main` does *not* start it (no `branches:` filter is given).
- `pull_request: paths:`: only pull requests that change those files.

The build matrix

Code 20.14: The `wheels` job: one machine per platform

```
wheels:
  name: ${{ matrix.os }} ${{ matrix.archs }}
  runs-on: ${{ matrix.os }}
  strategy:
    fail-fast: false
    matrix:
      include:
        - { os: ubuntu-24.04,     archs: x86_64 }
        - { os: ubuntu-24.04-arm, archs: aarch64 }
        - { os: macos-14,         archs: arm64 }
        - { os: macos-14,         archs: x86_64 }   # cross-compiled, tested under Rosetta
        - { os: windows-latest,   archs: AMD64 }    # Windows on Intel/AMD 64-bit (win_amd64)
  steps:
    - uses: actions/checkout@v5
    - uses: pypa/cibuildwheel@v4.3.4
      with:
        package-dir: cpp
        output-dir: wheelhouse
      env:
        CIBW_ARCHS: ${{ matrix.archs }}
    - uses: actions/upload-artifact@v4
      with:
        name: wheels-${{ matrix.os }}-${{ matrix.archs }}
        path: wheelhouse/*.whl
```

- `strategy.matrix.include` lists five combinations, so this definition becomes five jobs running in parallel, each with its own `${{ matrix.os }}` and `${{ matrix.archs }}`.
- `fail-fast: false`: a failure on one platform does not cancel the others.
- `runs-on` picks the machine from the matrix.
- Steps: clone the repository; run cibuildwheel on `cpp` (the action installs it and passes `with:` values as options and `env:` values as `CIBW_*` settings); upload the wheels as an artifact named after the platform. Each runner builds the five Python versions of its platform.
- `macos-14` is an Apple silicon machine; the `x86_64` entry cross-compiles Intel wheels there and tests them under Rosetta.

Collecting everything

Code 20.15: The `collect` job: every wheel and the sdist in one artifact

```
collect:
  name: all wheels and sdist
  needs: wheels
  if: ${{ !cancelled() }}   # keep the wheels of the platforms that succeeded
  runs-on: ubuntu-24.04
  steps:
    - uses: actions/checkout@v5
    - uses: actions/download-artifact@v4
      with:
        pattern: wheels-*
        merge-multiple: true
        path: dist
    - name: Source distribution
      run: pipx run build --sdist --outdir dist cpp
    - name: List
      run: ls -l dist
    - uses: actions/upload-artifact@v4
      with:
        name: all-wheels
        path: dist/*
```

- `needs:` `wheels` waits for all five build jobs.
- By default a job runs only if everything it needs succeeded. `if: ${{ !cancelled() }}` overrides that: it runs even if a platform failed, so the good wheels are not lost.
- `download-artifact` with `pattern` and `merge-multiple` puts the files of all `wheels-*` artifacts into one folder.
- `pipx run build -sdist` builds the source package, the fallback for platforms without a wheel.

Publishing a release

Code 20.16: The `release` job: a GitHub Release on version tags

```
release:
  name: GitHub Release
  needs: collect
  if: ${{ startsWith(github.ref, 'refs/tags/v') && needs.collect.result == 'success' }}
  runs-on: ubuntu-24.04
  permissions:
    contents: write
  steps:
    - uses: actions/download-artifact@v4
      with:
        name: all-wheels
        path: dist
    - name: Create the release with every wheel attached
      env:
        GH_TOKEN: ${{ github.token }}
      run: >
        gh release create "$GITHUB_REF_NAME" dist/*
        --repo "$GITHUB_REPOSITORY" --title "nabia $GITHUB_REF_NAME" --generate-notes
```

- The `if:` makes it run only for tags starting with `v`, and only if collecting succeeded. On manual runs it shows as *skipped*.
- `permissions:` `contents:` `write`: the run's token is read-only by default; creating a release needs write access, granted to this job only.
- `gh` (the GitHub CLI, installed on every runner) creates the release with all files attached and notes generated from the commits since the previous tag.

Testing the Windows scripts

Code 20.17: The `windows-script` job

```
windows-script:
  name: Windows build script
  runs-on: windows-latest
  steps:
    - uses: actions/checkout@v5
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - name: cpp/scripts/build_wheels.ps1 (CPython 3.12, tested)
      shell: pwsh
      run: cpp/scripts/build_wheels.ps1 -Versions 3.12 -PythonExe python
    - name: nabia_python/scripts/setup_env.ps1 and the Python project's tests
      shell: pwsh
      run: |
        nabia_python/scripts/setup_env.ps1 -PythonExe python
        nabia_python/.venv/Scripts/python.exe -m pytest -q nabia_python/tests
        if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
```

`actions/setup-python` installs a Python and puts it on the `PATH`; `shell: pwsh` runs the steps with PowerShell. The job builds one wheel with `build_wheels.ps1`, installs it with `setup_env.ps1`, and runs the Python project's tests: the scripts a Windows user would run, run on Windows.

20.4 Doing it yourself

20.4.1 Writing a job

A recipe:

1. **Pick the machine** (`runs-on`), or several with a matrix.
2. **Start with `actions/checkout`**: a runner starts empty.
3. **Install what is missing**: `actions/setup-python` for a given Python; compilers, CMake, Git and `gh` are preinstalled on GitHub's images.
4. **Run commands** with `run:`, one step per logical action, each with a `name:` so the log reads well. Use `working-directory:` instead of `cd`.
5. **Keep results** with `upload-artifact`; get them in another job with `needs:` and `download-artifact`.
6. **Grant only the permissions needed**, per job.

The examples below are not in the workflow; paste them under `jobs:` to use them.

Example: the C++ tests on Linux and macOS

```
cpp-tests:
  name: C++ tests ${{ matrix.os }}
  strategy:
    fail-fast: false
    matrix:
      os: [ubuntu-24.04, macos-14]
  runs-on: ${{ matrix.os }}
  defaults:
    run:
      working-directory: cpp
  steps:
    - uses: actions/checkout@v5
    - name: Configure
      run: cmake --preset release -DNABIA_BUILD_DOCS=OFF
    - name: Build
      run: cmake --build --preset release -j 4
    - name: Test
      run: ctest --preset release
```

A list under `matrix` (`os: [a, b]`) is the short form of `include`; with several lists, the matrix is their cross product.

Example: the Python project's tests against the Linux wheel

```
python-tests:
  needs: wheels
  runs-on: ubuntu-24.04
  steps:
    - uses: actions/checkout@v5
    - uses: actions/setup-python@v5
      with:
```

```
          python-version: "3.12"
      - uses: actions/download-artifact@v4
        with:
          name: wheels-ubuntu-24.04-x86_64
          path: dist
      - name: Install the wheel and nabia
        run: PYTHON=python nabia_python/scripts/setup_env.sh
      - name: Tests
        working-directory: nabia_python
        run: .venv/bin/python -m pytest -q
```

Example: publishing to PyPI without a password

PyPI's *trusted publishing* lets a workflow upload without storing a token: on pypi.org, under the project's Publishing settings, register this repository, the workflow file `wheels.yml` and an environment name (`pypi`). Then:

```
  pypi:
    needs: collect
    if: ${{ startsWith(github.ref, 'refs/tags/v') }}
    runs-on: ubuntu-24.04
    environment: pypi              # optional approval rules live in Settings > Environments
    permissions:
      id-token: write              # proves to PyPI which workflow is uploading
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: all-wheels
          path: dist
      - uses: pypa/gh-action-pypi-publish@release/v1
```

Every upload needs a new version number in `cpp/pyproject.toml`: PyPI never accepts the same version twice. Try it first on test.pypi.org (`with: repository-url: https://test.pypi.org/legacy/`).

Useful extras

```
concurrency:                       # top level: a new run of the same branch cancels the older one
  group: wheels-${{ github.ref }}
  cancel-in-progress: true

    timeout-minutes: 60            # in a job: stop a hung job early (default 360)

      - uses: actions/cache@v4     # in steps: reuse downloads between runs
        with:
          path: ~/.cache/pip
          key: pip-${{ runner.os }}-${{ hashFiles('cpp/pyproject.toml') }}
```

20.4.2 Running, watching and debugging

From the terminal

Command	Does
`gh workflow run wheels.yml`	starts a run (the file must be on `main`)
`gh run list -workflow wheels.yml`	recent runs with their ids and status
`gh run watch <id>`	live status until it finishes
`gh run view <id>`	jobs and their results
`gh run view <id> -log-failed`	the log lines of failed steps (after the run ends)
`gh run rerun <id> -failed`	reruns only the failed jobs
`gh run cancel <id>`	stops a run
`gh run download <id> -n all-wheels -D dist`	downloads an artifact
`gh release list`, `gh release view v0.1.0`	releases and their files

On the web

Repository → **Actions** tab → *wheels* in the sidebar → a run. The summary page shows the jobs as a graph (`wheels` → `collect` → `release`) and, at the bottom, the **Artifacts**. Click a job, then a step, to read its log; the gear menu downloads the full log. *Re-run jobs* and *Cancel workflow* are at the top right.

Reading a failure

cibuildwheel ends a failed build with a summary such as

```
Error: cibuildwheel: Command [... '-m', 'build', '.../cpp', '--wheel', ...] failed with code 1.
```

which only says *that* the build failed. The cause is above it: search the log for `error:` (compiler), `CMake Error`, `FAILED:` (the file that did not compile) or a Python traceback. Failures met while setting up this workflow, and their fixes:

Log	Cause	Fix
`HTTP 404: workflow wheels.yml not found on the default branch`	the file was not pushed to `main`	commit and push it
`AssertionError: uv not found` (macOS, Windows)	`build-frontend = "build[uv]"` needs uv on the runner	use the default frontend
`error: no member named 'jthread' in namespace 'std'`	the Xcode 15 libc++ of `macos-14` is older than the local one	a joining `std::thread` wrapper
`'from_chars' is unavailable: introduced in macOS 26.0`	a deployment target older than the library function	`strtod`/`snprintf`
`The run was canceled by @user`	cancelled by hand	nothing; rerun if needed
an `include/` folder (Eigen headers) in the CI wheels	install rules of a downloaded dependency	install only the component `python`

Reproduce locally first. A CI round trip takes minutes; most failures reproduce on your machine: `cpp/scripts/build_wheels.sh 3.12` for macOS, `pipx run cibuildwheel --platform linux cpp` with Docker for Linux, an older `MACOSX_DEPLOYMENT_TARGET` for macOS availability errors.

20.4.3 Releasing

Where	Who can install	How
Run artifacts	you and collaborators, logged in, for 90 days	`gh run download`
GitHub Release	anyone who can see the repository (it is private: collaborators); permanent	`git tag v0.1.0 && git push origin v0.1.0`, then download from the Releases page
PyPI	anyone: `pip install nabia-python-bridge`	trusted publishing job, new version each time

A release checklist:

1. Set the version in `cpp/pyproject.toml` (and in `nabia_python/` if it is published too); commit and push.

2. Start a manual run and check every job is green.

3. Tag and push the tag: `git tag v0.2.0 && git push origin v0.2.0`. The tag run builds again and creates the release.

20.4.4 Costs and limits

- **Minutes.** The repository is private, so runs use the account's included Actions minutes, and macOS and Windows minutes count more than Linux ones (see Settings → Billing). A full run builds 25 wheels on five machines plus the Windows script job; trigger it when needed rather than on every push.
- **Storage.** Artifacts count toward the account's storage until they expire (90 days by default); releases do not expire.
- **Versions.** Pin actions to versions (`@v5`, `@v4.3.4`) so a new release of an action cannot change the build unannounced; update them deliberately.

Part VI

The Python package

Chapter 21

Portfolio backtests: `nabia.backtest`

`nabia.portfolio` runs the C++ portfolio engine through `_rb_backtest`, a module of the wheel's package `nabia_python_bridge`. The module reuses the command-line application's JSON parser: Python and the command line accept the same parameters with the same error messages (Chapter 18). Data crosses the boundary in simple forms: dates as `int64` days since 1970-01-01, prices as a `float64` matrix, and parameters as JSON text.

Two functions are exposed:

- `nabia.backtest(prices, params, budget_schedule=None)` returns a result with pandas frames and metrics;
- `nabia.latest_weights(prices, params)` solves only the last rebalance: the target weights to hold now.

Code 21.1: pandas to C++
`nabia_python/nabia/portfolio.py:103-109`

```
def _inputs(prices: pd.DataFrame, params: Mapping[str, Any] | None) -> tuple[np.ndarray, list[str], np.
    ↪ ndarray, str]:
    if not isinstance(prices, pd.DataFrame):
        raise TypeError("prices must be a pandas DataFrame (dates x assets)")
    if prices.shape[1] == 0:
        raise ValueError("prices has no columns")
    values = np.ascontiguousarray(prices.to_numpy(dtype=np.float64, na_value=np.nan))
    return _to_days(prices.index), [str(c) for c in prices.columns], values, _config_json(params)
```

Code

Python `backtest` — `nabia_python/nabia/portfolio.py:112--182`
Python `latest_weights` — `nabia_python/nabia/portfolio.py:185--202`
dates to day numbers — `nabia_python/nabia/portfolio.py:72--80`

Chapter 22

Market data

22.1 Universe and sources

The default download universe has four asset classes:

- equity indices: S&P 500, Nasdaq 100, Russell 2000, Euro Stoxx 50, DAX, FTSE 100, Nikkei 225, Hang Seng;
- US Treasury futures: 2Y, 5Y, 10Y, Bond, Ultra Bond;
- commodity futures: WTI, Brent, natural gas, gold, silver, copper, corn, wheat, soybeans;
- crypto against USD: BTC, ETH, SOL, XRP, ADA, LTC.

The AI portfolio uses the nine assets of Table 22.1.

Table 22.1: A default nine-asset portfolio universe.

Symbol	Asset	Class
`^GSPC`	S&P 500	equity index
`^STOXX50E`	Euro Stoxx 50	equity index
`^N225`	Nikkei 225	equity index
`ZN=F`	US 10-Year T-Note future	bond future
`ZB=F`	US Treasury Bond future	bond future
`BZ=F`	Brent crude oil future	commodity
`GC=F`	Gold future	commodity
`HG=F`	Copper future	commodity
`BTC/USD`	Bitcoin	crypto

Sources.

- **Yahoo Finance** (`yfinance`) provides indices and futures, in batches. Futures are front-month continuous series, which jump at contract rolls. WTI settled negative on 2020-04-20, where simple returns are undefined, so the portfolio uses Brent.
- **Crypto exchanges** (`ccxt`, Coinbase by default) provide daily candles from public data, with no key. History is paged in windows of 300 candles. Windows before a coin's listing date are empty and are stepped over. Paging stops if the exchange ignores the start date.

Code 22.1: Crypto daily candles with pagination
`nabia_python/nabia/marketdata/sources.py:147-188`

```
def fetch(self, symbols: Sequence[str], start: dt.date, end: dt.date | None = None) -> pd.DataFrame:
    client = self._exchange()
    start_ms = int(dt.datetime.combine(start, dt.time(), dt.UTC).timestamp() * 1000)
    last_day = end or dt.datetime.now(dt.UTC).date()
    stop_ms = int(dt.datetime.combine(last_day + dt.timedelta(days=1), dt.time(), dt.UTC).timestamp() *
    ↪  1000)
    frames = []
    for symbol in symbols:
        if symbol not in client.markets:
            log.warning("%s is not listed on %s", symbol, getattr(client, "id", "the exchange"))
            continue
        candles: dict[int, list[float]] = {}
        since = start_ms
        while since < stop_ms:
            page = self._page(client, symbol, since)
            if not page:
                # Nothing in this window: before the listing date (or a gap). Step
                # forward a page; stop once past what has been collected so far.
                if candles:
                    break
                since += self.page_size * _DAY_MS
                continue
            for candle in page:
                if start_ms <= candle[0] < stop_ms:
                    candles[int(candle[0])] = candle
            next_since = int(page[-1][0]) + _DAY_MS
            if next_since <= since:  # no progress: the exchange ignores ‘since‘
                break
            since = next_since
        if not candles:
            continue
        rows = [candles[k] for k in sorted(candles)]
        frames.append(pd.DataFrame({
            "symbol": symbol,
            "date": [dt.datetime.fromtimestamp(r[0] / 1000, dt.UTC).date() for r in rows],
            "open": [r[1] for r in rows],
            "high": [r[2] for r in rows],
            "low": [r[3] for r in rows],
            "close": [r[4] for r in rows],
            "adj_close": [r[4] for r in rows],
            "volume": [r[5] for r in rows],
        }).astype({c: "float64" for c in PRICE_COLUMNS[2:]}))
    return pd.concat(frames, ignore_index=True)[PRICE_COLUMNS] if frames else empty_prices()
```

22.2 Storage and incremental updates

Prices are stored per (symbol, date): open, high, low, close, adjusted close and volume.

- **Upserts.** Writes use `INSERT ... ON CONFLICT DO UPDATE`, in the SQLite or PostgreSQL dialect, so re-downloading is idempotent and picks up revisions.
- **Incremental updates.** A stored symbol is re-fetched from five days before its last date, which corrects recent revisions.
- **Batching.** Symbols are grouped by source and start date.

Code 22.2: Portable upsert
`nabia_python/nabia/marketdata/store.py:66-81`

```
def upsert_rows(engine: Engine, table: Table, rows: list[dict], keys: Sequence[str], chunk: int = 500)
    ↪ -> None:
    """Insert rows, updating the non-key columns of rows whose keys already exist."""
    if not rows:
        return
    dialect = engine.dialect.name
    if dialect == "sqlite":
        from sqlalchemy.dialects.sqlite import insert
```

```
    elif dialect == "postgresql":
        from sqlalchemy.dialects.postgresql import insert
    else:  # pragma: no cover - other databases are not supported yet
        raise NotImplementedError(f"upsert is not implemented for {dialect}")
    with engine.begin() as connection:
        for i in range(0, len(rows), chunk):
            statement = insert(table).values(rows[i : i + chunk])
            updates = {c.name: statement.excluded[c.name] for c in table.columns if c.name not in keys}
            connection.execute(statement.on_conflict_do_update(index_elements=list(keys), set_=updates)
    ↪ )
```

Code 22.3: Incremental download

nabia_python/nabia/marketdata/download.py:19-72

```
def update_market_data(
    store: MarketDataStore,
    source: MarketDataSource | Mapping[str, MarketDataSource],
    assets: Sequence[Asset],
    start: dt.date,
    end: dt.date | None = None,
    full: bool = False,
    overlap_days: int = DEFAULT_OVERLAP_DAYS,
) -> dict[str, int]:
    """Downloads daily prices for ``assets`` and upserts them into ``store``.

    ``source`` is either one source for every asset, or a mapping from source
    name to source, in which case each asset is fetched from ``asset.source``
    (e.g. ``{"yahoo": YahooFinanceSource(), "crypto": CryptoSource()}``).

    Incremental by default: a symbol already in the store is fetched from
    ``overlap_days`` before its last stored date (so recent revisions are
    picked up); a new symbol from ``start``. ``full=True`` re-downloads
    everything from ``start``. Returns the number of rows written per symbol
    (0 for symbols the source returned no data for).
    """
    sources: dict[str, MarketDataSource]
    if isinstance(source, Mapping):
        sources = dict(source)
        unknown = {a.source for a in assets} - sources.keys()
        if unknown:
            raise ValueError(f"no source given for: {sorted(unknown)}")
        source_of = {a.symbol: a.source for a in assets}
    else:
        sources = {source.name: source}
        source_of = {a.symbol: source.name for a in assets}
    for name in sorted({source_of[a.symbol] for a in assets}):
        store.upsert_assets([a for a in assets if source_of[a.symbol] == name], source=name)
    last = {} if full else store.last_dates()

    # Group symbols by source and fetch start, so each group is one batched request.
    groups: dict[tuple[str, dt.date], list[str]] = {}
    for asset in assets:
        from_date = start
        if asset.symbol in last:
            from_date = max(start, last[asset.symbol] - dt.timedelta(days=overlap_days))
        groups.setdefault((source_of[asset.symbol], from_date), []).append(asset.symbol)

    written = {asset.symbol: 0 for asset in assets}
    for (name, from_date), symbols in sorted(groups.items()):
        log.info("fetching %d symbols from %s since %s", len(symbols), name, from_date)
        prices = sources[name].fetch(symbols, from_date, end)
        for symbol, rows in prices.groupby("symbol"):
            written[str(symbol)] = store.upsert_prices(rows)
            log.info("%s: %d prices stored (%s to %s)", symbol, written[str(symbol)], rows["date"].min
    ↪ (), rows["date"].max())
    for symbol, count in written.items():
        if count == 0:
            log.warning("no data for %s", symbol)
    return written
```

Code

universe — `nabia_python/nabia/marketdata/universe.py:24--70`
Yahoo source — `nabia_python/nabia/marketdata/sources.py:48--67`
loading a price matrix — `nabia_python/nabia/marketdata/store.py:136--168`

Chapter 23

Analytics on price frames

`nabia.analytics` computes descriptive statistics and trend signals on any pandas frame of prices (dates by assets).

All statistics use daily close prices up to `as_of`, over `stats_history_days` (400) calendar days. Volatilities are annualized with $\sqrt{252}$, standard deviations are sample standard deviations, and returns are simple returns.

Aligned returns keep the dates where all assets have a price, then take returns. They are used for correlations only. Single-asset statistics use each asset's own history.

Correlation is the Pearson correlation matrix ρ of the last w aligned returns (60 by default, at least 3). The *average correlation* is

$$\bar{\rho} = \frac{1}{n(n-1)} \sum_{i \neq j} \rho_{ij}.$$

Rolling volatility is $\sigma^{(w)} = \mathrm{sd}(r_{T-w+1..T})\sqrt{252}$ for $w \in \{20, 60, 120\}$. The *volatility percentile* is computed from the series v_s of rolling 20-day standard deviations over the last 252 dates:

$$\pi = \frac{1}{|v|} \sum_{s} \mathbf{1}[v_s \leq v_T].$$

So $\pi = 1$ means the highest of the year.

Past returns $P_T / P_{T-n} - 1$ for $n \in \{21, 63, 126, 252\}$ observations (1, 3, 6 and 12 months).

Moving-average distance $P_T / \overline{P}_n - 1$, where $\overline{P}_n$ is the mean of the last $n \in \{50, 200\}$ prices.

Volatility-scaled 12-1 momentum

$$m = \frac{P_{T-21}/P_{T-252} - 1}{\mathrm{sd}(r_{T-251..T})\sqrt{252}}.$$

It is the 11-month return ending one month ago, which skips the short-term reversal, measured in units of the asset's own annual volatility.

Drawdowns over the last 253 prices: $D_t = 1 - P_t / \max_{s \leq t} P_s$. The tools report the current value and the maximum.

Market regime combines five numbers:

- $\bar{\rho}$ over 60 and over 252 aligned returns;
- the median volatility percentile;

- the share of assets above their 200-day average;
- the share of assets with a positive 12-month return.

Code 23.1: Trend signals
nabia_python/nabia/analytics.py:51-75

```
def trend_signals(prices: pd.DataFrame) -> pd.DataFrame:
    """Per symbol: past returns (1m, 3m, 6m, 12m), distance from the 50- and
    200-day moving averages, a volatility-scaled 12-1 momentum score, and the
    current and 1-year maximum drawdowns."""
    out = {}
    for symbol in prices.columns:
        p = prices[symbol].dropna()
        last = p.iloc[-1]
        row: dict[str, float] = {}
        for label, n in (("ret_1m", 21), ("ret_3m", 63), ("ret_6m", 126), ("ret_12m", 252)):
            row[label] = float(last / p.iloc[-n - 1] - 1) if len(p) > n else float("nan")
        for n in (50, 200):
            row[f"vs_ma{n}"] = float(last / p.tail(n).mean() - 1) if len(p) >= n else float("nan")
        r = p.pct_change().dropna()
        vol = r.tail(252).std() * np.sqrt(PERIODS_PER_YEAR)
        if len(p) > 252 and vol > 0:
            row["momentum_12_1"] = float((p.iloc[-22] / p.iloc[-253] - 1) / vol)  # skip the last month
        else:
            row["momentum_12_1"] = float("nan")
        year = p.tail(253)
        drawdown = 1 - year / year.cummax()
        row["drawdown"] = float(drawdown.iloc[-1])
        row["max_drawdown_1y"] = float(drawdown.max())
        out[symbol] = row
    return pd.DataFrame(out).T
```

Code 23.2: Volatilities and their percentile
nabia_python/nabia/analytics.py:36-48

```
def rolling_volatility(prices: pd.DataFrame, windows: tuple[int, ...] = (20, 60, 120)) -> pd.DataFrame:
    """Per symbol: annualized volatility over each window (its own returns, not
    aligned with the other assets), plus where the 20-day volatility sits in
    its last year of history (percentile, 0..1)."""
    out = {}
    for symbol in prices.columns:
        r = prices[symbol].dropna().pct_change().dropna()
        row = {f"vol_{w}d": float(r.tail(w).std() * np.sqrt(PERIODS_PER_YEAR)) if len(r) >= w else
    ↪ float("nan")
               for w in windows}
        rolling = r.rolling(20).std().dropna().tail(PERIODS_PER_YEAR)
        row["vol_20d_percentile_1y"] = float((rolling <= rolling.iloc[-1]).mean()) if len(rolling) else
    ↪  float("nan")
        out[symbol] = row
    return pd.DataFrame(out).T
```

Code 23.3: Market regime
nabia_python/nabia/analytics.py:78-91

```
def market_regime(prices: pd.DataFrame) -> dict[str, float]:
    """Cross-asset summary: average correlation now (60d) vs over the last year,
    median volatility percentile, and the share of assets in an uptrend."""
    corr_now = average_correlation(correlation_matrix(prices, 60))
    corr_year = average_correlation(correlation_matrix(prices, 252)) if len(aligned_returns(prices)) >=
    ↪  60 else float("nan")
    vols = rolling_volatility(prices)
    trends = trend_signals(prices)
    return {
        "avg_correlation_60d": corr_now,
        "avg_correlation_1y": corr_year,
        "median_vol_percentile_1y": float(vols["vol_20d_percentile_1y"].median()),
        "share_above_ma200": float((trends["vs_ma200"] > 0).mean()),
        "share_positive_12m": float((trends["ret_12m"] > 0).mean()),
    }
```

Code

correlation matrix — `nabia_python/nabia/analytics.py:21--26`
average correlation — `nabia_python/nabia/analytics.py:29--33`

Part VII

Sentiment-modulated risk budgeting

Chapter 24

Overview of the pipeline

The pipeline implements the specification `doc/sentiment_risk_budgeting_spec.md`: each portfolio asset (equity, commodity, government bond, credit) is mapped to a set of sentiment indicators, inherited down an indicator tree; an elastic net per asset aggregates them into one sentiment score; the score is normalized to $[0, 1]$ and modulates the asset's risk budget, or, in long-short mode, decides whether the asset is held long or short; a risk-budgeting optimizer turns budgets into weights; the result is backtested against plain equal risk contribution (ERC). News sentiment comes from a language model. Every numerical step that the C++ library implements runs in the `nabia_python_bridge` wheel:

Step	Method	nabia
News aggregation, volatility, smoothing	exponentially weighted averages	`signal.exponential_moving_average`
Elastic net per asset	coordinate descent, (α, λ) grid, purged walk-forward CV, block bootstrap	`RegressionEngine` (`frames.regress`)
Normalization (option)	$\Phi(S/\hat{\sigma}_S)$	`stats.normal_cdf`
Weights	risk budgeting (Spinu), risk contributions	`risk.RiskBudgeting`, `risk.Volatility`
Long-only backtest, ERC benchmark	walk-forward engine with a budget schedule	`frames.backtest`
Metrics	Sharpe, drawdown, turnover, ...	`strategy.compute_performance`

pandas and numpy do the bookkeeping (alignment, rolling z-scores, ranks, correlations). The long-short backtest runs on a small simulator that repeats the engine's accounting with a direction per asset (§28.4).

Data flow. For one configuration, `run` (`nabia_python/nabia/sentiment_rb/pipeline.py:16--54`) executes:

1. synthetic point-in-time market data, indicators and news (Chapter 29);
2. the news pipeline: dedup, triage, scoring, daily digest, node features (Chapter 26);
3. point-in-time transforms and the candidate features of each asset (Chapter 25);
4. the walk-forward elastic net, the score S and its normalization s (Chapter 27);

5. budgets or directions, the weights and the backtests (Chapter 28);

6. tables, CSV outputs and figures.

Code 24.1: The pipeline for one configuration
`nabia_python/nabia/sentiment_rb/pipeline.py:16-54`

```
def run(cfg, out, live=False, show=False, verbose=True):
    """Data -> news -> features -> elastic net -> s -> budgets -> backtests -> report; returns headline
    ↪  numbers."""
    out.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(cfg["seed"])
    assets = list(cfg["assets"])
    b0 = np.array([cfg["assets"][a][2] for a in assets])

    f, returns, raw = synthetic_market(cfg, rng)
    dates = returns.index
    truth = news_truth(cfg, f, returns, rng)
    news = synthetic_news(cfg, truth, rng)
    live_from = dates[-cfg["llm"]["live_days"]] if live else dates[-1] + pd.Timedelta(days=1)
    scores, digests = run_news(cfg, news, truth, LLM(cfg, out, live, rng), live_from)
    print(f"news: {len(news)} articles, {len(scores)} node scores, {len(digests)} digests "
          f"({'live from ' + str(live_from.date()) if live else 'mocked LLM'})")

    feats = build_features(cfg, raw, node_features(cfg, scores, dates), digests, dates)
    X = {a: asset_matrix(cfg, feats, a) for a in assets}
    y = {a: vol_scaled_target(cfg, returns[a]) for a in assets}
    S, s, betas, ics = walk_forward_scores(cfg, X, y, dates)
    print(f"elastic net: {len(betas)} fits, signal from {S.index[0].date()}")

    bench, overlay, sched, dirs = backtests(cfg, returns, s, b0)
    rc_err = check_risk_contributions(cfg, returns, sched, dirs)

    t = report.tables(cfg, X, y, S, s, betas, bench, overlay, sched, b0)
    if verbose:
        report.show_tables(t, sched.index[-1])
    print("\nkey drivers:", digests[max(digests)].key_drivers, f"| max |RC - b| (nabia RiskBudgeting) =
    ↪  {rc_err:.1e}",
          f"| max budget change {(sched.pct_change().abs().max().max()):.1%} "
          f"| mode {cfg['budgets']['mode']}, short positions {(dirs < 0).to_numpy().mean():.0%}")
    report.save(out, t, sched, s, news, bench, overlay, ics)
    report.plots(cfg, out, bench, overlay, sched, s, b0, assets, ics, t["selection_frequency"], S.index
    ↪ [0], show)
    if verbose:
        print(f"\noutputs (tables, figures, budgets, news, LLM audit log) in {out}")
    return {"sentiment_ic": cfg["synthetic"]["sentiment_ic"], "mode": cfg["budgets"]["mode"], "
    ↪ erc_sharpe": bench.metrics["sharpe_ratio"],
            "overlay_sharpe": overlay.metrics["sharpe_ratio"],
            "excess_return": overlay.metrics["annualized_return"] - bench.metrics["annualized_return"],
            "ic_S": t["ic"].loc["pooled", "IC(S, y)"], "ic_llm_level": t["llm_ic"]["level"]}
```

Code layout (`nabia_python/nabia/sentiment_rb`):

Folder	Files	Area
—	`config.py`, `pipeline.py`, `__main__.py`	settings, `run()`, command line and sweep
`data/`	`synthetic.py`	synthetic market data, indicators and news
`features/`	`tree.py`, `transforms.py`	indicator tree; point-in-time transforms, candidate features, target
`llm/`	`schemas.py`, `client.py`, `news.py`	output schemas; mock or Anthropic client; dedup, scoring, node features
`model/`	`elastic_net.py`	elastic net per asset, walk-forward S and s
`portfolio/`	`budgets.py`, `backtest.py`	budgets and directions; backtests, simulator, checks
`report/`	`tables.py`, `figures.py`	report tables and CSV; figures
`prompts/`	`triage_v1.md`, ...	versioned prompt files (system and user templates)

All settings are in one dictionary, `CONFIG` (`nabia_python/nabia/sentiment_rb/config.py:14--76`); `load_config` (`nabia_python/nabia/sentiment_rb/config.py:85--87`) deep-merges a JSON file of overrides, so taxonomy, features, horizons, bounds, models and the normalization method change without code edits.

Chapter 25

Indicator tree, point-in-time features and target

25.1 The indicator tree

The taxonomy is a tree of nodes: `L0.GLOBAL` (global risk appetite) at the root; the asset classes `L1.EQ`, `L1.CMD`, `L1.FI`; buckets `L2.*` (US and European equities, energy, precious metals, 10-year Treasuries, high yield); and one node per underlying `L3.*`. Each node has a parent; the ancestors of n are $\mathrm{anc}(n) = \{n, \mathrm{parent}(n), \ldots, \texttt{L0.GLOBAL}\}$ (`nabia_python/nabia/sentiment_rb/features/tree.py:8--13`). Asset i sits at an L3 node n_i and belongs to an asset class $c(i) \in \{\mathrm{EQ}, \mathrm{CMD}, \mathrm{GOVT}, \mathrm{CREDIT}\}$.

Every feature k is attached to a node $\nu(k)$ and carries a *sign per asset class* $\sigma_{k,c} \in \{-1, 0, 1\}$ that orients it: after orientation, higher means more bullish for the asset's price. Risk-off is bullish for government bonds, so a risk-appetite indicator has $\sigma_{k,\mathrm{GOVT}} = -1$. The candidate features of asset i are those of all its ancestors that are not switched off for its class:

$$\mathcal{F}_i = \{\, k : \ \nu(k) \in \mathrm{anc}(n_i),\ \sigma_{k,c(i)} \neq 0 \,\}.$$

Each feature enters once (the matrix is keyed by feature), which the tests check (`test_assets_inherit_every_ancestor_feature_once`).

Code 25.1: Candidate features of an asset, oriented
`nabia_python/nabia/sentiment_rb/features/transforms.py:38-43`

```
def asset_matrix(cfg, feats, asset):
    """Candidate features of an asset: those of every ancestor node, oriented (+ = bullish), each
     ↪ exactly once."""
    node, cls = cfg["assets"][asset][:2]
    path = set(ancestors(cfg, node))
    cols = {fid: sign_for(sign, cls) * z for fid, (z, n, sign) in feats.items() if n in path and
     ↪ sign_for(sign, cls)}
    return pd.DataFrame(cols)
```

25.2 Point-in-time alignment and transforms

A value observed for date t (*as of* t) becomes usable ℓ_k days later (*available at* $t + \ell_k$): a COT report three days, financial-conditions indices five, LLM features one day after the news is received. The usable series is therefore $\tilde{x}_{k,t} = x_{k,t-\ell_k}$, and nothing earlier than its availability is ever used. The series is then made stationary and standardized:

$$u_{k,t} = \begin{cases} \tilde{x}_{k,t} - \tilde{x}_{k,t-5} & \text{(transform “diff”)} \\ \tilde{x}_{k,t} & \text{(transform “z”)} \end{cases}, \qquad z_{k,t} = \mathrm{clip}_{[-3,3]}\left(\frac{u_{k,t} - m_t^{(W)}(u_k)}{s_t^{(W)}(u_k)} \right),$$

with the rolling mean $m^{(W)}$ and standard deviation $s^{(W)}$ over $W = 756$ trading days (about three years, at least 252 observations). Rolling, not full-sample, statistics keep every value point in time. The oriented feature of asset i is $\hat{z}^{(i)}_{k,t} = \sigma_{k,c(i)}\, z_{k,t}$.

Code 25.2: Point in time, stationary, rolling z-score
`nabia_python/nabia/sentiment_rb/features/transforms.py:14-19`

```
def transform(cfg, series, lag, kind):
    """Point in time (value usable ‘lag‘ days after as_of), stationary, rolling z-score clipped at +-
    ↪ z_clip."""
    x = series.shift(lag)
    x = x.diff(5) if kind == "diff" else x
    r = x.rolling(cfg["z_window"], min_periods=cfg["z_min"])
    return ((x - r.mean()) / r.std()).clip(-cfg["z_clip"], cfg["z_clip"])
```

25.3 The target

The elastic net predicts the forward vol-scaled return over $h = 21$ days,

$$y_{i,t} = \frac{P_{i,t+h}/P_{i,t} - 1}{\hat{\sigma}_{i,t}\sqrt{h}}, \qquad \hat{\sigma}^2_{i,t} = \mathrm{EWMA}_{H=20}\left(r^2_{i,\cdot}\right)_t,$$

where the exponentially weighted mean of squared daily returns (half-life 20 days, smoothing factor $\alpha = 1 - 2^{-1/20}$) comes from `signal.exponential_moving_average`. Scaling by volatility makes targets comparable across assets and calm or turbulent periods. $y_{i,t}$ is only known at $t + h$, which the training windows respect (§27.2).

Code 25.3: Vol-scaled forward return
`nabia_python/nabia/sentiment_rb/features/transforms.py:46-51`

```
def vol_scaled_target(cfg, r):
    """y_t = r_{t->t+h} / sigma_t, sigma from a nabia EWMA of squared returns, scaled to the horizon."
    ↪ "
    h = cfg["horizon"]
    sigma = np.sqrt(signal.exponential_moving_average(r.values**2, 1 - 0.5 ** (1 / cfg["vol_halflife"])
    ↪ ) * h)
    price = (1 + r).cumprod()
    return (price.shift(-h) / price - 1) / pd.Series(sigma, r.index)
```

Code

all features: indicators, node LLM features, digest outputs — `nabia_python/nabia/sentiment_rb/features/transforms.py:22--35`
orientation of the latent factors (synthetic data) — `nabia_python/nabia/sentiment_rb/features/tree.py:20--21`

Chapter 26

The LLM news pipeline

26.1 Prompts, schemas and the client

Three prompts, stored as versioned files in `prompts/` (system and user templates, separated by a marker):

A, triage (a small model): is the article relevant, which taxonomy nodes does it concern, with which relevance and channel (*direct*: the node's asset is the subject; *spillover*: affected through a sector, competitor or macro link). Market recaps are marked irrelevant.

B, scoring (a stronger model): for each node, the sentiment in $[-1, 1]$ for the asset's price over the next 1–20 trading days, a confidence, magnitude, horizon, surprise versus expectations, fact type and a short rationale. Government bonds: hawkish policy and upside inflation are negative.

C, digest (one call per day, on the day's deduplicated headlines): risk appetite, growth tone, inflation tone, geopolitical stress, credit tone, central-bank tone per bank, commodity supply tone per complex.

Each response is validated against a pydantic schema (`nabia_python/nabia/sentiment_rb/llm/schemas.py:16--22`, `nabia_python/nabia/sentiment_rb/llm/schemas.py:25--33`, `nabia_python/nabia/sentiment_rb/llm/schemas.py:40--49`): numbers in range, categories from closed lists. A malformed response is retried up to twice, then logged and skipped. Every call is appended to `llm_audit.jsonl` with the prompt version (file name and hash of its text), the model, a hash of the request, the article or date it concerns, the attempt, and the raw response, so each score can be reproduced or re-scored. Templates are filled by `fill` (`nabia_python/nabia/sentiment_rb/llm/client.py:80--86`).

Code 26.1: One LLM call: fill, call, validate, retry, audit
`nabia_python/nabia/sentiment_rb/llm/client.py:55-69`

```
def __call__(self, kind, fill, ctx, live=False):
    system, template, version = self.prompts[kind]
    user, schema, model = fill(template), self.SCHEMAS[kind], self.cfg["llm"][self.MODEL[kind]]
    for attempt in range(1 + self.cfg["llm"]["max_retries"]):
        raw = self._anthropic(model, system, user, schema) if live else mock_response(kind, ctx, self.
    ↪ rng, attempt)
        try:
            result = schema.model_validate(raw)
        except ValidationError:
            result = None
        self.audit.write(json.dumps({"kind": kind, "model": model if live else "mock", "prompt":
    ↪ version,
                                     "request_sha": hashlib.sha256(user.encode()).hexdigest()[:16], "
    ↪ ref": ctx.get("ref"),
                                     "attempt": attempt, "valid": result is not None, "response": raw},
    ↪  default=str) + "\n")
        if result is not None:
```

```
            return result
    return None  # logged and skipped
```

With `-live`, the call goes to the Anthropic API at temperature 0, with the output forced through a tool whose input schema is the pydantic schema, and the system prompt cached; the SDK retries rate limits and server errors with exponential backoff. Offline, `mock_response` (`nabia_python/nabia/sentiment_rb/llm/client.py:16--38`) answers from the synthetic truth of the article, with noise, and returns 1% of first attempts malformed so the retry path is exercised.

Code 26.2: The Anthropic call: structured output through a tool
`nabia_python/nabia/sentiment_rb/llm/client.py:71-77`

```
def _anthropic(self, model, system, user, schema):
    r = self.client.messages.create(
        model=model, max_tokens=2048, temperature=0, messages=[{"role": "user", "content": user}],
        system=[{"type": "text", "text": system, "cache_control": {"type": "ephemeral"}}],
        tools=[{"name": "emit", "description": "Return the result.", "input_schema": schema.
    ↪ model_json_schema()}],
        tool_choice={"type": "tool", "name": "emit"})
    return next(b.input for b in r.content if b.type == "tool_use")
```

Look-ahead. A model trained after an article's date may "know" what followed. The pipeline offers anonymization (entity names replaced by *Entity k* in every prompt, the mapping to node IDs kept outside the model), and limits live scoring to recent news (`llm.live_days`); out-of-sample performance after the model's training cutoff is the real validation.

26.2 Deduplication and novelty

A story is often published several times. Each headline is embedded as a hashed bag of words: tokens are hashed into $d = 256$ buckets, counted and normalized, $v_a \in \mathbb{R}^{256}$, $\|v_a\| = 1$ (`nabia_python/nabia/sentiment_rb/llm/news.py:16--20`). Article a joins the cluster of an earlier article b received within 48 hours if $v_a^\top v_b > 0.9$. Its *novelty* is

$$\nu_a = \frac{1}{1 + n_a},$$

with n_a the number of earlier articles of its cluster in the window: 1 for a new story, $\frac{1}{2}$ for the first repeat, and so on.

Code 26.3: Clusters and novelty
`nabia_python/nabia/sentiment_rb/llm/news.py:33-37`

```
recent = [j for j in recent if a.received_ts - news.received_ts[j] < pd.Timedelta(hours=L["dedup_hours"
    ↪ ])]
same = [j for j in recent if vecs[i] @ vecs[j] > L["dedup_cosine"]]
cluster[i] = cluster[same[0]] if same else i
novelty = 1 / (1 + sum(cluster[j] == cluster[i] for j in recent))
recent.append(i)
```

26.3 Weights and decay-weighted aggregation

Each (article, node) score s_a gets the weight

$$w_a = \text{relevance}_a \times \text{confidence}_a \times \nu_a \times \omega_{\text{source}(a)},$$

with source weights ω (wire 1, newspaper 0.8, blog 0.4). The node's sentiment on day t is the decay-weighted mean of the scores received up to t:

$$\bar{s}_{n,t} = \frac{\sum_{a:\, t_a \le t} w_a \, s_a \, e^{-\lambda(t - t_a)}}{\sum_{a:\, t_a \le t} w_a \, e^{-\lambda(t - t_a)}}, \qquad \lambda = \frac{\ln 2}{H}, \quad H = 3 \text{ days}. \tag{26.1}$$

Code 26.4: Weight of each (article, node) score
`nabia_python/nabia/sentiment_rb/llm/news.py:46-50`

```
rel = {m.node_id: m.relevance for m in nodes}
for s in sc.scores if sc else []:
    if s.node_id in rel:
        w = rel[s.node_id] * s.confidence * novelty * L["source_weight"].get(a.source, 0.5)
        records.append((a.day, s.node_id, s.sentiment, w))
```

Proposition 26.3.1. *Let* $X_t = \sum_{a:\,t_a=t} w_a s_a$ *and* $W_t = \sum_{a:\,t_a=t} w_a$ *be the day sums, and* $E(x)$ *the exponential moving average with smoothing factor* $\alpha = 1 - e^{-\lambda}$ *started from zero,* $E_t = \alpha x_t + (1-\alpha)E_{t-1}$, $E_{-1} = 0$. *Then* $\bar{s}_{n,t} = E_t(X)/E_t(W)$ *whenever* $E_t(W) > 0$.

Proof. Unrolling the recursion, $E_t(x) = \alpha \sum_{j \le t} (1-\alpha)^{t-j} x_j = \alpha \sum_{j \le t} e^{-\lambda(t-j)} x_j$. Substituting the day sums, $E_t(X) = \alpha \sum_a w_a s_a e^{-\lambda(t-t_a)}$ and $E_t(W) = \alpha \sum_a w_a e^{-\lambda(t-t_a)}$; the factor α cancels in the ratio, which is (26.1). □

The code therefore computes the aggregation with nabia's EWMA, prepending a zero so that it starts from $E_{-1} = 0$:

Code 26.5: Zero-started EWMA (nabia)
`nabia_python/nabia/sentiment_rb/llm/news.py:61-64`

```
def ema0(x, half_life):
    """Zero-started EWMA (nabia): y_t ? sum_j (1-a)^(t-j) x_j, so ratios of two ema0 are decay-weighted
    ↪ means."""
    a = 1 - 0.5 ** (1 / half_life)
    return signal.exponential_moving_average(np.r_[0.0, np.asarray(x, float)], a)[1:]
```

26.4 The five features per node

For every node n and day t:

Feature	Definition	Orientation
`llm_level`	$\bar{s}_{n,t}$, eq. (26.1)	+
`llm_volume`	$\log(1 + N_{n,t})$, N the day's relevant articles	− (attention spikes on bad news)
`llm_dispersion`	standard deviation of the day's scores	− (disagreement)
`llm_mom5`	$\bar{s}_{n,t} - \bar{s}_{n,t-5}$	+
`llm_surprise`	$\bar{s}_{n,t} - \frac{1}{60}\sum_{j<60} \bar{s}_{n,t-j}$	+

LLM sentiment is defined with respect to the price of the node's asset, so its sign is +1 for the node's subtree, except at `L0.GLOBAL`, where it is risk appetite and takes the risk-on sign of each class. Like every indicator they are lagged (one day), z-scored and oriented (Chapter 25).

Code 26.6: The five LLM features of every node
`nabia_python/nabia/sentiment_rb/llm/news.py:67-78`

```
def node_features(cfg, scores, dates):
    """The 5 LLM features of every node: decay-weighted level, volume, dispersion, 5-day momentum,
    ↪ surprise."""
    out = {}
    for node in cfg["taxonomy"]:
        g = scores[scores.node == node].assign(ws=lambda d: d.weight * d.sentiment).groupby("day")
        day = g.agg(ws=("ws", "sum"), w=("weight", "sum"), n=("sentiment", "size"), sd=("sentiment", "
    ↪ std")).reindex(dates)
        num, den = ema0(day.ws.fillna(0), cfg["llm"]["half_life"]), ema0(day.w.fillna(0), cfg["llm"]["
    ↪ half_life"])
        level = pd.Series(np.divide(num, den, out=np.zeros_like(num), where=den > 1e-12), dates)
        out |= {f"llm_level@{node}": level, f"llm_volume@{node}": np.log1p(day.n.fillna(0)),
                f"llm_dispersion@{node}": day.sd.fillna(0), f"llm_mom5@{node}": level - level.shift(5),
                f"llm_surprise@{node}": level - level.rolling(60, min_periods=20).mean()}
    return pd.DataFrame(out)
```

26.5 The daily digest

Prompt C's outputs become features of their node, with a sign per class, e.g. inflation tone negative for equities, bonds and credit but positive for commodities, a hawkish Fed negative for everything, energy supply tightening positive for the energy bucket (`digest_features` in `nabia_python/nabia/sentiment_rb/config.py:14--76`). Null outputs (no relevant news for a central bank) count as neutral.

Code

the news pipeline: dedup, triage, scoring, digests — `nabia_python/nabia/sentiment_rb/llm/news.py:23--58`

Chapter 27

Elastic net per asset and the sentiment score

27.1 The model

For each asset i and refit date T, the elastic net (Chapter 16) is fitted on the oriented candidates $\hat{z}^{(i)}_{k,t}$, $k \in \mathcal{F}_i$:

$$\min_{\beta_0,\beta} \frac{1}{2n} \sum_{t \in \mathcal{T}_T} \left(y_{i,t} - \beta_0 - \textstyle\sum_k \beta_k \hat{z}^{(i)}_{k,t}\right)^2 + \lambda\Big(\alpha\|\beta\|_1 + \tfrac{1-\alpha}{2}\|\beta\|_2^2\Big),$$

with standardized features, $\alpha \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ and a λ path, chosen by cross-validation. The fit is nabia's regression engine (`frames.regress`) with all candidates in one subset, so the engine tunes only (α, λ).

27.2 Purged walk-forward training

Targets overlap: $y_{i,t}$ spans $t+1, \ldots, t+h$. Two precautions keep the future out:

1. **The training rows end h days before the refit date**: $\mathcal{T}_T = \{t \le T - h\}$, the rows whose target is known at T (an assertion checks it). The window is expanding (or rolling, by configuration), and starts once enough history exists for the z-scores; a missing z-score counts as neutral (0).
2. **Walk-forward cross-validation with a gap**: the engine's folds train on the past and validate on the next block, with h rows removed between them (`validation_gap`), the analogue of `TimeSeriesSplit(gap=h)`.

Refits happen at the end of each quarter (or month); the assets of one refit date are fitted in parallel threads, since the engine releases Python's GIL.

Code 27.1: Training rows of one refit: purged and point in time
`nabia_python/nabia/sentiment_rb/model/elastic_net.py:36-44`

```
def fit_at(p, a):
    X, lo = X_by_asset[a], 0 if M["window"] == "expanding" else max(0, p - h - M["rolling_days"])
    Xw, yw = X.iloc[lo:p - h + 1], y_by_asset[a].iloc[lo:p - h + 1]
    rows = (Xw.notna().mean(axis=1) >= 0.5) & yw.notna()  # past the z-score warm-up; missing z =
    ↪ neutral 0
    assert yw[rows].index.max() + pd.tseries.offsets.BDay(h) <= dates[p], "purge: target overlaps the
    ↪ refit date"
    if rows.sum() < M["min_train"]:
        return pd.Series(0.0, X.columns), np.nan
    beta = fit_asset(cfg, Xw[rows].fillna(0.0), yw[rows])
    return beta, np.corrcoef(Xw[rows].fillna(0.0).values @ beta.values, yw[rows])[0, 1] if beta.any()
    ↪ else np.nan
```

27.3 Positivity and stability selection

The features are pre-oriented, so a negative coefficient would be a nonsensical sign flip; the specification asks for $\beta \geq 0$. nabia's elastic net has no sign constraint, so the pipeline applies it by elimination: fit, drop the features with $\beta_k < 0$, refit, until all coefficients are non-negative. At the same time it applies *stability selection*: each fit includes a moving-block bootstrap of $B = 50$ samples (blocks of h rows, so overlapping targets stay together), and the selection frequency

$$\pi_k = \frac{1}{B} \sum_{b=1}^{B} \mathbf{1}[\beta_k^{(b)} \neq 0]$$

must exceed 0.6; features below are dropped and the model refitted. If nothing survives, $\beta = 0$: the asset's score is neutral.

Code 27.2: Elastic net with positivity and stability selection
`nabia_python/nabia/sentiment_rb/model/elastic_net.py:14-26`

```
def fit_asset(cfg, X, y):
    M, cols = cfg["model"], list(X.columns)
    while cols:
        res = frames.regress(y, X[cols], min_factors=len(cols), max_factors=len(cols), train_fraction
    ↪ =1.0,
                             alphas=M["l1_ratios"], validation=regression.ValidationScheme.WalkForward,
                             validation_folds=M["cv_folds"], validation_gap=cfg["horizon"],
                             bootstrap_samples=M["stability"]["samples"], bootstrap_block_size=cfg["
    ↪ horizon"])
        beta, freq = res.coefficients, res.bootstrap.selection_frequency.reindex(cols).fillna(0)
        drop = set(beta[beta < 0].index if M["positive"] else []) | set(freq[freq <= M["stability"]["
    ↪ threshold"]].index)
        if not drop:
            return beta.reindex(X.columns).fillna(0.0)
        cols = [c for c in cols if c not in drop]
    return pd.Series(0.0, X.columns)  # nothing survives: S = 0, neutral
```

27.4 The score and its normalization

With the coefficients of the latest refit, the sentiment score is the prediction minus the intercept,

$$S_{i,t} = \sum_{k \in \mathcal{F}_i} \beta_{ik}\, \hat{z}_{k,t}^{(i)},$$

so all-zero coefficients give $S = 0$. It is normalized to $[0, 1]$ without full-sample statistics, which would look ahead and be dominated by outliers:

Rolling percentile (default): the share of the last $L = 756$ scores (computed with the current coefficients) below today's, counting ties as half,

$$s_{i,t}^{\text{ts}} = \frac{1}{L} \sum_{u=t-L}^{t-1} \Big(\mathbf{1}[S_{i,u} < S_{i,t}] + \tfrac{1}{2}\, \mathbf{1}[S_{i,u} = S_{i,t}] \Big).$$

The mid-rank makes a constant score (an asset without features) exactly neutral, $s = \frac{1}{2}$.

Normal CDF : $s_{i,t}^{\text{ts}} = \Phi(S_{i,t}/\hat{\sigma}_{S,t})$, with the rolling standard deviation $\hat{\sigma}_S$ (`stats.normal_cdf`).

An optional cross-sectional rank across assets, $s_{i,t}^{\text{xs}} = (\text{rank}_i\, S_{\cdot,t} - 1)/(N - 1)$, is blended, $s = \alpha_{\text{ts}} s^{\text{ts}} + (1 - \alpha_{\text{ts}}) s^{\text{xs}}$ ($\alpha_{\text{ts}} = 1$ by default), and the result is smoothed by an EWMA with a 5-day half-life. Every value is checked to lie in $[0, 1]$.

Code 27.3: Score and rolling percentile with the current coefficients
`nabia_python/nabia/sentiment_rb/model/elastic_net.py:54-59`

```
for t in range(p + 1, stop + 1):
    hist = cur[max(0, t - N["lookback"]):t]
    S.iat[t, S.columns.get_loc(a)] = cur[t]
    s.iat[t, s.columns.get_loc(a)] = ((hist < cur[t]).mean() + 0.5 * (hist == cur[t]).mean()  # mid-
    ↪ rank: S = 0 always -> 0.5
                                      if N["method"] == "percentile" else
                                      float(stats.normal_cdf(np.array([cur[t] / (hist.std() or 1)]))
    ↪ [0]))
```

Code 27.4: Cross-sectional blend and EWMA smoothing
`nabia_python/nabia/sentiment_rb/model/elastic_net.py:60-64`

```
live = S.dropna(how="all").index
xs = (S.loc[live].rank(axis=1) - 1) / (S.shape[1] - 1)  # cross-sectional rank in [0,1]
blend = N["alpha_ts"] * s.loc[live] + (1 - N["alpha_ts"]) * xs
smooth = blend.apply(lambda c: signal.exponential_moving_average(c.values, 1 - 0.5 ** (1 / N["
    ↪ ewma_halflife"])))
assert ((smooth >= 0) & (smooth <= 1)).all().all()
```

27.5 In-sample and out-of-sample diagnostics

For every refit, the pipeline records the information coefficient (IC, the correlation of the score with the realized target) on the training rows and on the next, unseen block. In-sample IC above out-of-sample IC measures overfitting; the report also gives the IC of S and of s per asset and pooled over the whole out-of-sample period, and the pooled IC of each LLM feature.

Code

walk-forward refits, scores, normalization, IC records — `nabia_python/nabia/sentiment_rb/model/elastic_net.py:29--65`
report tables: comparison, IC, snapshot, selection frequencies — `nabia_python/nabia/sentiment_rb/report/tables.py:8--27`

Chapter 28

From sentiment to positions

28.1 Risk budgeting

Given a covariance Σ and budgets $b_i > 0$, $\sum_i b_i = 1$, the risk-budgeting portfolio (Chapter 8) solves

$$\min_{w>0} \; R(w) - \sum_i b_i \ln w_i, \qquad R(w) = \sqrt{w^\top \Sigma w},$$

and is rescaled to sum to one. At the optimum the relative risk contributions equal the budgets:

$$\mathrm{RC}_i(w) = \frac{w_i (\Sigma w)_i}{w^\top \Sigma w} = b_i.$$

(The specification's $\frac{1}{2} w^\top \Sigma w$ gives the same weights up to scale.) The pipeline checks this at every rebalance with nabia's `RiskBudgeting` to 10^{-4}; observed errors are below 10^{-8} (`nabia_python/nabia/sentiment_rb/portfolio/backtest.py:96--103`).

28.2 Long-only: modulated budgets

With base budgets b^0 (ERC weights per asset) and normalized sentiment s_i,

$$b_i = \frac{b_i^0 \left(m_{\min} + (m_{\max} - m_{\min})\, s_i\right)}{\sum_j b_j^0 \left(m_{\min} + (m_{\max} - m_{\min})\, s_j\right)}, \qquad m_{\min} = 0.5,\; m_{\max} = 1.5,$$

so neutral sentiment ($s = \frac{1}{2}$) keeps b^0 and the multiplier ranges over $[0.5, 1.5]$. Budgets stay strictly positive (the log barrier needs $b > 0$). The change per rebalance is capped at $\pm 25\%$ relative: the budgets are clipped to $[b^{\mathrm{prev}}(1-c),\, b^{\mathrm{prev}}(1+c)]$ and renormalized, repeatedly.

Code 28.1: Budget modulation with a change cap
`nabia_python/nabia/sentiment_rb/portfolio/budgets.py:9-17`

```
def modulate(cfg, s, b0, prev):
    B = cfg["budgets"]
    b = b0 * (B["m_min"] + (B["m_max"] - B["m_min"]) * s)
    b /= b.sum()
    for _ in range(50 if prev is not None else 0):  # cap the relative change per rebalance
        b = np.clip(b, prev * (1 - B["max_change"]), prev * (1 + B["max_change"]))
        b /= b.sum()
    assert (b > 0).all() and abs(b.sum() - 1) < 1e-12
    return b
```

28.3 Long-short: directions

In long-short mode, sentiment decides the side of each position and the budgets stay b^0 (optionally scaled by conviction, s_i for a long and $1 - s_i$ for a short). The direction $d_i \in \{+1, -1\}$ follows a

neutral band with hysteresis, so an asset hovering around one threshold does not flip back and forth:

$$d_{i,t} = \begin{cases} -1 & s_{i,t} < s_{\text{short}}\ (0.35) \\ +1 & s_{i,t} > s_{\text{long}}\ (0.5) \\ d_{i,t^-} & \text{otherwise.} \end{cases}$$

Code 28.2: Directions with hysteresis
`nabia_python/nabia/sentiment_rb/portfolio/budgets.py:20-24`

```
def directions(cfg, s, prev):
    """+1 long, -1 short. Short below short_below, long again above long_above; in between the side is
    ↪ kept."""
    B, d = cfg["budgets"], prev.copy()
    d[s < B["short_below"]], d[s > B["long_above"]] = -1.0, 1.0
    return d
```

A short position earns $-r_i$. With $D = \operatorname{diag}(d)$, the directed returns are Dr, whose covariance is $\Sigma' = D\Sigma D$: a shorted asset's correlations with the others change sign, so it is sized as the hedge or diversifier it becomes. The optimizer runs on Σ' and returns positive exposures w'; the weights are $w = Dw'$.

Proposition 28.3.1. *Let $w' > 0$ be the risk-budgeting solution on $\Sigma' = D\Sigma D$ with budgets b, and $w = Dw'$. Then the risk contributions of w on the original covariance equal the budgets: $\mathrm{RC}_i(w;\Sigma) = b_i$. Every position, long or short, carries its risk budget.*

Proof. Since $d_i^2 = 1$, $w^\top \Sigma w = w'^\top D\Sigma D w' = w'^\top \Sigma' w'$ and $w_i(\Sigma w)_i = d_i w_i'(\Sigma D w')_i = w_i'\,(D\Sigma D w')_i = w_i'(\Sigma' w')_i$. Hence $\mathrm{RC}_i(w;\Sigma) = \mathrm{RC}_i(w';\Sigma') = b_i$ by optimality of w'. □

Code 28.3: Risk budgeting on $D\Sigma D$ (nabia)
`nabia_python/nabia/sentiment_rb/portfolio/backtest.py:32-38`

```
def solve(cfg, cov, b, d):
    """nabia risk budgeting on D Sigma D: weights D w' and relative risk contributions (= b)."""
    vol = risk.Volatility(d[:, None] * cov * d[None, :])
    P = cfg["portfolio"]
    x = np.asarray(risk.RiskBudgeting(risk.RiskBudgetingParams(budgets=b, tolerance=P.get("
    ↪ solver_tolerance", 1e-8),
                                                               max_iter=P.get("solver_max_iter",
    ↪ 5000)), vol).optimize())
    return d * x, np.asarray(vol.compute_risk_contributions(x)) / vol.compute_risk(x)
```

28.4 The long-short simulator

nabia's backtest engine has no notion of direction, and directions change between rebalances while the covariance window spans several of them, so flipping columns of its input would mix signs within one window. The simulator `simulate` repeats the engine's accounting (Chapter 9) and applies the current directions to the whole trailing window at each rebalance:

- **Covariance**, the engine's estimator: weights $\omega_j \propto 2^{-a_j/H}$ for a row of age a_j (half-life $H = 60$; equal weights for the sample estimator), normalized, and

$$\hat{\Sigma} = \frac{\sum_j \omega_j (r_j - \bar{r})(r_j - \bar{r})^\top}{1 - \sum_j \omega_j^2}, \qquad \bar{r} = \sum_j \omega_j r_j,$$

  over $W = 252$ rows (`nabia_python/nabia/sentiment_rb/portfolio/backtest.py:22--29`).

- **Rebalances** at rows $W - 1 + kf$, $f = 21$; the budgets and directions in force are the last schedule row at or before the date.

- **Drift**: between rebalances each holding grows with its return, $w \leftarrow w \circ (1 + r_t)/(1 + w^\top r_t)$; short weights drift the same way.
- **Costs**: at a rebalance the turnover is $\tau = \|w^* - w\|_1$ and the cost $c\,\tau$ ($c = 5$ bp); the net return is $(1 + w^\top r_t)(1 - c\tau) - 1$.
- **Metrics** from the first trade on, with nabia's `compute_performance`.

Run with all-long directions, the simulator reproduces nabia's engine: equity to 6.5×10^{-10}, weights to 6×10^{-9}, identical Sharpe ratio. The long-only path asserts this on every run, and a test checks it on independent data (`nabia_python/tests/test_sentiment_rb.py:59--68`); `nabia_python/tests/test_sentiment_rb.py:48--56` checks Proposition 28.3.1.

Code 28.4: The engine's accounting with directions
`nabia_python/nabia/sentiment_rb/portfolio/backtest.py:41-68`

```
def simulate(cfg, returns, sched, dirs):
    """The engine's walk-forward accounting with directions: budgets and directions in force at each
     ↪ rebalance
    (the last schedule row at or before it; b0 / long before the first), drifting weights, costs on
     ↪ turnover."""
    P, R, dates = cfg["portfolio"], returns.values, returns.index
    W, (T, N) = P["covariance_window"], R.shape
    b0 = np.array([cfg["assets"][a][2] for a in returns.columns])
    rows = set(range(W - 1, T, P["rebalance_frequency"]))
    w, equity, rec, targets, rcs = np.zeros(N), 1.0, [], {}, {}
    for t in range(T):
        gross = w @ R[t]
        w = w * (1 + R[t]) / (1 + gross)
        cost = traded = 0.0
        if t in rows:
            k = sched.index.searchsorted(dates[t], side="right") - 1
            b, d = (sched.iloc[k].values, dirs.iloc[k].values) if k >= 0 else (b0, np.ones(N))
            target, rcs[dates[t]] = solve(cfg, window_cov(cfg, R[t - W + 1:t + 1]), b, d)
            traded = np.abs(target - w).sum()
            cost, w, targets[dates[t]] = P["transaction_cost"] * traded, target, target
        net = (1 + gross) * (1 - cost) - 1
        equity *= 1 + net
        rec.append((gross, cost, net, traded, equity))
    perf = pd.DataFrame(rec, dates, ["gross_returns", "costs", "net_returns", "turnover", "equity"])
    perf["drawdown"] = strategy.drawdown_series(perf.equity.values, 1.0)
    metrics = frames.performance(perf.net_returns.iloc[W - 1:], P.get("periods_per_year", 252))
    metrics |= {"rebalances": len(targets), "average_turnover": perf.turnover.sum() / len(targets),
                "annualized_turnover": perf.turnover.sum() * 252 / (T - W + 1), "total_costs": perf.
     ↪ costs.sum()}
    return SimpleNamespace(performance=perf, metrics=metrics, target_weights=pd.DataFrame(targets,
     ↪ returns.columns).T,
                           risk_contributions=pd.DataFrame(rcs, returns.columns).T)
```

Code 28.5: Benchmark and overlay: same covariance, rebalancing and costs
`nabia_python/nabia/sentiment_rb/portfolio/backtest.py:71-93`

```
def backtests(cfg, returns, s, b0):
    """Plain ERC with b0 (nabia engine) and the sentiment overlay in the configured mode, with the same
     ↪  covariance,
    rebalancing and costs. Returns (bench, overlay, budget schedule, direction schedule)."""
    assets, P, B = list(returns.columns), cfg["portfolio"], cfg["budgets"]
    returns = returns.loc[returns.index[returns.index.get_loc(s.index[0]) - P["covariance_window"]]:]
    bench = frames.backtest(returns, budgets=dict(zip(assets, b0)), **P)
    sched, dirs, prev_b, prev_d = {}, {}, b0, np.ones(len(assets))
    for day in bench.target_weights.index:
        if day not in s.index:
            continue
        sd = s.loc[day].values
        prev_d = dirs[day] = directions(cfg, sd, prev_d) if B["mode"] == "long_short" else np.ones(len(
     ↪ assets))
        conviction = np.where(prev_d > 0, sd, 1 - sd)  # long_only: = s
        prev_b = sched[day] = (b0 if B["mode"] == "long_short" and not B["modulate_long_short"]
                               else modulate(cfg, conviction, b0, prev_b))
```

```
    sched, dirs = pd.DataFrame(sched, index=assets).T, pd.DataFrame(dirs, index=assets).T
    if B["mode"] == "long_only":
        overlay = frames.backtest(returns, budgets=dict(zip(assets, b0)), budget_schedule=sched, **P)
        replay = simulate(cfg, returns, sched, dirs)  # the long-short simulator must reproduce the
     ↪ engine
        assert np.allclose(replay.performance.equity, overlay.performance.equity, rtol=0, atol=1e-6)
    else:
        overlay = simulate(cfg, returns, sched, dirs)
    return bench, overlay, sched, dirs
```

Chapter 29

Synthetic data, experiments and results

29.1 Synthetic market data

So that the pipeline runs offline and its behaviour can be tested against a known truth, the data are synthetic (`nabia_python/nabia/sentiment_rb/data/synthetic.py:20--39`). Every taxonomy node n has a latent factor, a standardized AR(1), $f_{n,t} = \phi f_{n,t-1} + \sqrt{1-\phi^2}\,\varepsilon_{n,t}$, $\phi = 0.97$ (half-life about three weeks). Asset i's daily return is

$$r_{i,t} = \frac{\sigma_i}{\sqrt{252}}\Big(\kappa\,\frac{\sum_{n\in\mathrm{anc}(n_i)}\sigma^{\mathrm{f}}_{n,c(i)}f_{n,t}}{\sqrt{|\,\mathrm{anc}(n_i)|}} + \ell_{c(i)}\,g_t + \sqrt{1-\ell^2_{c(i)}}\,\eta_{i,t}\Big) + \mathrm{SR}_{c(i)}\frac{\sigma_i}{252},$$

with annual volatilities σ_i, signal strength $\kappa = 0.15$, a common shock g with class loadings ℓ (government bonds negative), and a risk premium of Sharpe ratio SR per class; the factor sign σ^{f} is the risk-on sign at `L0.GLOBAL` and $+1$ below. Each indicator is a noisy reading of its node's factor with a slowly drifting level (or, for transform "diff", a stock whose changes carry the signal); some indicators are pure noise, for the model to discard.

29.2 Synthetic news with a known skill

The default news generator is an *oracle* with a set skill (`nabia_python/nabia/sentiment_rb/data/synthetic.py:42--56`). For node n, let $x_{n,t}$ be the mean over its assets of the oriented forward target, shifted by the LLM lag so that it starts when the news becomes usable, and $z_{n,t}$ its standardized value. The node's true tone is

$$T_{n,t} = \rho\, z_{n,t} + \sqrt{1-\rho^2}\,\epsilon_{n,t}, \qquad \mathrm{corr}(T_{n,t}, z_{n,t}) = \rho,$$

where ρ is the *news IC* (`synthetic.sentiment_ic`, 0.2 by default): 0 is pure noise, 1 perfect foresight. About four articles a day are drawn (more often for global and asset-class nodes), each with tone $\tanh(T_{n,t} + 0.3\,\epsilon_a)$; 12% are market recaps (irrelevant) and 10% re-publications (duplicates); the mock LLM adds scoring noise; the digest derives from the same truths. The rest of the pipeline is unchanged: it must find the edge through point-in-time features, the purged elastic net, normalization and the budget rules. Setting `"sentiment": "latent"` ties the news to the latent factors instead.

Code 29.1: Oracle news truth
`nabia_python/nabia/sentiment_rb/data/synthetic.py:42-56`

```
def news_truth(cfg, f, returns, rng):
    """Daily 'true tone' per node, ~N(0,1). Oracle: corr(tone, node's forward vol-scaled return) =
    ↪ sentiment_ic,
```

```
    over the h days that start when the news becomes usable (after the LLM publication lag)."""
    syn = cfg["synthetic"]
    if syn["sentiment"] == "latent":
        return f
    ic, lag = syn["sentiment_ic"], cfg["llm"]["lag"]
    fwd = pd.DataFrame({a: vol_scaled_target(cfg, returns[a]).shift(-lag) for a in returns})
    truth = {}
    for node in cfg["taxonomy"]:
        members = [a for a, v in cfg["assets"].items() if node in ancestors(cfg, v[0])]
        x = sum(node_sign(cfg, node, cfg["assets"][a][1]) * fwd[a] for a in members) / len(members)
        z = ((x - x.mean()) / x.std()).fillna(0.0)  # the last h days have no future: pure noise
        truth[node] = ic * z + np.sqrt(1 - ic**2) * rng.standard_normal(len(z))
    return pd.DataFrame(truth)
```

The information coefficient. The IC of a signal is its correlation with the subsequent realized return, here $\mathrm{corr}(S_{i,t}, y_{i,t})$, per asset or pooled over assets. Real signals have ICs of a few percent; by the fundamental law of active management the information ratio grows roughly like IC $\times \sqrt{\text{breadth}}$, so small ICs applied often can matter.

29.3 Results

The figures and tables below come from runs with the default configuration (seed 7); `doc/build.sh -figures` regenerates them. Signals start in 2018 after the warm-up of the z-scores and the first training window.

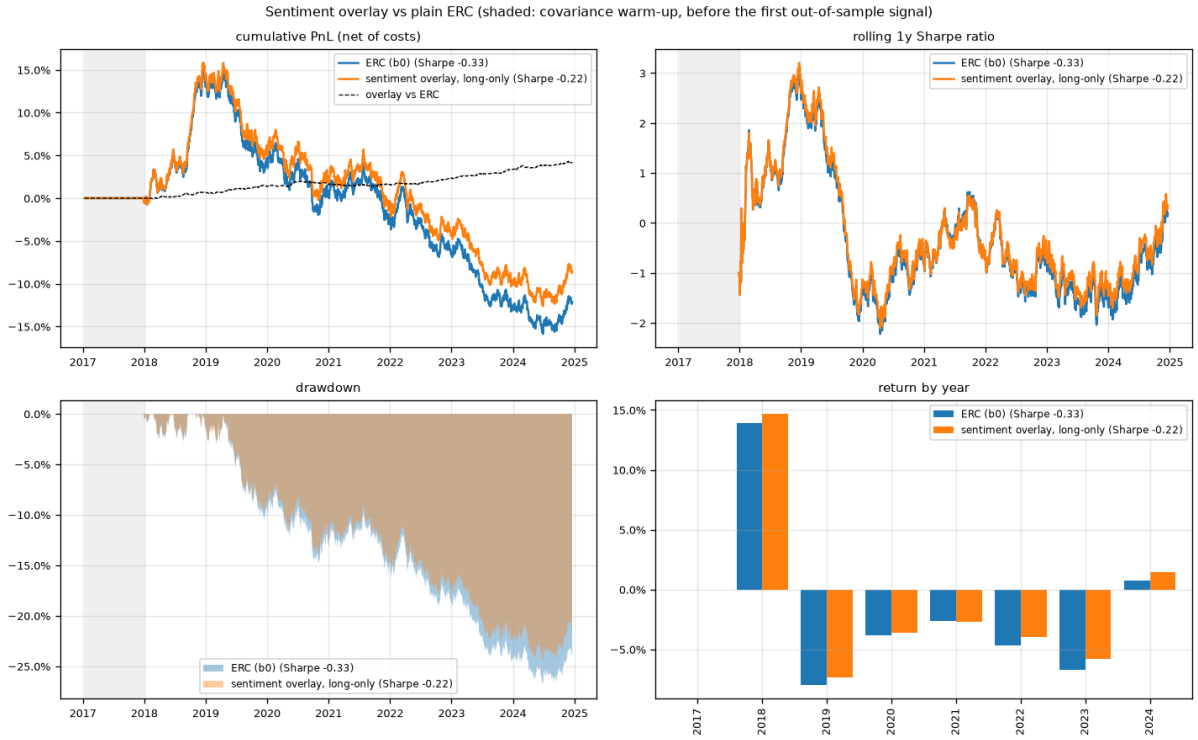


Figure 29.1: Long-only overlay against plain ERC (news IC 0.2): cumulative PnL net of costs, rolling one-year Sharpe ratio, drawdown, return by year. Shaded: the covariance warm-up before the first out-of-sample signal.

Does correct sentiment produce alpha? The sweep reruns the pipeline on the same market data for several news ICs. Overlay Sharpe ratio and excess return over ERC grow with the news

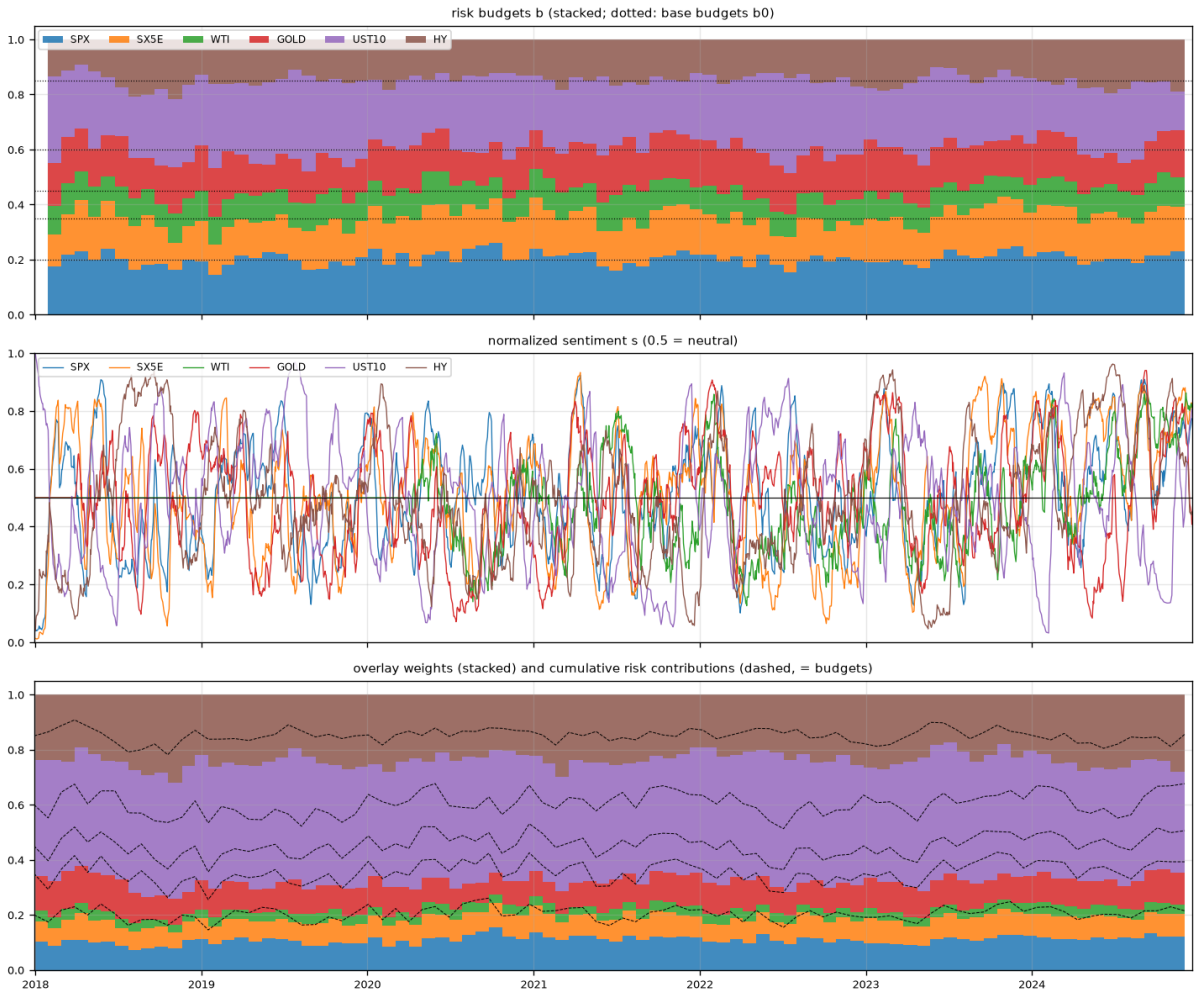


Figure 29.2: Long-only: modulated budgets against b^0, normalized sentiment, weights and cumulative risk contributions (which track the budgets).

Table 29.1: Long-only overlay against plain ERC (news IC 0.2).

metric	ERC (b^0)	sentiment overlay
annualized return	-0.0178	-0.0123
annualized volatility	0.0503	0.0506
sharpe ratio	-0.3313	-0.2181
max drawdown	0.2688	0.2455
annualized turnover	0.6737	1.0579
total costs	0.0024	0.0038

Table 29.2: Long-short overlay against plain ERC (news IC 0.2).

metric	ERC (b^0)	sentiment overlay
annualized return	-0.0178	0.0780
annualized volatility	0.0503	0.0485
sharpe ratio	-0.3313	1.5741
max drawdown	0.2688	0.0733
annualized turnover	0.6737	7.9842
total costs	0.0024	0.0288

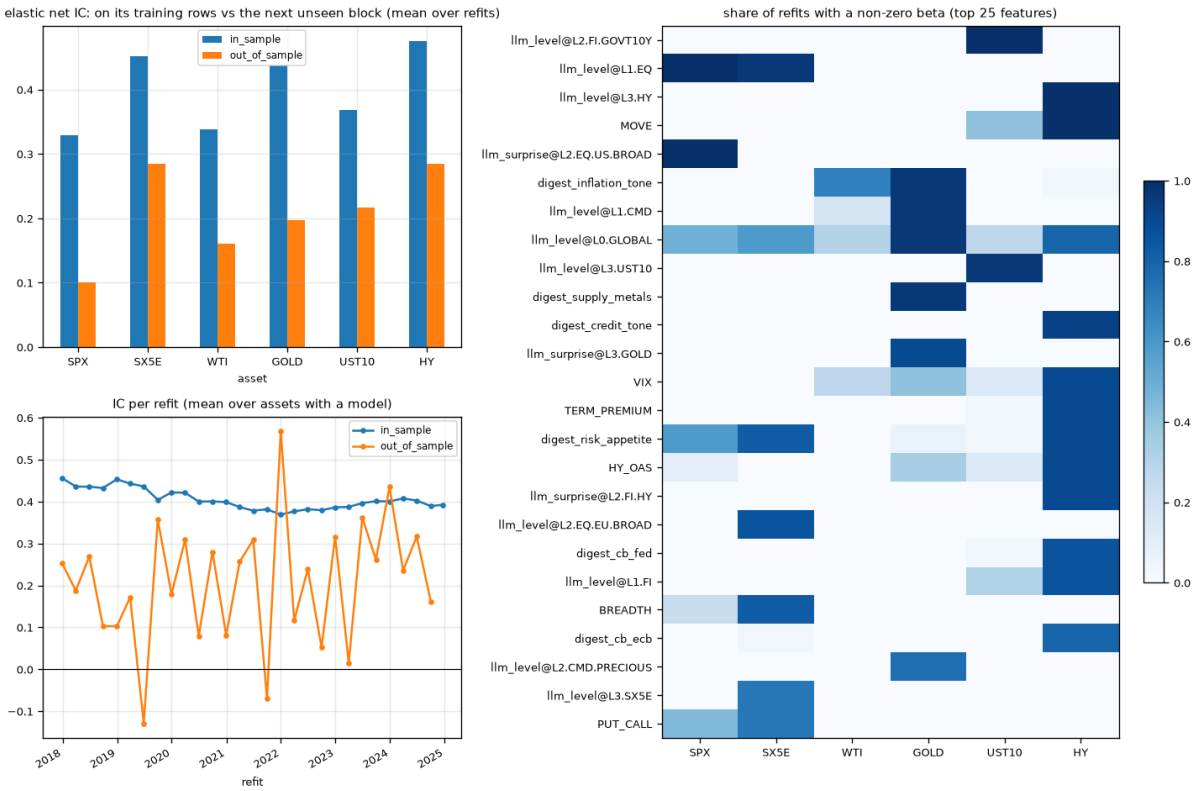


Figure 29.3: Elastic net diagnostics: in-sample against out-of-sample IC per asset and per refit, and the share of refits in which each feature has a non-zero coefficient.

IC; below an IC of about 0.1 the elastic net mostly cannot separate the news from noise, and the market indicators alone give a small edge.

Table 29.3: News-quality sweep, long-only (ERC Sharpe ratio for reference).

news IC	ERC Sharpe	overlay Sharpe	excess return	IC of S	IC of LLM level
0.0	-0.33	-0.30	+0.14%	0.12	-0.00
0.05	-0.33	-0.30	+0.14%	0.13	0.04
0.1	-0.33	-0.29	+0.19%	0.16	0.08
0.2	-0.33	-0.22	+0.55%	0.29	0.15
0.3	-0.33	-0.14	+0.94%	0.43	0.23
0.5	-0.33	-0.04	+1.45%	0.62	0.37

Reading the results. Long-only modulation can at most halve or increase by half a budget, capped at 25% per rebalance, so even good news moves the allocation modestly. Long-short acts on the direction of the view and captures much more of it, at the price of turnover (several times a year) and costs; with uninformative news it earns almost nothing, which also shows that no future information leaks through the pipeline. The ERC benchmark's negative return is a property of this seed's synthetic premiums; only the difference matters.

29.4 Running and extending

Chapter 1 lists the commands. The command line (`nabia_python/nabia/sentiment_rb/__main__.py:16--40`) runs one configuration or a sweep; figures are produced by `nabia_python/nabia/sentiment`

Table 29.4: News-quality sweep, long-short.

news IC	ERC Sharpe	overlay Sharpe	excess return	IC of S	IC of LLM level
0.0	-0.33	0.04	+1.87%	0.12	-0.00
0.05	-0.33	0.13	+2.26%	0.13	0.04
0.1	-0.33	0.60	+4.73%	0.16	0.08
0.2	-0.33	1.57	+9.58%	0.29	0.15
0.3	-0.33	2.69	+16.23%	0.43	0.23
0.5	-0.33	2.91	+18.08%	0.62	0.37

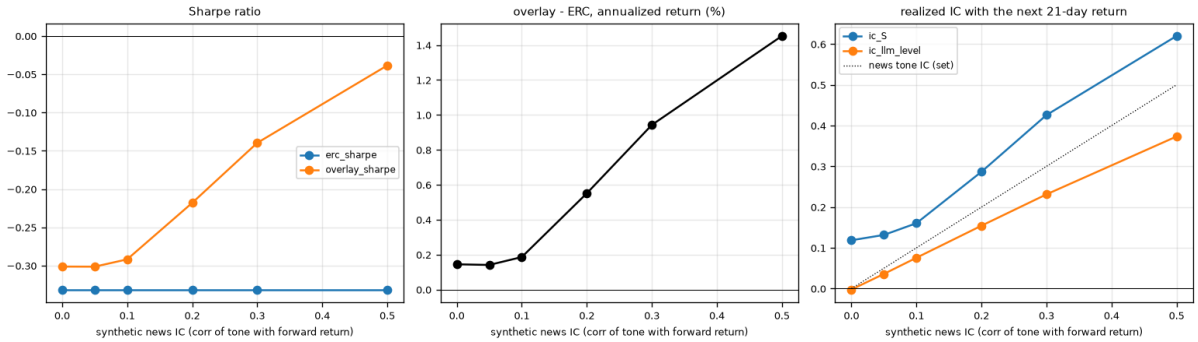


Figure 29.4: Long-only sweep: Sharpe ratio, excess return and realized IC against the news IC.

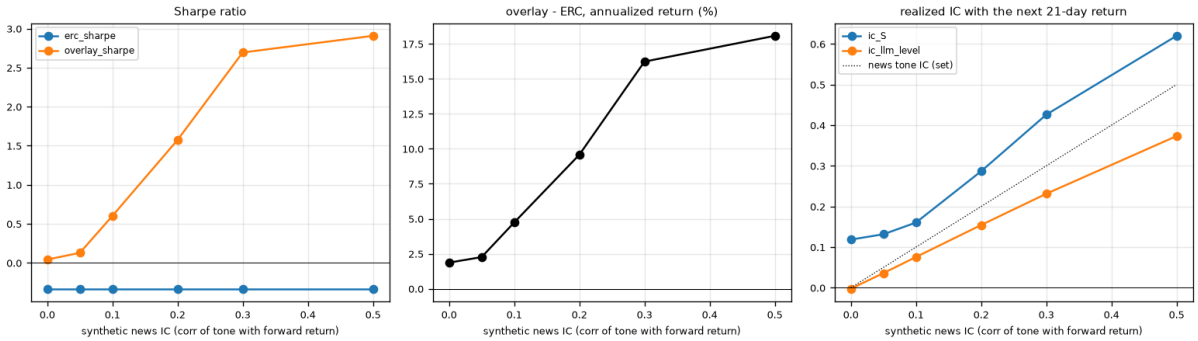


Figure 29.5: Long-short sweep.

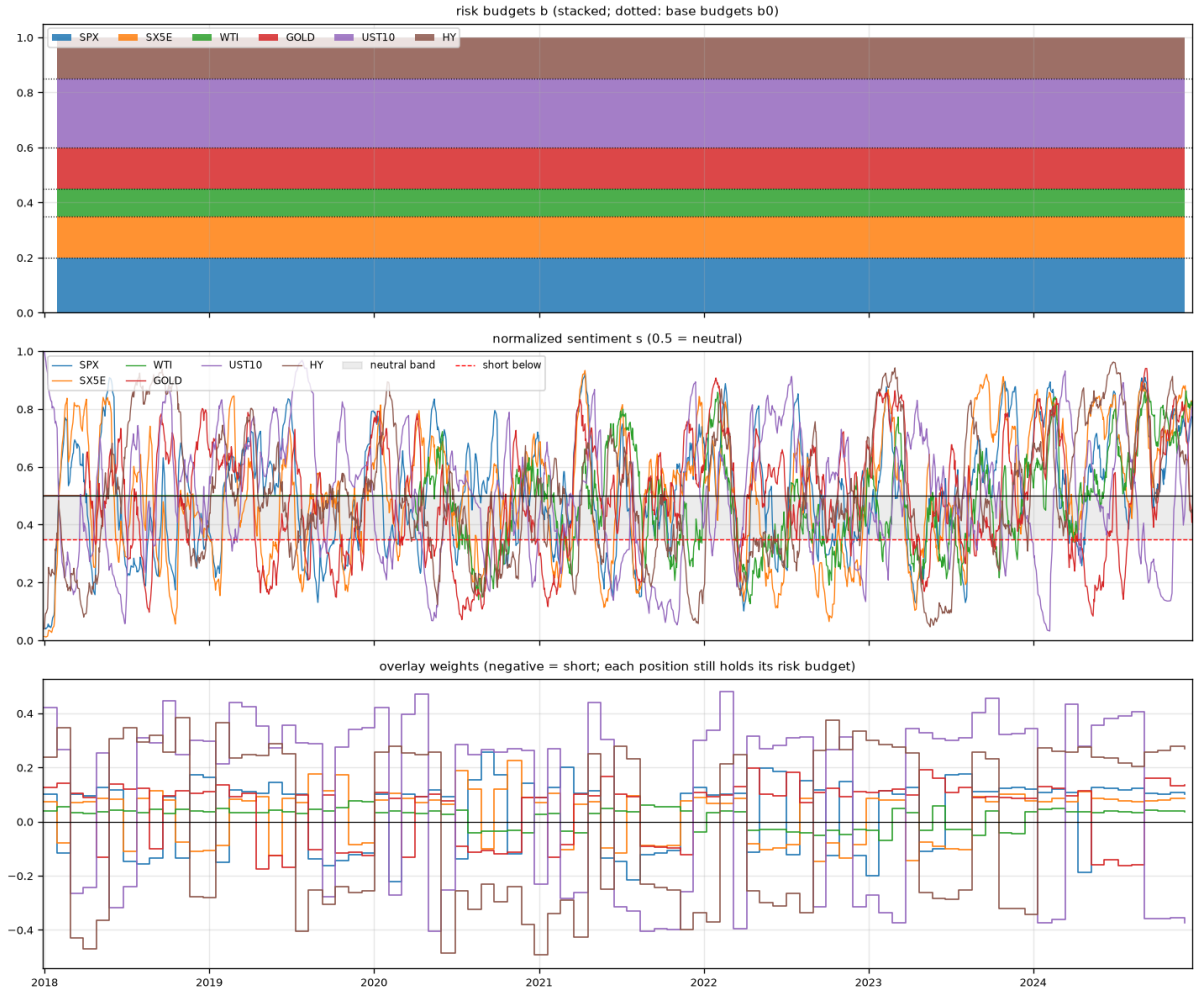


Figure 29.6: Long-short: budgets stay b^0; the sentiment panel shades the neutral band; negative weights are short positions.

`_rb/report/figures.py:18--85` and `nabia_python/nabia/sentiment_rb/report/figures.py:88--99`. To use real data, replace `synthetic_market` and `synthetic_news` (`nabia_python/nabia/sentiment_rb/data/synthetic.py:64--82`) by loaders returning the same frames: returns by asset, indicator levels by *as-of* date, and articles with publish and received timestamps; everything downstream is unchanged.

Part VIII

Verification and limitations

Chapter 30

Verification

C++ (Catch2) 104 test cases, plus an end-to-end run of the command-line application on its example (`ctest -preset debug`).

- Results against closed forms: inverse volatility for uncorrelated equal budgets, Gaussian VaR and ES, SES and Kalman steady states, filter gains at the band edges.
- Optimizer results against known optima, with constraints.
- Accounting identities of the portfolio engine to 10^{-15}.
- The elastic net against OLS, the ridge closed form and its optimality conditions; the regression engine recovering known factors (exhaustive, forward, in-sample BIC), and bootstrap intervals covering the true coefficients.
- No look-ahead, thread-count independence, invalid input; the application's JSON and CSV parsing and writing.

The suites run in debug, under AddressSanitizer with UndefinedBehaviorSanitizer, and the multithreaded code under ThreadSanitizer.

Bindings (pytest, `nabia_python/tests/bridge`) every module of `nabia_python_bridge` against closed forms and numpy, and the binding semantics (keyword parameters, Python objectives called from C++ threads, exceptions, kernel composition, frame alignment, budget schedules). The wheel scripts and cibuildwheel run them on every wheel they build, on every platform.

Python package (`nabia_python/tests`) `nabia.backtest` against the command-line application on its own example (a copy in `tests/data`), schedules and errors; the market data store, sources and incremental download against fakes; two opt-in tests (`NABIA_NETWORK_TESTS=1`) download live data.

Sentiment pipeline `tests/test_sentiment_rb.py`: budgets positive, normalized, neutral at $s = \frac{1}{2}$ and within the change cap; directions with hysteresis; shorted positions keeping their risk budget (Proposition 28.3.1); the simulator reproducing nabia's engine; every ancestor feature inherited exactly once. Each run also asserts: training rows purged by h, normalized scores in $[0, 1]$, risk contributions equal to the budgets at every rebalance ($< 10^{-4}$, observed $< 10^{-8}$), and, in long-only mode, the simulator matching the engine.

This document Its excerpts, code locations and file pieces are extracted at build time; a function that no longer exists stops the build.

Chapter 31

Limitations and future work

- **Gaussian risk.** VaR and ES are Gaussian; fat tails are not modelled. A historical or Cornish–Fisher measure would fit the same interface.
- **Factor regression** has constant coefficients, and its bootstrap is conditional on the selected model (Chapter 11). Exhaustive search grows as $\binom{N}{k}$.
- **SQP** solves its QP subproblems with the augmented Lagrangian to `qp_tolerance`; on some strongly convex mean-variance problems it was seen to stall at its iteration limit, so prefer the augmented Lagrangian there until the QP step is made exact (an active-set QP).
- **Wheels** for Linux, Intel macOS and Windows are built and tested in CI only; locally on macOS arm64.
- **Sentiment pipeline.**
  - The data and news are synthetic; the results measure how much of a *known* news edge survives the pipeline, not the value of real news. Real loaders replace two functions (Chapter 29).
  - Positivity is imposed by eliminating negative coefficients and refitting, not by a constrained solver; the selected set can differ from a non-negative elastic net's.
  - LLM scores are uncalibrated model judgements, and backtests before a model's training cutoff are optimistic. The live path (`-live`) is implemented but scores only the most recent days; Message Batches backfills are not implemented.
  - Long-short turnover is high (several times a year at news IC 0.2); a wider neutral band or higher costs show how much of the edge survives trading.
  - Market data caveats: continuous futures jump at rolls; asynchronous closes bias same-day correlations downwards.

Appendix A

Glossary

ABI application binary interface: how compiled code calls other compiled code. CPython's changes between minor versions.

auditwheel, delocate, delvewheel tools that copy shared libraries into a wheel and fix its tag (Linux, macOS, Windows).

install component a label on CMake install rules, to install a subset of them.

build backend / frontend the backend (scikit-build-core) turns a project into a wheel; the frontend (pip, `build`, `uv build`) calls it in an isolated environment.

deployment target the oldest macOS a binary supports.

editable install an install that points to the source folder instead of copying it.

glibc the GNU C library of Linux; `manylinux_2_28` means version 2.28 or newer.

sdist source distribution: the sources and `pyproject.toml`, compiled on install.

site-packages the folder of an environment where packages are installed.

wheel tag the `python-abi-platform` part of a wheel's name.

Bibliography

[1] D. P. Bertsekas, *Constrained Optimization and Lagrange Multiplier Methods*, Academic Press, 1982.

[2] E. G. Birgin and J. M. Martínez, *Practical Augmented Lagrangian Methods for Constrained Optimization*, SIAM, 2014.

[3] R. G. Brown, *Statistical Forecasting for Inventory Control*, McGraw-Hill, 1959.

[4] R. S. Bucy and P. D. Joseph, *Filtering for Stochastic Processes with Applications to Guidance*, Interscience, 1968.

[5] J. Durbin and S. J. Koopman, *Time Series Analysis by State Space Methods*, 2nd ed., Oxford University Press, 2012.

[6] E. S. Gardner, Exponential smoothing: the state of the art, *J. Forecasting* 4 (1985) 1–28.

[7] W. W. Hager and H. Zhang, A new conjugate gradient method with guaranteed descent and an efficient line search, *SIAM J. Optim.* 16 (2005) 170–192.

[8] S.-P. Han, A globally convergent method for nonlinear programming, *J. Optim. Theory Appl.* 22 (1977) 297–309.

[9] J. Hartikainen and S. Särkkä, Kalman filtering and smoothing solutions to temporal Gaussian process regression models, *IEEE MLSP* (2010) 379–384.

[10] A. C. Harvey, *Forecasting, Structural Time Series Models and the Kalman Filter*, Cambridge University Press, 1989.

[11] M. R. Hestenes, Multiplier and gradient methods, *J. Optim. Theory Appl.* 4 (1969) 303–320.

[12] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed., Cambridge University Press, 2013.

[13] R. J. Hyndman, A. B. Koehler, J. K. Ord and R. D. Snyder, *Forecasting with Exponential Smoothing: The State Space Approach*, Springer, 2008.

[14] R. E. Kalman, A new approach to linear filtering and prediction problems, *J. Basic Engineering* 82 (1960) 35–45.

[15] D. C. Liu and J. Nocedal, On the limited memory BFGS method for large scale optimization, *Math. Programming* 45 (1989) 503–528.

[16] D. J. C. MacKay, Introduction to Gaussian processes, in C. M. Bishop (ed.), *Neural Networks and Machine Learning*, Springer, 1998, 133–165.

[17] J. F. Muth, Optimal properties of exponentially weighted forecasts, *J. Amer. Statist. Assoc.* 55 (1960) 299–306.

[18] J. Nocedal, Updating quasi-Newton matrices with limited storage, *Math. Comp.* 35 (1980) 773–782.

[19] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006, Chapters 7 and 17.

[20] M. J. D. Powell, A method for nonlinear constraints in minimization problems, in R. Fletcher (ed.), *Optimization*, Academic Press, 1969, 283–298.

[21] M. J. D. Powell, A fast algorithm for nonlinearly constrained optimization calculations, in G. A. Watson (ed.), *Numerical Analysis*, Lecture Notes in Mathematics 630, Springer, 1978, 144–157.

[22] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006.

[23] S. Roberts, M. Osborne, M. Ebden, S. Reece, N. Gibson and S. Aigrain, Gaussian processes for time-series modelling, *Phil. Trans. R. Soc. A* 371 (2013) 20110550.

[24] R. T. Rockafellar, A dual approach to solving nonlinear programming problems by unconstrained optimization, *Math. Programming* 5 (1973) 354–373.

[25] R. T. Rockafellar, Augmented Lagrangians and applications of the proximal point algorithm in convex programming, *Math. Oper. Res.* 1 (1976) 97–116.

[26] H. E. Rauch, F. Tung and C. T. Striebel, Maximum likelihood estimates of linear dynamic systems, *AIAA Journal* 3 (1965) 1445–1450.